OFFICIAL SYLLABUS V4.0.1 • VIETNAMESE STUDY EDITION

ISTQB® Certified Tester Foundation Level (CTFL v4.0.1)

Bản Dịch Tiếng Việt Chuẩn Thuật Ngữ — Đầy Đủ 100% Nội Dung 6 Chương Thi

PHIÊN BẢN GỐC	CẤU TRÚC BÀI THI	ĐIỂM ĐẬU CHUẨN	THỜI GIAN LÀM BÀI
v4.0.1 (15/09/2024)	**40 câu Trắc nghiệm**	**≥ 26 / 40 câu (65%)**	**60 phút (+15p Non-native)**

📊 Phân Bổ Câu Hỏi & Thời Lượng Học Theo Từng Chương (Chuẩn Đề Thi CTFL v4.0)

Chương	Chủ đề (Topic)	Thời lượng chuẩn	Số câu hỏi thi	Tỷ trọng	Mức K3 (Bài tập)
Chương 1	Cơ bản về Kiểm thử *(Fundamentals of Testing)*	180 phút	**8 câu**	20%	Không (Chỉ K1, K2)
Chương 2	Kiểm thử xuyên suốt SDLC *(Testing Throughout the SDLC)*	130 phút	**6 câu**	15%	Không (Chỉ K1, K2)
Chương 3	Kiểm thử Tĩnh *(Static Testing)*	80 phút	**4 câu**	10%	Không (Chỉ K1, K2)
Chương 4	Phân tích và Thiết kế Kiểm thử *(Test Analysis and Design)*	390 phút	**11 câu**	27.5%	**Có (EP, BVA, Decision Table, State, ATDD)**
Chương 5	Quản lý Hoạt động Kiểm thử *(Managing the Test Activities)*	335 phút	**9 câu**	22.5%	**Có (Ước lượng, Ưu tiên Test, Báo cáo lỗi)**
Chương 6	Công cụ Kiểm thử *(Test Tools)*	20 phút	**2 câu**	5%	Không (Chỉ K1, K2)
TỔNG CỘNG TOÀN ĐỀ CƯƠNG		**1.135 phút (18h55p)**	**40 câu**	**100%**	**8 câu K1 • 24 câu K2 • 8 câu K3**

CHƯƠNG 1

🕑 Thời lượng phân bổ: 180 phút

Cơ bản về Kiểm thử (Fundamentals of Testing)

K1 Bảng Từ Khóa Chuẩn ISTQB (Keywords) — Cần nhớ chính xác thuật ngữ gốc Tiếng Anh khi làm bài thi

coverage	độ bao phủ	**debugging**	gỡ lỗi
defect	khiếm khuyết / lỗi trong sản phẩm	**error**	sai sót của con người
failure	hỏng hóc / sự cố khi chạy	**quality**	chất lượng
quality assurance	đảm bảo chất lượng - QA	**root cause**	nguyên nhân gốc rễ
test analysis	phân tích kiểm thử	**test basis**	cơ sở kiểm thử
test case	ca kiểm thử	**test completion**	hoàn tất kiểm thử
test condition	điều kiện kiểm thử	**test control**	kiểm soát kiểm thử
test data	dữ liệu kiểm thử	**test design**	thiết kế kiểm thử
test execution	thực thi kiểm thử	**test implementation**	triển khai kiểm thử
test monitoring	giám sát kiểm thử	**test object**	đối tượng kiểm thử
test objective	mục tiêu kiểm thử	**test planning**	lập kế hoạch kiểm thử
test procedure	quy trình/thủ tục kiểm thử	**test process**	tiến trình kiểm thử
test result	kết quả kiểm thử	**testing**	kiểm thử
testware	sản phẩm/tài nguyên kiểm thử	**traceability**	khả năng truy vết
validation	thẩm định	**verification**	xác minh

Mục Tiêu Học Tập Trọng Tâm (Learning Objectives)

MÃ LO	MỨC NHẬN THỨC	NỘI DUNG YÊU CẦU TRONG ĐỀ THI
1.1 Kiểm thử là gì? (What is Testing?)		
FL-1.1.1	K1 · Ghi nhớ	Nhận diện các mục tiêu kiểm thử điển hình (*Identify typical test objectives*)
FL-1.1.2	K2 · Hiểu	Phân biệt kiểm thử với gỡ lỗi (*Differentiate testing from debugging*)
1.2 Tại sao Kiểm thử là cần thiết? (Why is Testing Necessary?)		
FL-1.2.1	K2 · Hiểu	Cho ví dụ về lý do tại sao kiểm thử là cần thiết (*Exemplify why testing is necessary*)

MÃ LO	MỨC NHẬN THỨC	NỘI DUNG YÊU CẦU TRONG ĐỀ THI
FL-1.2.2	K1 · Ghi nhớ	Nhớ lại mối quan hệ giữa kiểm thử và đảm bảo chất lượng (***Recall the relation between testing and quality assurance***)
FL-1.2.3	K2 · Hiểu	Phân biệt giữa nguyên nhân gốc rễ, sai sót, khiếm khuyết và hỏng hóc (***Distinguish between root cause, error, defect, and failure***)
1.3 Các nguyên lý Kiểm thử (Testing Principles)		
FL-1.3.1	K2 · Hiểu	Giải thích 7 nguyên lý kiểm thử (***Explain the seven testing principles***)
1.4 Các hoạt động Kiểm thử, Testware và các Vai trò trong Kiểm thử (Test Activities, Testware and Test Roles)		
FL-1.4.1	K2 · Hiểu	Giải thích các hoạt động kiểm thử khác nhau và các công việc liên quan (***Explain the different test activities and related tasks***)
FL-1.4.2	K2 · Hiểu	Giải thích tác động của ngữ cảnh lên tiến trình kiểm thử (***Explain the impact of context on the test process***)
FL-1.4.3	K2 · Hiểu	Phân biệt các sản phẩm kiểm thử (***testware***) hỗ trợ cho các hoạt động kiểm thử (***Differentiate the testware that supports the test activities***)
FL-1.4.4	K2 · Hiểu	Giải thích giá trị của việc duy trì khả năng truy vết (***Explain the value of maintaining traceability***)
FL-1.4.5	K2 · Hiểu	So sánh các vai trò khác nhau trong kiểm thử (***Compare the different roles in testing***)
1.5 Kỹ năng thiết yếu và Thực hành tốt trong Kiểm thử (Essential Skills and Good Practices in Testing)		
FL-1.5.1	K2 · Hiểu	Đưa ra ví dụ về các kỹ năng chung cần thiết cho kiểm thử (***Give examples of the generic skills required for testing***)
FL-1.5.2	K1 · Ghi nhớ	Nhớ lại các ưu điểm của cách tiếp cận toàn nhóm (***Recall the advantages of the whole team approach***)
FL-1.5.3	K2 · Hiểu	Phân biệt lợi ích và hạn chế của tính độc lập trong kiểm thử (***Distinguish the benefits and drawbacks of independence of testing***)

1.1. Kiểm thử là gì? (What is Testing?)

Các hệ thống phần mềm là một phần không thể thiếu trong cuộc sống hàng ngày của chúng ta. Hầu hết mọi người đều từng trải nghiệm phần mềm hoạt động không như mong đợi. Phần mềm hoạt động không đúng có thể dẫn đến nhiều vấn đề, bao gồm mất tiền bạc, thời gian hoặc uy tín kinh doanh, và trong những trường hợp cực đoan, thậm chí gây thương tích hoặc tử vong. Kiểm thử phần mềm (*Software testing*) đánh giá chất lượng phần mềm (*software quality*) và giúp giảm rủi ro xảy ra hỏng hóc phần mềm (*software failure*) trong quá trình vận hành.

Kiểm thử phần mềm là một tập hợp các hoạt động nhằm phát hiện các khiếm khuyết (*defects*) và đánh giá chất lượng của các sản phẩm công việc phần mềm (*software work products*). Các sản phẩm công việc này, khi được kiểm thử, được gọi là các **đối tượng kiểm thử** ***(test objects)***. Một quan niệm sai lầm phổ biến về kiểm thử là nó chỉ bao gồm việc thực thi các bài kiểm thử (*executing tests* - tức là chạy phần mềm và kiểm tra kết quả kiểm thử -

test results). Tuy nhiên, kiểm thử phần mềm còn bao gồm các hoạt động khác và phải được căn chỉnh phù hợp với vòng đời phát triển phần mềm (*software development lifecycle - SDLC*, xem Chương 2).

Một quan niệm sai lầm phổ biến khác về kiểm thử là kiểm thử chỉ tập trung hoàn toàn vào việc **xác minh** ***(verifying)*** đối tượng kiểm thử. Mặc dù kiểm thử bao gồm **xác minh** ***(verification)***, tức là kiểm tra xem hệ thống có đáp ứng các yêu cầu đã được quy định (*specified requirements*) hay không, nhưng nó cũng bao gồm **thẩm định** ***(validation)***, nghĩa là kiểm tra xem hệ thống có đáp ứng nhu cầu của người dùng và các bên liên quan khác (*stakeholders*) trong môi trường vận hành thực tế của nó hay không.

Kiểm thử có thể là **động** ***(dynamic)*** hoặc **tĩnh** ***(static)***:

- **Kiểm thử động** ***(Dynamic testing)*** bao gồm việc thực thi phần mềm (*execution of software*), trong khi **kiểm thử tĩnh** ***(static testing)*** thì không.
- Kiểm thử tĩnh bao gồm các hoạt động duyệt/đánh giá (***reviews***, xem Chương 3) và phân tích tĩnh ***(static analysis)***.
- Kiểm thử động sử dụng các loại kỹ thuật kiểm thử ***(test techniques)*** và phương pháp tiếp cận kiểm thử khác nhau để xây dựng các ca kiểm thử (***test cases***, xem Chương 4).

Kiểm thử không chỉ là một hoạt động kỹ thuật. Nó còn cần được lập kế hoạch, quản lý, ước lượng, giám sát và kiểm soát một cách hợp lý (xem Chương 5).

Kiểm thử viên ***(Testers)*** sử dụng các công cụ (xem Chương 6), nhưng điều quan trọng cần nhớ là kiểm thử phần lớn là một hoạt động trí tuệ, đòi hỏi kiểm thử viên phải có kiến thức chuyên môn, sử dụng kỹ năng phân tích và áp dụng tư duy phản biện ***(critical thinking)*** cũng như tư duy hệ thống ***(systems thinking)*** (Myers 2011, Roman 2018).

Tiêu chuẩn ISO/IEC/IEEE 29119-1 cung cấp thêm thông tin về các khái niệm kiểm thử phần mềm.

1.1.1. Mục tiêu Kiểm thử (Test Objectives) `FL-1.1.1 • K1`

Các mục tiêu kiểm thử ***(test objectives)*** điển hình là:

- Đánh giá các sản phẩm công việc ***(evaluating work products)*** như các yêu cầu ***(requirements)***, câu chuyện người dùng ***(user stories)***, thiết kế ***(designs)***, và mã nguồn ***(code)***.
- Gây ra các hỏng hóc ***(causing failures)*** và tìm ra các khiếm khuyết ***(finding defects)***.
- Đảm bảo độ bao phủ yêu cầu ***(required coverage)*** của một đối tượng kiểm thử ***(test object)***.
- Giảm mức độ rủi ro ***(risk level)*** về chất lượng phần mềm không đạt yêu cầu.
- Xác minh ***(verifying)*** xem các yêu cầu đã quy định có được đáp ứng hay chưa.
- Xác minh rằng đối tượng kiểm thử tuân thủ các yêu cầu về hợp đồng ***(contractual)***, pháp lý ***(legal)***, và quy định chế tài ***(regulatory)***.
- Cung cấp thông tin cho các bên liên quan ***(stakeholders)*** để cho phép họ đưa ra các quyết định sáng suốt ***(informed decisions)***.
- Xây dựng sự tự tin/tin tưởng ***(building confidence)*** vào chất lượng của đối tượng kiểm thử.
- Thẩm định ***(validating)*** xem đối tượng kiểm thử đã hoàn chỉnh và hoạt động đúng như mong đợi của các bên liên quan hay chưa.

Mục tiêu kiểm thử có thể thay đổi tùy thuộc vào ngữ cảnh *(context)*, bao gồm sản phẩm công việc đang được kiểm thử, cấp độ kiểm thử *(test level)*, rủi ro *(risks)*, vòng đời phát triển phần mềm *(SDLC)* đang được tuân theo, và các yếu tố liên quan đến bối cảnh kinh doanh (ví dụ: cơ cấu công ty, các cân nhắc cạnh tranh, hoặc thời gian đưa sản phẩm ra thị trường - ***time to market***).

1.1.2. Kiểm thử và Gỡ lỗi (Testing and Debugging) FL-1.1.2 • K2

Kiểm thử *(testing)* và gỡ lỗi *(debugging)* là hai hoạt động tách biệt nhau. Kiểm thử có thể kích hoạt các hỏng hóc *(trigger failures)* gây ra bởi các khiếm khuyết *(defects)* trong phần mềm (kiểm thử động - ***dynamic testing***) hoặc có thể trực tiếp tìm ra các khiếm khuyết trong đối tượng kiểm thử (kiểm thử tĩnh - ***static testing***).

Khi kiểm thử động (xem Chương 4) kích hoạt một hỏng hóc *(failure)*, gỡ lỗi *(debugging)* liên quan đến việc tìm ra nguyên nhân của hỏng hóc này *(defects)*, phân tích các nguyên nhân đó, và loại bỏ chúng. Quy trình gỡ lỗi điển hình trong trường hợp này bao gồm:

- Tái hiện lại một hỏng hóc *(Reproduction of a failure)*
- Chẩn đoán (***Diagnosis*** - tìm ra khiếm khuyết/***finding the defect***)
- Sửa khiếm khuyết *(Fixing the defect)*

Kiểm thử xác nhận *(Confirmation testing)* tiếp theo sẽ kiểm tra xem các bản sửa lỗi có giải quyết được vấn đề hay không. Tốt nhất, kiểm thử xác nhận nên được thực hiện bởi cùng một người đã thực hiện bài kiểm thử ban đầu. **Kiểm thử hồi quy** *(Regression testing)* sau đó cũng có thể được thực hiện, để kiểm tra xem các bản sửa lỗi có gây ra hỏng hóc ở các phần khác của đối tượng kiểm thử hay không (xem mục 2.2.3 để biết thêm thông tin về kiểm thử xác nhận và kiểm thử hồi quy).

Khi kiểm thử tĩnh xác định được một khiếm khuyết *(defect)*, việc gỡ lỗi chỉ liên quan đến việc loại bỏ nó. Không cần phải tái hiện *(reproduction)* hay chẩn đoán *(diagnosis)*, vì kiểm thử tĩnh trực tiếp tìm thấy các khiếm khuyết và không thể gây ra các hỏng hóc *(cannot cause failures)* (xem Chương 3).

1.2. Tại sao Kiểm thử là cần thiết? (Why is Testing Necessary?)

Kiểm thử, với tư cách là một hình thức kiểm soát chất lượng *(quality control)*, giúp đạt được các mục tiêu kiểm thử đã thỏa thuận trong phạm vi các ràng buộc về phạm vi *(scope)*, thời gian *(time)*, chất lượng *(quality)* và ngân sách *(budget)* đã đặt ra. Đóng góp của kiểm thử vào sự thành công không nên chỉ giới hạn ở các hoạt động của nhóm kiểm thử. Bất kỳ bên liên quan *(stakeholder)* nào cũng có thể sử dụng kỹ năng kiểm thử của mình để đưa dự án tiến gần hơn đến thành công. Việc kiểm thử các thành phần *(components)*, hệ thống *(systems)* và các sản phẩm công việc liên quan (ví dụ: tài liệu) giúp xác định các khiếm khuyết trong phần mềm.

1.2.1. Đóng góp của Kiểm thử vào Sự thành công (Testing's Contributions to Success) FL-1.2.1 • K2

- Kiểm thử cung cấp một phương tiện hiệu quả về mặt chi phí *(cost-effective means)* để phát hiện các khiếm khuyết *(defects)*. Các khiếm khuyết này sau đó có thể được loại bỏ (bằng việc gỡ lỗi - ***debugging*** – một hoạt động không phải là kiểm thử), do đó kiểm thử **gián tiếp** *(indirectly)* đóng góp vào việc tạo ra các đối tượng kiểm thử có chất lượng cao hơn.
- Kiểm thử cung cấp một phương tiện để **trực tiếp** *(directly)* đánh giá chất lượng của một đối tượng kiểm thử ở các giai đoạn *(phases)* khác nhau trong SDLC. Các thước đo này được sử dụng như một phần của hoạt động

quản lý dự án lớn hơn, góp phần vào các quyết định chuyển sang giai đoạn tiếp theo của SDLC, chẳng hạn như quyết định phát hành *(release decision)*.

- Kiểm thử cung cấp cho người dùng sự đại diện gián tiếp *(indirect representation)* trong dự án phát triển. Kiểm thử viên đảm bảo rằng sự hiểu biết của họ về nhu cầu của người dùng được xem xét xuyên suốt vòng đời phát triển. Giải pháp thay thế là lôi kéo một nhóm người dùng đại diện tham gia như một phần của dự án phát triển, điều này thường không khả thi do chi phí cao và sự thiếu sẵn sàng của những người dùng phù hợp.
- Kiểm thử cũng có thể được yêu cầu để đáp ứng các yêu cầu hợp đồng *(contractual)* hoặc pháp lý *(legal)*, hoặc để tuân thủ các tiêu chuẩn quy định chế tài *(regulatory standards)*.

1.2.2. Kiểm thử và Đảm bảo Chất lượng (Testing and Quality Assurance - QA) FL-1.2.2 • K1

Mặc dù mọi người thường sử dụng thuật ngữ "kiểm thử" *(testing)* và "đảm bảo chất lượng" *(quality assurance - QA)* thay thế cho nhau, nhưng kiểm thử và QA **không giống nhau**.

- **Kiểm thử** *(Testing)* là một cách tiếp cận **hướng sản phẩm** *(product-oriented)*, **mang tính khắc phục** *(corrective approach)*, tập trung vào các hoạt động hỗ trợ đạt được các mức chất lượng phù hợp. Kiểm thử là một hình thức chính của **kiểm soát chất lượng** *(quality control - QC)*, trong khi các hình thức QC khác bao gồm các phương pháp hình thức (*formal methods* - kiểm tra mô hình *model checking* và chứng minh tính đúng đắn *proof of correctness*), mô phỏng *(simulation)* và tạo mẫu *(prototyping)*.
- **QA** *(Quality Assurance)* là một cách tiếp cận **hướng quy trình** *(process-oriented)*, **mang tính phòng ngừa** *(preventive approach)*, tập trung vào việc triển khai và cải tiến các quy trình *(processes)*. Nó hoạt động trên cơ sở rằng nếu một quy trình tốt được tuân thủ một cách chính xác, thì nó sẽ tạo ra một sản phẩm tốt. QA áp dụng cho cả quy trình phát triển và quy trình kiểm thử, và là trách nhiệm của tất cả mọi người trong một dự án.

Kết quả kiểm thử *(Test results)* được sử dụng bởi cả QA và kiểm thử *(testing)*. Trong kiểm thử, chúng được dùng để sửa các khiếm khuyết *(fix defects)*, trong khi trong QA, chúng cung cấp phản hồi *(feedback)* về mức độ hiệu quả mà các quy trình phát triển và kiểm thử đang hoạt động.

1.2.3. Sai sót, Khiếm khuyết, Hỏng hóc và Nguyên nhân gốc rễ (Errors, Defects, Failures, and Root Causes) FL-1.2.3 • K2

Con người tạo ra các **sai sót (*errors* / *mistakes*)**, từ đó sinh ra các **khiếm khuyết (*defects* / *faults* / *bugs*)**, và đến lượt chúng có thể dẫn đến các **hỏng hóc** *(failures)*. Con người mắc sai sót vì nhiều lý do khác nhau, chẳng hạn như áp lực thời gian, độ phức tạp của sản phẩm công việc, quy trình, cơ sở hạ tầng hoặc các tương tác, hoặc đơn giản vì họ mệt mỏi hoặc thiếu được đào tạo đầy đủ.

Khiếm khuyết *(Defects)* có thể được tìm thấy trong tài liệu, chẳng hạn như đặc tả yêu cầu *(requirements specification)* hoặc kịch bản kiểm thử *(test script)*, trong mã nguồn *(source code)*, hoặc trong một sản phẩm công việc hỗ trợ như tệp dựng *(build file)*. Các khiếm khuyết trong các sản phẩm công việc được tạo ra sớm hơn trong SDLC, nếu không được phát hiện, thường dẫn đến các sản phẩm công việc bị khiếm khuyết ở giai đoạn sau của vòng đời. Nếu một khiếm khuyết trong mã nguồn được thực thi *(executed)*, hệ thống có thể không làm được những gì nó phải làm, hoặc làm điều gì đó mà nó không nên làm, gây ra một **hỏng hóc** *(failure)*. Một số khiếm khuyết sẽ luôn dẫn đến hỏng hóc nếu được thực thi, trong khi những khiếm khuyết khác chỉ dẫn đến hỏng hóc trong những hoàn cảnh cụ thể, và một số có thể không bao giờ dẫn đến hỏng hóc.

Sai sót *(Errors)* và khiếm khuyết *(defects)* không phải là nguyên nhân duy nhất gây ra hỏng hóc *(failures)*. Hỏng hóc cũng có thể do các điều kiện môi trường *(environmental conditions)* gây ra, chẳng hạn như khi bức xạ *(radiation)* hoặc trường điện từ *(electromagnetic fields)* gây ra khiếm khuyết trong phần sụn *(firmware)*.

Một **nguyên nhân gốc rễ** *(root cause)* là một lý do cơ bản dẫn đến sự xuất hiện của một vấn đề (ví dụ: một tình huống dẫn đến một sai sót *error*). Nguyên nhân gốc rễ được xác định thông qua phân tích nguyên nhân gốc rễ *(root cause analysis)*, thường được thực hiện khi một hỏng hóc xảy ra hoặc một khiếm khuyết được xác định. Người ta tin rằng các hỏng hóc hoặc khiếm khuyết tương tự trong tương lai có thể được ngăn chặn hoặc giảm tần suất xuất hiện bằng cách giải quyết nguyên nhân gốc rễ, chẳng hạn như loại bỏ nó.

1.3. Các nguyên lý Kiểm thử (Testing Principles)

FL-1.3.1 • K2

Một số nguyên lý kiểm thử đưa ra các hướng dẫn chung áp dụng cho mọi hoạt động kiểm thử đã được đề xuất qua nhiều năm. Đề cương này mô tả 7 nguyên lý như vậy:

1. **Kiểm thử cho thấy sự hiện diện, chứ không phải sự vắng mặt của khiếm khuyết** *(Testing shows the presence, not the absence of defects)***:** Kiểm thử có thể chỉ ra rằng các khiếm khuyết đang tồn tại trong đối tượng kiểm thử, nhưng không thể chứng minh rằng không có khiếm khuyết nào (Buxton 1970). Kiểm thử làm giảm xác suất các khiếm khuyết còn sót lại chưa được khám phá trong đối tượng kiểm thử, nhưng ngay cả khi không tìm thấy khiếm khuyết nào, kiểm thử cũng không thể chứng minh tính đúng đắn *(correctness)* của đối tượng kiểm thử.
2. **Kiểm thử vét cạn là bất khả thi** *(Exhaustive testing is impossible)***:** Việc kiểm thử mọi thứ là không khả thi ngoại trừ những trường hợp tầm thường/đơn giản *(trivial cases)* (Manna 1978). Thay vì cố gắng kiểm thử vét cạn, các kỹ thuật kiểm thử (*test techniques*, xem Chương 4), sắp xếp độ ưu tiên ca kiểm thử (*test case prioritization*, xem mục 5.1.5), và kiểm thử dựa trên rủi ro (*risk-based testing*, xem mục 5.2), nên được sử dụng để tập trung nỗ lực kiểm thử.
3. **Kiểm thử sớm tiết kiệm thời gian và tiền bạc** *(Early testing saves time and money)***:** Các khiếm khuyết được loại bỏ sớm trong quy trình sẽ không gây ra các khiếm khuyết tiếp theo trong các sản phẩm công việc phái sinh *(derived work products)*. Chi phí chất lượng *(cost of quality)* sẽ giảm xuống vì sẽ có ít hỏng hóc xảy ra hơn ở giai đoạn sau của SDLC (Boehm 1981). Để tìm ra khiếm khuyết sớm, cả kiểm thử tĩnh (*static testing*, xem Chương 3) và kiểm thử động (*dynamic testing*, xem Chương 4) đều nên được bắt đầu càng sớm càng tốt.
4. **Các khiếm khuyết thường tập trung lại với nhau** *(Defects cluster together)***:** Một số lượng nhỏ các thành phần hệ thống *(system components)* thường chứa hầu hết các khiếm khuyết được phát hiện hoặc chịu trách nhiệm cho hầu hết các hỏng hóc trong vận hành *(operational failures)* (Enders 1975). Hiện tượng này là một minh họa của nguyên lý Pareto (*Pareto principle* - quy tắc 80/20). Các cụm khiếm khuyết được dự đoán *(predicted defect clusters)* và các cụm khiếm khuyết thực tế *(actual defect clusters)* quan sát được trong quá trình kiểm thử hoặc vận hành là một đầu vào quan trọng cho kiểm thử dựa trên rủi ro (*risk-based testing*, xem mục 5.2).
5. **Bài kiểm thử bị hao mòn / giảm hiệu quả (*Tests wear out* - trước đây gọi là Pesticide Paradox):** Nếu cùng một tập các bài kiểm thử được lặp đi lặp lại nhiều lần, chúng sẽ ngày càng trở nên kém hiệu quả trong việc phát hiện các khiếm khuyết mới (Beizer 1990). Để khắc phục hiệu ứng này, các bài kiểm thử và dữ liệu kiểm thử hiện có có thể cần được sửa đổi, và các bài kiểm thử mới có thể cần được viết thêm. Tuy nhiên, trong một số trường hợp, việc lặp lại cùng các bài kiểm thử có thể mang lại kết quả có lợi, ví dụ: trong kiểm thử hồi quy tự động (*automated regression testing*, xem mục 2.2.3).

6. **Kiểm thử phụ thuộc vào ngữ cảnh *(Testing is context dependent)*:** Không có một cách tiếp cận kiểm thử đơn lẻ nào có thể áp dụng chung cho mọi trường hợp. Kiểm thử được thực hiện khác nhau trong các ngữ cảnh khác nhau (Kaner 2011).
7. **Ngụy biện về sự không có khiếm khuyết *(Absence-of-defects fallacy)*:** Thật là một ngụy biện (tức là một quan niệm sai lầm) khi kỳ vọng rằng việc xác minh phần mềm *(software verification)* sẽ đảm bảo sự thành công của một hệ thống. Việc kiểm thử kỹ lưỡng tất cả các yêu cầu đã quy định và sửa tất cả các khiếm khuyết tìm thấy vẫn có thể tạo ra một hệ thống không đáp ứng được nhu cầu và kỳ vọng của người dùng, không giúp đạt được mục tiêu kinh doanh của khách hàng, và kém cỏi hơn so với các hệ thống cạnh tranh khác. Ngoài việc xác minh *(verification)*, việc thẩm định *(validation)* cũng phải được thực hiện (Boehm 1981).

1.4. Các hoạt động Kiểm thử, Testware và Vai trò trong Kiểm thử (Test Activities, Testware and Test Roles)

Kiểm thử phụ thuộc vào ngữ cảnh, nhưng ở mức độ tổng quát *(high level)*, có những tập hợp hoạt động kiểm thử chung mà nếu thiếu chúng thì kiểm thử ít có khả năng đạt được các mục tiêu kiểm thử. Các tập hợp hoạt động kiểm thử này tạo thành một **tiến trình kiểm thử** *(test process)*. Tiến trình kiểm thử có thể được điều chỉnh *(tailored)* cho phù hợp với một tình huống cụ thể dựa trên nhiều yếu tố khác nhau. Những hoạt động kiểm thử nào được đưa vào tiến trình kiểm thử này, cách chúng được triển khai, và thời điểm chúng diễn ra thường được quyết định như một phần của việc lập kế hoạch kiểm thử cho tình huống cụ thể đó (xem mục 5.1).

Các phần tiếp theo mô tả các khía cạnh chung của tiến trình kiểm thử này xét theo các hoạt động và nhiệm vụ kiểm thử, tác động của ngữ cảnh, sản phẩm kiểm thử *(testware)*, khả năng truy vết giữa cơ sở kiểm thử *(test basis)* và *testware*, cũng như các vai trò kiểm thử *(testing roles)*.

Tiêu chuẩn ISO/IEC/IEEE 29119-2 cung cấp thêm thông tin về các tiến trình kiểm thử.

1.4.1. Các hoạt động và Nhiệm vụ Kiểm thử (Test Activities and Tasks) FL-1.4.1 • K2

Một tiến trình kiểm thử thường bao gồm các nhóm hoạt động chính được mô tả dưới đây. Mặc dù nhiều hoạt động trong số này có vẻ tuân theo một trình tự hợp lý *(logical sequence)*, nhưng chúng thường được thực hiện lặp đi lặp lại *(iteratively)* hoặc song song *(in parallel)*. Các hoạt động kiểm thử này thường cần được điều chỉnh cho phù hợp với hệ thống và dự án.

- **Lập kế hoạch kiểm thử *(Test planning)*:** Bao gồm việc xác định các mục tiêu kiểm thử *(test objectives)* và sau đó lựa chọn một cách tiếp cận *(approach)* đạt được các mục tiêu đó tốt nhất trong phạm vi các ràng buộc do ngữ cảnh tổng thể đặt ra. Lập kế hoạch kiểm thử được giải thích chi tiết hơn trong mục 5.1.
- **Giám sát kiểm thử và kiểm soát kiểm thử *(Test monitoring and test control)*:** Giám sát kiểm thử *(Test monitoring)* bao gồm việc kiểm tra liên tục tất cả các hoạt động kiểm thử và so sánh tiến độ thực tế với kế hoạch. Kiểm soát kiểm thử *(Test control)* bao gồm việc thực hiện các hành động cần thiết để đáp ứng các mục tiêu kiểm thử. Giám sát kiểm thử và kiểm soát kiểm thử được giải thích chi tiết hơn trong mục 5.3.
- **Phân tích kiểm thử *(Test analysis)*:** Bao gồm việc phân tích cơ sở kiểm thử *(test basis)* để xác định các tính năng có thể kiểm thử được *(testable features)*. Các **điều kiện kiểm thử** *(test conditions)* liên quan được xác định và sắp xếp độ ưu tiên, có tính đến các rủi ro và mức độ rủi ro liên quan (xem mục 5.2). Cơ sở kiểm thử và đối tượng kiểm thử cũng được đánh giá để xác định các khiếm khuyết mà chúng có thể chứa đựng và để đánh giá khả năng kiểm thử *(testability)* của chúng. Phân tích kiểm thử thường được hỗ trợ bởi việc sử dụng các kỹ

thuật kiểm thử (xem Chương 4). Phân tích kiểm thử trả lời cho câu hỏi **"kiểm thử cái gì?"** *("what to test?")* xét theo các tiêu chí bao phủ có thể đo lường được *(measurable coverage criteria)*.

- **Thiết kế kiểm thử** *(Test design)*: Bao gồm việc chi tiết hóa các điều kiện kiểm thử *(test conditions)* thành các ca kiểm thử *(test cases)* và các ***testware*** khác (ví dụ: điều lệ kiểm thử - ***test charters***). Hoạt động này thường bao gồm việc xác định các **mục bao phủ** *(coverage items)*, đóng vai trò làm hướng dẫn để chỉ định các đầu vào của ca kiểm thử *(test case inputs)*. Các kỹ thuật kiểm thử (xem Chương 4) có thể được sử dụng để hỗ trợ hoạt động này. Thiết kế kiểm thử cũng bao gồm việc xác định các yêu cầu về dữ liệu kiểm thử *(test data requirements)*, thiết kế môi trường kiểm thử *(test environment)* và xác định cơ sở hạ tầng cũng như công cụ cần thiết. Thiết kế kiểm thử trả lời cho câu hỏi **"kiểm thử như thế nào?"** *("how to test?")*.
- **Triển khai kiểm thử** *(Test implementation)*: Bao gồm việc tạo ra hoặc thu thập các ***testware*** cần thiết cho việc thực thi kiểm thử (ví dụ: dữ liệu kiểm thử - ***test data***). Các ca kiểm thử *(test cases)* có thể được tổ chức thành các **quy trình/thủ tục kiểm thử** *(test procedures)*, và các thủ tục này thường được tập hợp thành các **bộ kiểm thử** *(test suites)*. Các kịch bản kiểm thử thủ công và tự động *(manual and automated test scripts)* được tạo ra. Các thủ tục kiểm thử được sắp xếp độ ưu tiên và bố trí trong một lịch trình thực thi kiểm thử *(test execution schedule)* để thực thi kiểm thử hiệu quả (xem mục 5.1.5). Môi trường kiểm thử được xây dựng và được xác minh là đã thiết lập đúng đắn.
- **Thực thi kiểm thử** *(Test execution)*: Bao gồm việc chạy các bài kiểm thử theo đúng lịch trình thực thi kiểm thử (các lần chạy kiểm thử - ***test runs***). Thực thi kiểm thử có thể là thủ công hoặc tự động. Thực thi kiểm thử có thể diễn ra dưới nhiều hình thức, bao gồm kiểm thử liên tục *(continuous testing)* hoặc các phiên kiểm thử theo cặp *(pair testing sessions)*. Kết quả kiểm thử thực tế *(actual test results)* được so sánh với kết quả mong đợi *(expected results)*. Kết quả kiểm thử được ghi nhật ký *(logged)*. Các điểm bất thường *(anomalies)* được phân tích để xác định nguyên nhân có khả năng xảy ra của chúng. Phân tích này cho phép chúng ta báo cáo các điểm bất thường dựa trên các hỏng hóc quan sát được (xem mục 5.5).
- **Hoàn tất kiểm thử** *(Test completion)*: Thường diễn ra tại các cột mốc của dự án (***project milestones***, ví dụ: phát hành, kết thúc vòng lặp ***iteration***, hoàn thành một cấp độ kiểm thử). Đối với bất kỳ khiếm khuyết nào chưa được giải quyết, các yêu cầu thay đổi *(change requests)* hoặc các mục tồn đọng sản phẩm *(product backlog items)* sẽ được tạo ra. Bất kỳ ***testware*** nào có thể hữu ích trong tương lai đều được xác định và lưu trữ *(archived)* hoặc bàn giao cho các nhóm thích hợp. Môi trường kiểm thử được đóng lại về một trạng thái đã thỏa thuận. Các hoạt động kiểm thử được phân tích để xác định các bài học kinh nghiệm *(lessons learned)* và các cải tiến cho các vòng lặp, bản phát hành hoặc dự án trong tương lai (xem mục 2.1.6). Một báo cáo hoàn tất kiểm thử *(test completion report)* được tạo ra và truyền đạt tới các bên liên quan.

1.4.2. Tiến trình Kiểm thử trong Ngữ cảnh (Test Process in Context) FL-1.4.2 • K2

Kiểm thử không được thực hiện một cách cô lập. Các hoạt động kiểm thử là một phần không thể tách rời của các quy trình phát triển được thực hiện trong một tổ chức. Kiểm thử cũng được tài trợ bởi các bên liên quan và mục tiêu cuối cùng của nó là giúp đáp ứng các nhu cầu kinh doanh của các bên liên quan. Do đó, cách thức thực hiện kiểm thử sẽ phụ thuộc vào một số yếu tố ngữ cảnh *(contextual factors)* bao gồm:

- **Các bên liên quan** *(Stakeholders)*: nhu cầu, kỳ vọng, yêu cầu, sự sẵn lòng hợp tác, v.v.
- **Các thành viên trong nhóm** *(Team members)*: kỹ năng, kiến thức, mức độ kinh nghiệm, sự sẵn sàng, nhu cầu đào tạo, v.v.
- **Lĩnh vực kinh doanh** *(Business domain)*: mức độ quan trọng/tối mật của đối tượng kiểm thử, các rủi ro đã xác định, nhu cầu thị trường, các quy định pháp lý cụ thể, v.v.

- **Các yếu tố kỹ thuật** *(Technical factors)*: loại phần mềm, kiến trúc sản phẩm, công nghệ sử dụng, v.v.
- **Các ràng buộc dự án** *(Project constraints)*: phạm vi, thời gian, ngân sách, nguồn lực, v.v.
- **Các yếu tố tổ chức** *(Organizational factors)*: cơ cấu tổ chức, các chính sách hiện hành, các thực hành đang áp dụng, v.v.
- **Vòng đời phát triển phần mềm** *(Software development lifecycle)*: các thực hành kỹ thuật, phương pháp phát triển, v.v.
- **Các công cụ** *(Tools)*: tính sẵn có, khả năng sử dụng, sự tuân thủ, v.v.

Các yếu tố này sẽ có tác động lên nhiều vấn đề liên quan đến kiểm thử, bao gồm: chiến lược kiểm thử *(test strategy)*, kỹ thuật kiểm thử được sử dụng, mức độ tự động hóa kiểm thử, mức độ bao phủ yêu cầu, mức độ chi tiết của *testware*, việc báo cáo kiểm thử, v.v.

1.4.3. Sản phẩm Kiểm thử (Testware) FL-1.4.3 • K2

Testware được tạo ra dưới dạng các sản phẩm công việc đầu ra *(output work products)* từ các hoạt động kiểm thử được mô tả trong mục 1.4.1. Có sự khác biệt đáng kể trong cách các tổ chức khác nhau tạo ra, định hình, đặt tên, tổ chức và quản lý các sản phẩm công việc của họ. Quản lý cấu hình (*configuration management*, xem mục 5.4) đúng đắn sẽ đảm bảo tính nhất quán và toàn vẹn của các sản phẩm công việc. Danh sách các sản phẩm công việc dưới đây chưa phải là toàn bộ:

- **Các sản phẩm công việc của Lập kế hoạch kiểm thử** *(Test planning work products)* bao gồm: kế hoạch kiểm thử *(test plan)*, lịch trình kiểm thử *(test schedule)*, sổ đăng ký rủi ro *(risk register)*, tiêu chí bắt đầu và tiêu chí kết thúc (*entry criteria and exit criteria*, xem mục 5.1). Sổ đăng ký rủi ro *(risk register)* là một danh sách các rủi ro cùng với khả năng xảy ra rủi ro *(risk likelihood)*, mức độ tác động của rủi ro *(risk impact)* và thông tin về giảm thiểu rủi ro (*risk mitigation*, xem mục 5.2). Lịch trình kiểm thử, sổ đăng ký rủi ro, tiêu chí bắt đầu và tiêu chí kết thúc thường là một phần của kế hoạch kiểm thử.
- **Các sản phẩm công việc của Giám sát kiểm thử và Kiểm soát kiểm thử** *(Test monitoring and test control work products)* bao gồm: các báo cáo tiến độ kiểm thử (*test progress reports*, xem mục 5.3.2), tài liệu về các chỉ thị kiểm soát (*documentation of control directives*, xem mục 5.3) và thông tin về rủi ro (xem mục 5.2).
- **Các sản phẩm công việc của Phân tích kiểm thử** *(Test analysis work products)* bao gồm: các điều kiện kiểm thử (đã được sắp xếp độ ưu tiên) (*(prioritized) test conditions*, ví dụ: các tiêu chí chấp nhận - *acceptance criteria*, xem mục 4.5.2), và các báo cáo khiếm khuyết liên quan đến các khiếm khuyết trong cơ sở kiểm thử (*defect reports regarding defects in the test basis* - nếu không được sửa trực tiếp).
- **Các sản phẩm công việc của Thiết kế kiểm thử** *(Test design work products)* bao gồm: các ca kiểm thử (đã được sắp xếp độ ưu tiên) *((prioritized) test cases)*, điều lệ kiểm thử *(test charters)*, các mục bao phủ *(coverage items)*, các yêu cầu dữ liệu kiểm thử *(test data requirements)* và các yêu cầu môi trường kiểm thử *(test environment requirements)*.
- **Các sản phẩm công việc của Triển khai kiểm thử** *(Test implementation work products)* bao gồm: các thủ tục kiểm thử *(test procedures)*, các kịch bản kiểm thử thủ công và tự động *(manual and automated test scripts)*, các bộ kiểm thử *(test suites)*, dữ liệu kiểm thử *(test data)*, lịch trình thực thi kiểm thử *(test execution schedule)*, và các hạng mục môi trường kiểm thử *(test environment items)*. Ví dụ về các hạng mục môi trường kiểm thử bao gồm: *stubs* (phần tử giả lập module cấp dưới), *drivers* (trình điều khiển giả lập module cấp trên), *simulators* (trình mô phỏng), và *service virtualizations* (ảo hóa dịch vụ).

- **Các sản phẩm công việc của Thực thi kiểm thử** *(Test execution work products)* bao gồm: nhật ký kiểm thử *(test logs)*, và báo cáo khiếm khuyết (*defect reports*, xem mục 5.5).
- **Các sản phẩm công việc của Hoàn tất kiểm thử** *(Test completion work products)* bao gồm: báo cáo hoàn tất kiểm thử (*test completion report*, xem mục 5.3.2), các hạng mục hành động *(action items)* để cải tiến cho các dự án hoặc vòng lặp tiếp theo, các bài học kinh nghiệm đã được ghi nhận *(documented lessons learned)*, và các yêu cầu thay đổi (*change requests*, ví dụ: dưới dạng các mục tồn đọng sản phẩm - *product backlog items*).

1.4.4. Khả năng truy vết giữa Cơ sở Kiểm thử và Testware (Traceability between the Test Basis and Testware)

FL-1.4.4 • K2

Để thực hiện giám sát kiểm thử và kiểm soát kiểm thử hiệu quả, điều quan trọng là phải thiết lập và duy trì khả năng truy vết *(traceability)* xuyên suốt tiến trình kiểm thử giữa các phần tử của cơ sở kiểm thử *(test basis elements)*, các *testware* liên kết với các phần tử này (ví dụ: điều kiện kiểm thử, rủi ro, ca kiểm thử), kết quả kiểm thử *(test results)*, và các khiếm khuyết *(defects)*.

Khả năng truy vết chính xác hỗ trợ việc đánh giá độ bao phủ *(coverage evaluation)*, vì vậy sẽ rất hữu ích nếu các tiêu chí bao phủ có thể đo lường được *(measurable coverage criteria)* được định nghĩa trong cơ sở kiểm thử. Các tiêu chí bao phủ có thể hoạt động như các chỉ số đo lường hiệu suất chính *(key performance indicators - KPIs)* để thúc đẩy các hoạt động cho thấy các mục tiêu kiểm thử đã đạt được đến mức độ nào (xem mục 1.1.1). Ví dụ:

- Truy vết từ các ca kiểm thử tới các yêu cầu *(Traceability of test cases to requirements)* có thể xác minh rằng các yêu cầu đã được bao phủ bởi các ca kiểm thử.
- Truy vết từ kết quả kiểm thử tới các rủi ro *(Traceability of test results to risks)* có thể được sử dụng để đánh giá mức độ **rủi ro còn lại** *(residual risk)* trong một đối tượng kiểm thử.

Ngoài việc đánh giá độ bao phủ, khả năng truy vết tốt còn giúp xác định tác động của các thay đổi *(impact of changes)*, tạo điều kiện thuận lợi cho việc kiểm toán *(audits)*, và giúp đáp ứng các tiêu chí quản trị CNTT *(IT governance criteria)*. Khả năng truy vết tốt cũng làm cho các báo cáo tiến độ kiểm thử và báo cáo hoàn tất kiểm thử trở nên dễ hiểu hơn bằng cách bao gồm trạng thái của các phần tử thuộc cơ sở kiểm thử. Điều này cũng có thể hỗ trợ trong việc truyền đạt các khía cạnh kỹ thuật của kiểm thử tới các bên liên quan theo một cách dễ hiểu. Khả năng truy vết cung cấp thông tin để đánh giá chất lượng sản phẩm, năng lực quy trình *(process capability)*, và tiến độ dự án so với các mục tiêu kinh doanh.

1.4.5. Các Vai trò trong Kiểm thử (Roles in Testing)

FL-1.4.5 • K2

Trong đề cương này, hai vai trò chính trong kiểm thử được đề cập: **vai trò quản lý kiểm thử** *(test management role)* và **vai trò kiểm thử** *(testing role)*. Các hoạt động và nhiệm vụ được giao cho hai vai trò này phụ thuộc vào các yếu tố như bối cảnh dự án và sản phẩm, kỹ năng của những người đảm nhận vai trò, và tổ chức.

- **Vai trò quản lý kiểm thử** *(Test management role)* chịu trách nhiệm tổng thể về tiến trình kiểm thử, nhóm kiểm thử và việc lãnh đạo các hoạt động kiểm thử. Vai trò quản lý kiểm thử chủ yếu tập trung vào các hoạt động: **lập kế hoạch kiểm thử** *(test planning)*, **giám sát kiểm thử** *(test monitoring)*, **kiểm soát kiểm thử** *(test control)* **và hoàn tất kiểm thử** *(test completion)*. Cách thức thực hiện vai trò quản lý kiểm thử thay đổi tùy theo ngữ cảnh. Ví dụ, trong phát triển phần mềm Agile, một số nhiệm vụ quản lý kiểm thử có thể được xử lý bởi chính nhóm Agile. Các nhiệm vụ trải rộng qua nhiều nhóm hoặc toàn bộ tổ chức có thể được thực hiện bởi các quản lý kiểm thử *(test managers)* bên ngoài nhóm phát triển.

- **Vai trò kiểm thử** *(Testing role)* chịu trách nhiệm tổng thể về khía cạnh kỹ thuật *(engineering / technical aspect)* của kiểm thử. Vai trò kiểm thử chủ yếu tập trung vào các hoạt động: **phân tích kiểm thử** *(test analysis)*, **thiết kế kiểm thử** *(test design)*, **triển khai kiểm thử** *(test implementation)* **và thực thi kiểm thử** *(test execution)*.

Những người khác nhau có thể đảm nhận các vai trò này vào những thời điểm khác nhau. Ví dụ, vai trò quản lý kiểm thử có thể được thực hiện bởi một trưởng nhóm *(team leader)*, bởi một quản lý kiểm thử *(test manager)*, bởi một quản lý phát triển *(development manager)*, v.v. Cũng có thể có trường hợp một người đảm nhận cả hai vai trò kiểm thử và quản lý kiểm thử cùng một lúc.

1.5. Kỹ năng thiết yếu và Thực hành tốt trong Kiểm thử (Essential Skills and Good Practices in Testing)

Kỹ năng *(Skill)* là khả năng làm tốt một việc gì đó xuất phát từ kiến thức, sự rèn luyện thực hành và năng khiếu của một người. Những kiểm thử viên giỏi cần sở hữu một số kỹ năng thiết yếu để làm tốt công việc của mình. Kiểm thử viên giỏi nên là những người làm việc nhóm hiệu quả *(effective team players)* và có khả năng thực hiện kiểm thử ở các mức độ độc lập kiểm thử *(levels of test independence)* khác nhau.

1.5.1. Các Kỹ năng chung cần thiết cho Kiểm thử (Generic Skills Required for Testing)

FL-1.5.1 • K2

Mặc dù mang tính chung chung, các kỹ năng sau đây đặc biệt phù hợp với kiểm thử viên:

- **Kiến thức về kiểm thử** *(Testing knowledge)*: để tăng tính hiệu quả (*effectiveness* - làm đúng việc/tìm được nhiều lỗi) của kiểm thử, ví dụ: bằng cách sử dụng các kỹ thuật kiểm thử.
- **Sự kỹ lưỡng, cẩn thận, tò mò, chú ý đến chi tiết, làm việc có phương pháp** *(Thoroughness, carefulness, curiosity, attention to details, being methodical)*: để xác định các khiếm khuyết, đặc biệt là những khiếm khuyết khó tìm.
- **Kỹ năng giao tiếp tốt, lắng nghe chủ động, tinh thần đồng đội** *(Good communication skills, active listening, being a team player)*: để tương tác hiệu quả với tất cả các bên liên quan, truyền đạt thông tin cho người khác, để được thấu hiểu, và để báo cáo cũng như thảo luận về các khiếm khuyết.
- **Tư duy phân tích, tư duy phản biện, sự sáng tạo** *(Analytical thinking, critical thinking, creativity)*: để tăng tính hiệu quả *(effectiveness)* của kiểm thử.
- **Kiến thức kỹ thuật** *(Technical knowledge)*: để tăng hiệu suất (*efficiency* - làm nhanh, tiết kiệm nguồn lực) của kiểm thử, ví dụ: bằng cách sử dụng các công cụ kiểm thử thích hợp.
- **Kiến thức nghiệp vụ / miền ứng dụng** *(Domain knowledge)*: để có thể hiểu và giao tiếp với người dùng cuối / đại diện nghiệp vụ *(end users / business representatives)*.

Kiểm thử viên thường là người mang tin xấu *(bearers of bad news)*. Một đặc tính tâm lý phổ biến của con người là đổ lỗi cho người mang tin xấu *(blame the bearer of bad news)*. Điều này làm cho kỹ năng giao tiếp trở nên tối quan trọng đối với kiểm thử viên. Việc truyền đạt kết quả kiểm thử có thể bị nhìn nhận như là sự chỉ trích đối với sản phẩm và tác giả của nó. **Thiên kiến xác nhận** *(Confirmation bias)* có thể khiến người ta khó chấp nhận những thông tin trái ngược với niềm tin hiện tại. Một số người có thể coi kiểm thử là một hoạt động mang tính phá hoại *(destructive activity)*, mặc dù nó đóng góp rất lớn vào sự thành công của dự án và chất lượng sản phẩm. Để cố

gắng cải thiện quan điểm này, thông tin về các khiếm khuyết và hỏng hóc nên được truyền đạt theo **cách mang tính xây dựng** *(constructive way)*.

1.5.2. Cách tiếp cận Toàn nhóm (Whole Team Approach) FL-1.5.2 • K1

Một trong những kỹ năng quan trọng đối với một kiểm thử viên là khả năng làm việc hiệu quả trong bối cảnh nhóm và đóng góp tích cực vào mục tiêu chung của nhóm. **Cách tiếp cận toàn nhóm** *(Whole team approach)* – một thực hành bắt nguồn từ Lập trình cực hạn (***Extreme Programming - XP***, xem mục 2.1) – được xây dựng dựa trên kỹ năng này.

Trong cách tiếp cận toàn nhóm, bất kỳ thành viên nào trong nhóm có kiến thức và kỹ năng cần thiết đều có thể thực hiện bất kỳ công việc nào, và **mọi người đều chịu trách nhiệm về chất lượng** *(everyone is responsible for quality)*. Các thành viên trong nhóm chia sẻ cùng một không gian làm việc (vật lý hoặc ảo), vì việc ngồi chung *(co-location)* tạo điều kiện thuận lợi cho giao tiếp và tương tác. Cách tiếp cận toàn nhóm cải thiện động lực nhóm *(team dynamics)*, tăng cường giao tiếp và cộng tác trong nhóm, đồng thời tạo ra sức mạnh tổng hợp *(synergy)* bằng cách cho phép tận dụng các bộ kỹ năng khác nhau trong nhóm vì lợi ích của dự án.

Kiểm thử viên làm việc chặt chẽ với các thành viên khác trong nhóm để đảm bảo đạt được các mức chất lượng mong muốn. Điều này bao gồm việc cộng tác với các đại diện nghiệp vụ *(business representatives)* để giúp họ tạo ra các bài kiểm thử chấp nhận *(acceptance tests)* phù hợp và làm việc với các lập trình viên *(developers)* để thống nhất về chiến lược kiểm thử và quyết định các cách tiếp cận tự động hóa kiểm thử. Nhờ đó, kiểm thử viên có thể chuyển giao kiến thức kiểm thử cho các thành viên khác trong nhóm và gây ảnh hưởng tích cực đến việc phát triển sản phẩm.

Tùy thuộc vào ngữ cảnh, cách tiếp cận toàn nhóm không phải lúc nào cũng phù hợp. Chẳng hạn, trong một số tình huống như hệ thống quan trọng về an toàn *(safety-critical)*, một mức độ độc lập kiểm thử cao có thể là cần thiết.

1.5.3. Tính độc lập của Kiểm thử (Independence of Testing) FL-1.5.3 • K2

Một mức độ độc lập nhất định giúp kiểm thử viên tìm kiếm khiếm khuyết hiệu quả hơn do sự khác biệt về **thiên kiến nhận thức** *(cognitive biases)* giữa tác giả *(author)* và kiểm thử viên *(tester)* (Salman 1995). Tuy nhiên, sự độc lập không phải là sự thay thế cho sự quen thuộc *(familiarity)*, ví dụ: các lập trình viên có thể tìm thấy nhiều khiếm khuyết trong chính mã nguồn của họ một cách hiệu quả.

Các sản phẩm công việc có thể được kiểm thử theo thứ tự mức độ độc lập tăng dần:

1. **Không có tính độc lập** *(No independence)*: được kiểm thử bởi chính tác giả tạo ra chúng.
2. **Có một chút độc lập** *(Some independence)*: được kiểm thử bởi các đồng nghiệp cùng nhóm với tác giả *(author's peers from the same team)*.
3. **Tính độc lập cao** *(High independence)*: được kiểm thử bởi các kiểm thử viên bên ngoài nhóm của tác giả nhưng vẫn trong cùng tổ chức.
4. **Tính độc lập rất cao** *(Very high independence)*: được kiểm thử bởi các kiểm thử viên từ bên ngoài tổ chức.

Đối với hầu hết các dự án, thông thường tốt nhất là thực hiện kiểm thử với **nhiều mức độ độc lập khác nhau** (ví dụ: các lập trình viên thực hiện kiểm thử thành phần và kiểm thử tích hợp thành phần, nhóm kiểm thử thực hiện kiểm thử hệ thống và kiểm thử tích hợp hệ thống, và các đại diện nghiệp vụ thực hiện kiểm thử chấp nhận).

- **Lợi ích chính** ***(Main benefit)*** của tính độc lập trong kiểm thử là các kiểm thử viên độc lập có khả năng nhận ra các loại hỏng hóc và khiếm khuyết khác so với các lập trình viên vì họ có nền tảng, góc nhìn kỹ thuật và các thiên kiến khác nhau. Hơn nữa, một kiểm thử viên độc lập có thể xác minh, phản biện hoặc bác bỏ các giả định ***(assumptions)*** do các bên liên quan đưa ra trong quá trình đặc tả và triển khai hệ thống.
- **Tuy nhiên, cũng có một số hạn chế** ***(drawbacks)*****:** Kiểm thử viên độc lập có thể bị cô lập khỏi nhóm phát triển, dẫn đến thiếu sự hợp tác, gặp các vấn đề giao tiếp, hoặc mối quan hệ đối đầu ***(adversarial relationship)*** với nhóm phát triển. Các lập trình viên có thể mất đi ý thức trách nhiệm đối với chất lượng. Kiểm thử viên độc lập có thể bị coi là nút thắt cổ chai ***(bottleneck)*** hoặc bị đổ lỗi cho sự chậm trễ trong việc phát hành.

CHƯƠNG 2

🕑 Thời lượng phân bổ: 130 phút

Kiểm thử Xuyên suốt Vòng đời Phát triển Phần mềm (Testing Throughout the Software Development Lifecycle)

K1 Bảng Từ Khóa Chuẩn ISTQB (Keywords) — Cần nhớ chính xác thuật ngữ gốc Tiếng Anh khi làm bài thi

acceptance testing	kiểm thử chấp nhận	**black-box testing**	kiểm thử hộp đen
component integration testing	kiểm thử tích hợp thành phần	**component testing**	kiểm thử thành phần/đơn vị
confirmation testing	kiểm thử xác nhận	**functional testing**	kiểm thử chức năng
integration testing	kiểm thử tích hợp	**maintenance testing**	kiểm thử bảo trì
non-functional testing	kiểm thử phi chức năng	**regression testing**	kiểm thử hồi quy
shift left	dịch trái - kiểm thử sớm	**system integration testing**	kiểm thử tích hợp hệ thống
system testing	kiểm thử hệ thống	**test level**	cấp độ kiểm thử
test object	đối tượng kiểm thử	**test type**	loại kiểm thử
white-box testing	kiểm thử hộp trắng		

Mục Tiêu Học Tập Trọng Tâm (Learning Objectives)

MÃ LO	MỨC NHẬN THỨC	NỘI DUNG YÊU CẦU TRONG ĐỀ THI
2.1 Kiểm thử trong Ngữ cảnh của Vòng đời Phát triển Phần mềm (Testing in the Context of a Software Development Lifecycle)		
FL-2.1.1	K2 · Hiểu	Giải thích tác động của vòng đời phát triển phần mềm được chọn lên hoạt động kiểm thử
FL-2.1.2	K1 · Ghi nhớ	Nhớ lại các thực hành kiểm thử tốt áp dụng cho mọi vòng đời phát triển phần mềm
FL-2.1.3	K1 · Ghi nhớ	Nhớ lại các ví dụ về cách tiếp cận kiểm thử trước (*test-first approaches*) trong phát triển
FL-2.1.4	K2 · Hiểu	Tóm tắt cách DevOps có thể tác động đến kiểm thử
FL-2.1.5	K2 · Hiểu	Giải thích khái niệm *shift left*
FL-2.1.6	K2 · Hiểu	Giải thích cách các buổi họp nhìn lại (*retrospectives*) có thể được sử dụng làm cơ chế cải tiến quy trình

MÃ LO	MỨC NHẬN THỨC	NỘI DUNG YÊU CẦU TRONG ĐỀ THI
2.2 Các Cấp độ Kiểm thử và Các Loại Kiểm thử (Test Levels and Test Types)		
FL-2.2.1	K2 · Hiểu	Phân biệt các cấp độ kiểm thử khác nhau (*test levels*)
FL-2.2.2	K2 · Hiểu	Phân biệt các loại kiểm thử khác nhau (*test types*)
FL-2.2.3	K2 · Hiểu	Phân biệt kiểm thử xác nhận (*confirmation testing*) với kiểm thử hồi quy (*regression testing*)
2.3 Kiểm thử Bảo trì (Maintenance Testing)		
FL-2.3.1	K2 · Hiểu	Tóm tắt về kiểm thử bảo trì và các tác nhân kích hoạt nó (*triggers*)

2.1. Kiểm thử trong Ngữ cảnh của Vòng đời Phát triển Phần mềm (SDLC)

Mô hình SDLC là một biểu diễn trừu tượng, ở mức độ cao của quy trình phát triển phần mềm. Một mô hình SDLC xác định cách thức mà các giai đoạn phát triển khác nhau và các loại hoạt động được thực hiện trong quy trình này liên hệ với nhau, cả về mặt logic lẫn trình tự thời gian. Ví dụ về các mô hình SDLC bao gồm:

- **Mô hình phát triển tuần tự** *(Sequential development models)*: ví dụ: mô hình thác nước *(waterfall model)*, mô hình chữ V *(V-model)*.
- **Mô hình phát triển lặp** *(Iterative development models)*: ví dụ: mô hình xoắn ốc *(spiral model)*, tạo mẫu *(prototyping)*.
- **Mô hình phát triển tăng trưởng** *(Incremental development models)*: ví dụ: Tiến trình Hợp nhất *(Unified Process)*.

Một số hoạt động trong quy trình phát triển phần mềm cũng có thể được mô tả bằng các phương pháp phát triển phần mềm chi tiết hơn và các thực hành Agile. Ví dụ bao gồm: phát triển hướng kiểm thử chấp nhận *(Acceptance Test-Driven Development - ATDD)*, phát triển hướng hành vi *(Behavior-Driven Development - BDD)*, thiết kế hướng miền *(Domain-Driven Design - DDD)*, lập trình cực hạn *(Extreme Programming - XP)*, phát triển hướng tính năng *(Feature-Driven Development - FDD)*, Kanban, Lean IT, Scrum, và phát triển hướng kiểm thử *(Test-Driven Development - TDD)*.

2.1.1. Tác động của Vòng đời Phát triển Phần mềm lên Kiểm thử

FL-2.1.1 • K2

Kiểm thử phải được điều chỉnh cho thích ứng với SDLC để thành công. Việc lựa chọn SDLC tác động lên:

- Phạm vi và thời điểm của các hoạt động kiểm thử (ví dụ: các cấp độ kiểm thử và các loại kiểm thử)
- Mức độ chi tiết của tài liệu kiểm thử
- Việc lựa chọn các kỹ thuật kiểm thử và cách tiếp cận kiểm thử
- Mức độ tự động hóa kiểm thử
- Vai trò và trách nhiệm của một kiểm thử viên
- **Trong các mô hình phát triển tuần tự** *(sequential development models)*: ở các giai đoạn ban đầu, kiểm thử viên thường tham gia vào việc duyệt yêu cầu *(requirement reviews)*, phân tích kiểm thử, và thiết kế kiểm thử.

Mã nguồn có thể thực thi thường được tạo ra ở các giai đoạn muộn hơn, vì vậy thông thường kiểm thử động không thể thực hiện sớm trong SDLC.

- **Trong một số mô hình phát triển lặp *(iterative development models)* và mô hình phát triển tăng trưởng *(incremental development models)*:** người ta giả định rằng mỗi vòng lặp *(iteration)* sẽ chuyển giao một nguyên mẫu hoạt động được *(working prototype)* hoặc một phần tăng trưởng của sản phẩm *(product increment)*. Điều này ngụ ý rằng trong mỗi vòng lặp, cả kiểm thử tĩnh và kiểm thử động đều có thể được thực hiện ở tất cả các cấp độ kiểm thử. Việc chuyển giao thường xuyên các phần tăng trưởng đòi hỏi phản hồi nhanh *(fast feedback)* và kiểm thử hồi quy sâu rộng *(extensive regression testing)*.
- **Phát triển phần mềm Agile *(Agile software development)*:** giả định rằng sự thay đổi có thể xảy ra xuyên suốt dự án. Do đó, tài liệu sản phẩm công việc gọn nhẹ *(lightweight work product documentation)* và tự động hóa kiểm thử sâu rộng để giúp kiểm thử hồi quy dễ dàng hơn được ưu tiên trong các dự án Agile. Ngoài ra, hầu hết các kiểm thử thủ công có xu hướng được thực hiện bằng cách sử dụng các kỹ thuật kiểm thử dựa trên kinh nghiệm (***experience-based test techniques***, xem Mục 4.4) vốn không đòi hỏi việc phân tích và thiết kế kiểm thử sâu rộng từ trước.

2.1.2. Vòng đời Phát triển Phần mềm và các Thực hành Kiểm thử Tốt

FL-2.1.2 • K1

Các thực hành kiểm thử tốt, **không phụ thuộc vào mô hình SDLC đã chọn**, bao gồm:

- Đối với mỗi hoạt động phát triển phần mềm, đều có một hoạt động kiểm thử tương ứng, sao cho tất cả các hoạt động phát triển đều chịu sự kiểm soát chất lượng *(quality control)*.
- Các cấp độ kiểm thử khác nhau (xem mục 2.2.1) có các mục tiêu kiểm thử cụ thể và khác nhau, cho phép việc kiểm thử được toàn diện một cách hợp lý trong khi tránh được sự dư thừa/trùng lặp *(redundancy)*.
- Việc phân tích và thiết kế kiểm thử cho một cấp độ kiểm thử nhất định bắt đầu ngay trong giai đoạn phát triển tương ứng của SDLC, để kiểm thử có thể tuân thủ nguyên lý kiểm thử sớm (***early testing***, xem mục 1.3).
- Kiểm thử viên tham gia vào việc duyệt *(reviewing)* các sản phẩm công việc ngay khi các bản nháp *(drafts)* của các sản phẩm công việc này vừa có sẵn, để việc kiểm thử và phát hiện khiếm khuyết sớm hơn này có thể hỗ trợ cho ***shift left*** (xem mục 2.1.5).

2.1.3. Kiểm thử như một Động lực Dẫn dắt Phát triển Phần mềm (Testing as a Driver for Software Development)

FL-2.1.3 • K1

TDD, ATDD và BDD là các cách tiếp cận phát triển tương tự nhau, trong đó các bài kiểm thử được định nghĩa như một phương tiện để định hướng việc phát triển. Mỗi cách tiếp cận này đều hiện thực hóa nguyên lý kiểm thử sớm (xem mục 1.3) và tuân theo ***shift left*** (xem mục 2.1.5), bởi vì các bài kiểm thử được định nghĩa **trước khi mã nguồn được viết**. Chúng hỗ trợ mô hình phát triển lặp. Các cách tiếp cận này có đặc điểm như sau:

- **Phát triển hướng Kiểm thử *(Test-Driven Development - TDD)*:**
  - Định hướng việc viết mã thông qua các ca kiểm thử (thay vì thiết kế phần mềm sâu rộng) (Beck 2003).
  - Các bài kiểm thử được viết trước, sau đó mã nguồn được viết để thỏa mãn các bài kiểm thử đó, và cuối cùng các bài kiểm thử cùng mã nguồn được tái cấu trúc *(refactored)*.
- **Phát triển hướng Kiểm thử Chấp nhận *(Acceptance Test-Driven Development - ATDD)*** (xem mục 4.5.3):
  - Xây dựng các bài kiểm thử từ các tiêu chí chấp nhận *(acceptance criteria)* như một phần của quy trình thiết kế hệ thống (Gärtner 2011).

  - Các bài kiểm thử được viết trước khi phần ứng dụng đó được phát triển để thỏa mãn các bài kiểm thử.
- **Phát triển hướng Hành vi *(Behavior-Driven Development - BDD)*:**
  - Diễn đạt hành vi mong muốn của một ứng dụng bằng các ca kiểm thử được viết dưới dạng ngôn ngữ tự nhiên đơn giản, dễ hiểu đối với các bên liên quan – thường sử dụng định dạng **Given/When/Then** (Chelimsky 2010).
  - Các ca kiểm thử sau đó nên được tự động dịch thành các bài kiểm thử có thể thực thi được ***(executable tests)***.

Đối với tất cả các cách tiếp cận trên, các bài kiểm thử có thể được duy trì lâu dài dưới dạng các bài kiểm thử tự động để đảm bảo chất lượng mã nguồn trong các lần điều chỉnh / tái cấu trúc ***(refactoring)*** trong tương lai.

2.1.4. DevOps và Kiểm thử (DevOps and Testing) FL-2.1.4 • K2

DevOps là một cách tiếp cận mang tính tổ chức nhằm tạo ra sức mạnh tổng hợp ***(synergy)*** bằng cách đưa bộ phận phát triển (***development*** - bao gồm cả kiểm thử) và bộ phận vận hành ***(operations)*** làm việc cùng nhau để đạt được một tập hợp các mục tiêu chung. DevOps đòi hỏi một sự thay đổi văn hóa ***(cultural shift)*** trong tổ chức để thu hẹp khoảng cách giữa phát triển (bao gồm kiểm thử) và vận hành, đồng thời coi trọng chức năng của hai bên ngang nhau. DevOps thúc đẩy quyền tự chủ của nhóm ***(team autonomy)***, phản hồi nhanh ***(fast feedback)***, chuỗi công cụ tích hợp ***(integrated toolchains)***, và các thực hành kỹ thuật như tích hợp liên tục ***(continuous integration - CI)*** và chuyển giao liên tục ***(continuous delivery - CD)***. Điều này cho phép các nhóm xây dựng, kiểm thử và phát hành mã nguồn chất lượng cao nhanh hơn thông qua một đường ống chuyển giao DevOps ***(DevOps delivery pipeline)*** (Kim 2016).

Từ góc độ kiểm thử, một số **lợi ích của DevOps** là:

- Phản hồi nhanh về chất lượng mã nguồn, và liệu các thay đổi có gây ảnh hưởng bất lợi đến mã nguồn hiện có hay không.
- CI thúc đẩy ***shift left*** trong kiểm thử (xem mục 2.1.5) bằng cách khuyến khích các lập trình viên nộp mã nguồn chất lượng cao đi kèm với các bài kiểm thử thành phần ***(component tests)*** và phân tích tĩnh ***(static analysis)***.
- Các quy trình tự động được thúc đẩy như CI/CD giúp tạo điều kiện thiết lập các môi trường kiểm thử ổn định.
- Tăng khả năng quan sát ***(visibility)*** đối với các đặc tính chất lượng phi chức năng (ví dụ: hiệu suất hoạt động - ***performance efficiency***, độ tin cậy - ***reliability***).
- Tự động hóa thông qua đường ống chuyển giao làm giảm nhu cầu kiểm thử thủ công lặp đi lặp lại.
- Rủi ro hồi quy ***(risk of regression)*** được giảm thiểu nhờ quy mô và phạm vi của các bài kiểm thử hồi quy tự động.

DevOps cũng không tránh khỏi những **rủi ro và thách thức**, bao gồm:

- Đường ống chuyển giao DevOps phải được định nghĩa và thiết lập.
- Các công cụ CI / CD phải được đưa vào và bảo trì.
- Tự động hóa kiểm thử đòi hỏi thêm nguồn lực và có thể khó thiết lập cũng như bảo trì.

Mặc dù DevOps đi kèm với mức độ kiểm thử tự động hóa cao, kiểm thử thủ công – đặc biệt là từ góc nhìn của người dùng – vẫn sẽ cần thiết.

2.1.5. Dịch trái (Shift Left) FL-2.1.5 • K2

Nguyên lý kiểm thử sớm (xem mục 1.3) đôi khi được gọi là ***shift left*** vì đây là cách tiếp cận mà trong đó kiểm thử được thực hiện sớm hơn trong SDLC. ***Shift left*** về cơ bản gợi ý rằng kiểm thử nên được thực hiện sớm hơn (ví dụ: không chờ mã nguồn được viết xong hoặc không chờ các thành phần được tích hợp xong), nhưng điều đó **không có nghĩa là** việc kiểm thử ở giai đoạn muộn hơn trong SDLC bị bỏ qua.

Có một số thực hành tốt minh họa cách đạt được "shift left" trong kiểm thử, bao gồm:

- Duyệt đặc tả *(reviewing the specification)* từ góc nhìn của kiểm thử viên. Các hoạt động duyệt trên đặc tả này thường tìm thấy các khiếm khuyết tiềm ẩn, chẳng hạn như sự mơ hồ *(ambiguities)*, thiếu sót *(incompleteness)*, và mâu thuẫn *(inconsistencies)*.
- Viết các ca kiểm thử trước khi mã nguồn được viết và cho mã nguồn chạy trong một khung giá đỡ kiểm thử *(test harness)* trong quá trình viết mã.
- Sử dụng CI và tốt hơn nữa là CD vì nó đi kèm với phản hồi nhanh và các bài kiểm thử thành phần tự động đi kèm với mã nguồn khi được đẩy lên kho lưu trữ mã *(code repository)*.
- Hoàn thành phân tích tĩnh mã nguồn trước khi kiểm thử động, hoặc như một phần của quy trình tự động.
- Thực hiện kiểm thử phi chức năng *(non-functional testing)* bắt đầu ngay từ cấp độ kiểm thử thành phần *(component test level)* khi có thể. Đây là một hình thức ***shift left*** vì các loại kiểm thử phi chức năng này thường có xu hướng được thực hiện muộn hơn trong SDLC khi đã có một hệ thống hoàn chỉnh và một môi trường kiểm thử đại diện.

Shift left có thể dẫn đến việc tốn thêm đào tạo, công sức và/hoặc chi phí sớm hơn trong quy trình nhưng được kỳ vọng sẽ tiết kiệm công sức và/hoặc chi phí ở giai đoạn sau của quy trình.

Đối với ***shift left***, điều quan trọng là các bên liên quan phải bị thuyết phục và đồng thuận *(bought into)* với khái niệm này.

2.1.6. Họp nhìn lại và Cải tiến Quy trình (Retrospectives and Process Improvement) FL-2.1.6 • K2

Các buổi họp nhìn lại *(Retrospectives)* thường được tổ chức vào cuối một dự án hoặc một vòng lặp *(iteration)*, tại một cột mốc phát hành *(release milestone)*, hoặc có thể được tổ chức bất cứ khi nào cần thiết. Thời điểm và cách tổ chức các buổi họp nhìn lại phụ thuộc vào mô hình SDLC cụ thể đang được tuân theo. Trong các cuộc họp này, những người tham gia (không chỉ kiểm thử viên mà còn cả lập trình viên, kiến trúc sư, chủ sở hữu sản phẩm ***product owner***, chuyên viên phân tích nghiệp vụ ***business analysts***) thảo luận về:

- Điều gì đã thành công, và nên được giữ lại?
- Điều gì chưa thành công và có thể được cải thiện?
- Làm thế nào để đưa các cải tiến vào và duy trì các thành công đó trong tương lai?

Kết quả cần được ghi lại và thường là một phần của báo cáo hoàn tất kiểm thử (***test completion report***, xem mục 5.3.2). Các buổi họp nhìn lại đóng vai trò sống còn đối với việc triển khai thành công sự cải tiến liên tục *(continuous improvement)*, và điều quan trọng là mọi cải tiến được khuyến nghị đều phải được theo dõi thực hiện *(followed up)*.

Các lợi ích điển hình cho kiểm thử bao gồm:

- Tăng hiệu quả / hiệu suất kiểm thử *(test effectiveness / efficiency)* (ví dụ: bằng cách thực hiện các đề xuất cải tiến quy trình).
- Tăng chất lượng của ***testware*** (ví dụ: bằng cách cùng nhau rà soát lại các quy trình kiểm thử).
- Gắn kết nhóm và học hỏi *(team bonding and learning)* (ví dụ: nhờ cơ hội nêu ra các vấn đề và đề xuất các điểm cải tiến).
- Cải thiện chất lượng của cơ sở kiểm thử *(test basis)* (ví dụ: khi những thiếu sót về phạm vi và chất lượng của các yêu cầu được giải quyết).
- Hợp tác tốt hơn giữa phát triển và kiểm thử (ví dụ: khi sự cộng tác được rà soát và tối ưu hóa thường xuyên).

2.2. Các Cấp độ Kiểm thử và Các Loại Kiểm thử (Test Levels and Test Types)

- **Các cấp độ kiểm thử** *(Test levels)* là các nhóm hoạt động kiểm thử được tổ chức và quản lý cùng nhau. Mỗi cấp độ kiểm thử là một thể hiện *(instance)* của tiến trình kiểm thử, được thực hiện đối với phần mềm ở một giai đoạn phát triển nhất định, từ các thành phần riêng lẻ cho đến các hệ thống hoàn chỉnh hoặc, nếu có, hệ thống của các hệ thống *(systems of systems)*. Các cấp độ kiểm thử có liên quan đến các hoạt động khác trong SDLC. Trong các mô hình SDLC tuần tự, các cấp độ kiểm thử thường được định nghĩa sao cho tiêu chí kết thúc *(exit criteria)* của cấp độ này là một phần của tiêu chí bắt đầu *(entry criteria)* cho cấp độ tiếp theo. Trong một số mô hình lặp, điều này có thể không áp dụng. Các hoạt động phát triển có thể trải dài qua nhiều cấp độ kiểm thử. Các cấp độ kiểm thử có thể gối đầu/chồng lấp nhau về mặt thời gian *(overlap in time)*.
- **Các loại kiểm thử** *(Test types)* là các nhóm hoạt động kiểm thử liên quan đến các đặc tính chất lượng cụ thể *(specific quality characteristics)* và hầu hết các hoạt động kiểm thử đó **có thể được thực hiện ở mọi cấp độ kiểm thử** *(at every test level)*.

2.2.1. Các Cấp độ Kiểm thử (Test Levels) FL-2.2.1 • K2

Trong đề cương này, **5 cấp độ kiểm thử** sau đây được mô tả:

1. **Kiểm thử thành phần (*Component testing* - còn được gọi là kiểm thử đơn vị *unit testing*):** Tập trung vào việc kiểm thử các thành phần một cách cô lập *(in isolation)*. Nó thường đòi hỏi sự hỗ trợ cụ thể, chẳng hạn như các khung giá đỡ kiểm thử *(test harnesses)* hoặc các bộ khung kiểm thử đơn vị *(unit test frameworks)*. Kiểm thử thành phần thường được thực hiện bởi các lập trình viên trong môi trường phát triển của họ.
2. **Kiểm thử tích hợp thành phần (*Component integration testing* - còn gọi là *unit integration testing*):** Tập trung vào việc kiểm thử các giao diện *(interfaces)* và tương tác *(interactions)* giữa các thành phần. Kiểm thử tích hợp thành phần phụ thuộc rất nhiều vào các chiến lược tích hợp như từ dưới lên *(bottom-up)*, từ trên xuống *(top-down)* hoặc tích hợp đồng loạt *(big-bang)*.
3. **Kiểm thử hệ thống *(System testing)*:** Tập trung vào hành vi và khả năng tổng thể của toàn bộ một hệ thống hoặc sản phẩm, thường bao gồm kiểm thử chức năng của các tác vụ đầu-cuối *(end-to-end tasks)* và kiểm thử phi chức năng đối với các đặc tính chất lượng. Đối với một số đặc tính chất lượng phi chức năng, tốt hơn là nên kiểm thử chúng trên một hệ thống hoàn chỉnh trong một môi trường kiểm thử đại diện (***representative test environment***, ví dụ: khả năng sử dụng - ***usability***). Việc sử dụng các mô phỏng của các hệ thống con *(simulations of sub-systems)* cũng có thể thực hiện được. Kiểm thử hệ thống có thể được thực hiện bởi một nhóm kiểm thử độc lập, và gắn liền với các đặc tả của hệ thống.
4. **Kiểm thử tích hợp hệ thống *(System integration testing)*:** Tập trung vào việc kiểm thử các giao diện của hệ thống đang được kiểm thử *(system under test)* với các hệ thống khác và các dịch vụ bên ngoài *(external*

services). Kiểm thử tích hợp hệ thống đòi hỏi các môi trường kiểm thử phù hợp, tốt nhất là tương tự như môi trường vận hành thực tế *(operational environment)*.

5. **Kiểm thử chấp nhận** *(Acceptance testing)*: Tập trung vào việc thẩm định *(validation)* và chứng minh sự sẵn sàng để triển khai *(readiness for deployment)*, nghĩa là hệ thống đáp ứng được nhu cầu kinh doanh của người dùng. Lý tưởng nhất, kiểm thử chấp nhận nên được thực hiện bởi những người dùng dự kiến *(intended users)*. Các hình thức chính của kiểm thử chấp nhận là: kiểm thử chấp nhận người dùng *(user acceptance testing - UAT)*, kiểm thử chấp nhận vận hành *(operational acceptance testing)*, kiểm thử chấp nhận theo hợp đồng và quy định chế tài *(contractual and regulatory acceptance testing)*, kiểm thử alpha *(alpha testing)* và kiểm thử beta *(beta testing)*.

Các cấp độ kiểm thử được phân biệt bởi danh sách (chưa đầy đủ) các thuộc tính sau đây, nhằm tránh sự chồng chéo của các hoạt động kiểm thử:

- Đối tượng kiểm thử *(Test object)*
- Mục tiêu kiểm thử *(Test objectives)*
- Cơ sở kiểm thử *(Test basis)*
- Khiếm khuyết và hỏng hóc *(Defects and failures)*
- Cách tiếp cận và trách nhiệm *(Approach and responsibilities)*

2.2.2. Các Loại Kiểm thử (Test Types) FL-2.2.2 • K2

Có rất nhiều loại kiểm thử tồn tại và có thể áp dụng trong các dự án. Trong đề cương này, **4 loại kiểm thử** sau đây được giải quyết:

1. **Kiểm thử chức năng** *(Functional testing)*: Đánh giá các chức năng mà một thành phần hoặc hệ thống phải thực hiện. Các chức năng là **"cái gì"** *("what")* mà đối tượng kiểm thử phải làm. Mục tiêu chính của kiểm thử chức năng là kiểm tra tính đầy đủ chức năng *(functional completeness)*, tính đúng đắn chức năng *(functional correctness)* và tính phù hợp chức năng *(functional appropriateness)*.
2. **Kiểm thử phi chức năng** *(Non-functional testing)*: Đánh giá các thuộc tính khác ngoài các đặc tính chức năng của một thành phần hoặc hệ thống. Kiểm thử phi chức năng là việc kiểm thử **"hệ thống hoạt động tốt như thế nào"** *("how well the system behaves")*. Mục tiêu chính của kiểm thử phi chức năng là kiểm tra các đặc tính chất lượng phi chức năng. Tiêu chuẩn **ISO/IEC 25010** cung cấp phân loại các đặc tính chất lượng phi chức năng như sau:
    - Hiệu suất hoạt động *(Performance efficiency)*
    - Tính tương thích *(Compatibility)*
    - Khả năng sử dụng (*Usability* - còn được gọi là khả năng tương tác *interaction capability*)
    - Độ tin cậy *(Reliability)*
    - Tính bảo mật *(Security)*
    - Khả năng bảo trì *(Maintainability)*
    - Tính khả chuyển (*Portability* - còn được gọi là tính linh hoạt *flexibility*)
    - Tính an toàn *(Safety)*

Đôi khi việc bắt đầu kiểm thử phi chức năng sớm trong SDLC là phù hợp (ví dụ: như một phần của việc duyệt hoặc kiểm thử thành phần). Nhiều bài kiểm thử phi chức năng được dẫn xuất từ các bài kiểm thử chức năng vì chúng sử dụng cùng các bài kiểm thử chức năng đó, nhưng kiểm tra xem trong khi thực hiện chức năng, một ràng buộc phi chức năng có được thỏa mãn hay không (ví dụ: kiểm tra xem một chức năng có thực hiện trong một khoảng thời gian quy định hay không, hoặc một chức năng có thể chuyển đổi sang một nền tảng mới hay không). Việc phát hiện muộn các khiếm khuyết phi chức năng có thể tạo ra mối đe dọa nghiêm trọng cho sự thành công của dự án. Kiểm thử phi chức năng đôi khi cần một môi trường kiểm thử rất đặc thù, chẳng hạn như phòng thí nghiệm khả năng sử dụng *(usability lab)* cho kiểm thử khả năng sử dụng.

3. **Kiểm thử hộp đen (*Black-box testing*, xem mục 4.2):** Dựa trên đặc tả *(specification-based)* và dẫn xuất các bài kiểm thử từ tài liệu không liên quan đến cấu trúc bên trong của đối tượng kiểm thử. Mục tiêu chính của kiểm thử hộp đen là kiểm tra hành vi của hệ thống so với các đặc tả của nó.
4. **Kiểm thử hộp trắng (*White-box testing*, xem mục 4.3):** Dựa trên cấu trúc *(structure-based)* và dẫn xuất các bài kiểm thử từ việc triển khai hoặc cấu trúc bên trong của hệ thống (ví dụ: mã nguồn, kiến trúc, luồng công việc *workflows*, và luồng dữ liệu *data flows*). Mục tiêu chính của kiểm thử hộp trắng là bao phủ cấu trúc bên dưới bằng các bài kiểm thử tới một mức độ chấp nhận được.

Tất cả 4 loại kiểm thử nêu trên đều có thể áp dụng cho tất cả các cấp độ kiểm thử, mặc dù trọng tâm sẽ khác nhau ở mỗi cấp độ. Các kỹ thuật kiểm thử khác nhau có thể được sử dụng để dẫn xuất các điều kiện kiểm thử và các ca kiểm thử cho tất cả các loại kiểm thử đã nêu.

2.2.3. Kiểm thử Xác nhận và Kiểm thử Hồi quy (Confirmation Testing and Regression Testing)

FL-2.2.3 • K2

Các thay đổi thường được thực hiện đối với một thành phần hoặc hệ thống để nâng cấp nó bằng cách thêm một tính năng mới hoặc để sửa nó bằng cách loại bỏ một khiếm khuyết. Khi đó, việc kiểm thử cũng phải bao gồm **kiểm thử xác nhận** *(confirmation testing)* và **kiểm thử hồi quy** *(regression testing)*.

- **Kiểm thử xác nhận** *(Confirmation testing)*: Xác nhận rằng một khiếm khuyết ban đầu đã được sửa thành công. Tùy thuộc vào rủi ro, người ta có thể kiểm thử phiên bản đã sửa của phần mềm theo nhiều cách, bao gồm:
  - Thực thi tất cả các bài kiểm thử trước đó đã bị thất bại *(failed)* do khiếm khuyết đó, hoặc, ngoài ra bằng cách
  - Thêm các bài kiểm thử mới để bao phủ bất kỳ thay đổi nào cần thiết để sửa khiếm khuyết đó. Tuy nhiên, khi thời gian hoặc tiền bạc eo hẹp trong lúc sửa khiếm khuyết, kiểm thử xác nhận có thể chỉ giới hạn ở việc đơn thuần thực hiện lại các bước kiểm thử dùng để tái hiện hỏng hóc do khiếm khuyết đó gây ra và kiểm tra xem hỏng hóc đó không còn xảy ra nữa.
- **Kiểm thử hồi quy** *(Regression testing)*: Xác nhận rằng không có hệ quả bất lợi nào *(no adverse consequences)* bị gây ra bởi một thay đổi, bao gồm cả một bản sửa lỗi vốn đã được kiểm thử xác nhận. Những hệ quả bất lợi này có thể ảnh hưởng đến cùng thành phần nơi thay đổi được thực hiện, các thành phần khác trong cùng hệ thống, hoặc thậm chí các hệ thống được kết nối khác. Kiểm thử hồi quy có thể không chỉ giới hạn ở bản thân đối tượng kiểm thử mà còn có thể liên quan đến môi trường. Lời khuyên là trước tiên nên thực hiện **phân tích tác động** *(impact analysis)* để nhận biết phạm vi của kiểm thử hồi quy. Phân tích tác động cho thấy những phần nào của phần mềm có thể bị ảnh hưởng. Các bộ kiểm thử hồi quy *(regression test suites)* được chạy nhiều lần và nhìn chung số lượng ca kiểm thử hồi quy sẽ tăng lên sau mỗi vòng lặp hoặc bản phát hành, vì vậy **kiểm thử hồi quy là một ứng cử viên sáng giá cho việc tự động hóa** *(strong candidate for*

automation). Tự động hóa kiểm thử nên bắt đầu sớm trong dự án. Ở những nơi sử dụng CI, chẳng hạn như trong DevOps (xem mục 2.1.4), một thực hành tốt là bao gồm luôn cả các bài kiểm thử hồi quy tự động. Tùy thuộc vào tình huống, điều này có thể bao gồm các bài kiểm thử hồi quy ở các cấp độ kiểm thử khác nhau.

Kiểm thử xác nhận và/hoặc kiểm thử hồi quy cho đối tượng kiểm thử là cần thiết **ở tất cả các cấp độ kiểm thử** nếu các khiếm khuyết được sửa và/hoặc các thay đổi được thực hiện ở các cấp độ kiểm thử này.

2.3. Kiểm thử Bảo trì (Maintenance Testing)

FL-2.3.1 • K2

Có nhiều loại bảo trì khác nhau: có thể là bảo trì khắc phục lỗi *(corrective)*, thích ứng với các thay đổi trong môi trường *(adaptive)*, hoặc cải thiện hiệu suất hay khả năng bảo trì (xem ISO/IEC 14764 để biết chi tiết), vì vậy bảo trì có thể bao gồm cả các đợt phát hành/triển khai có kế hoạch *(planned releases/deployments)* và các đợt phát hành/triển khai không có kế hoạch (*unplanned releases/deployments* - các bản vá nóng *hot fixes*). Phân tích tác động *(impact analysis)* có thể được thực hiện trước khi một thay đổi được tiến hành, nhằm giúp quyết định xem có nên thực hiện thay đổi đó hay không, dựa trên các hệ quả tiềm tàng ở các khu vực khác của hệ thống. Việc kiểm thử các thay đổi đối với một hệ thống đang vận hành bao gồm cả việc đánh giá sự thành công của việc triển khai thay đổi đó lẫn việc kiểm tra các lỗi hồi quy có thể xảy ra ở các phần không thay đổi của hệ thống (vốn thường là phần lớn của hệ thống).

Phạm vi của kiểm thử bảo trì *(scope of maintenance testing)* thường phụ thuộc vào:

- Mức độ rủi ro của thay đổi *(The degree of risk of the change)*
- Kích thước của hệ thống hiện tại *(The size of the existing system)*
- Kích thước của thay đổi *(The size of the change)*

Các tác nhân kích hoạt *(triggers)* việc bảo trì và kiểm thử bảo trì có thể được phân loại như sau:

- **Các sửa đổi** *(Modifications)*: chẳng hạn như các nâng cấp có kế hoạch (*planned enhancements* - tức là theo đợt phát hành), các thay đổi khắc phục lỗi *(corrective changes)* hoặc các bản vá nóng *(hot fixes)*.
- **Nâng cấp hoặc di chuyển môi trường vận hành** *(Upgrades or migrations of the operational environment)*: chẳng hạn như từ nền tảng này sang nền tảng khác, có thể đòi hỏi các bài kiểm thử gắn với môi trường mới cũng như đối với phần mềm đã thay đổi, hoặc kiểm thử việc chuyển đổi dữ liệu *(data conversion)* khi dữ liệu từ một ứng dụng khác được di chuyển *(migrated)* vào hệ thống đang được bảo trì.
- **Ngừng hoạt động / Nghỉ hưu hệ thống** *(Retirement)*: chẳng hạn như khi một ứng dụng kết thúc vòng đời của nó *(reaches the end of its life)*. Khi một hệ thống ngừng hoạt động, điều này có thể đòi hỏi việc kiểm thử lưu trữ dữ liệu *(data archiving)* nếu yêu cầu thời gian lưu giữ dữ liệu dài. Việc kiểm thử các thủ tục khôi phục và truy xuất *(restore and retrieval procedures)* sau khi lưu trữ cũng có thể cần thiết trong trường hợp cần đến một số dữ liệu nhất định trong suốt thời gian lưu trữ.

CHƯƠNG 3

Kiểm thử Tĩnh (Static Testing)

🕑 Thời lượng phân bổ: 80 phút

K1 Bảng Từ Khóa Chuẩn ISTQB (Keywords) — Cần nhớ chính xác thuật ngữ gốc Tiếng Anh khi làm bài thi

anomaly	điểm bất thường	**dynamic testing**	kiểm thử động
formal review	duyệt chính thức/trang trọng	**informal review**	duyệt không chính thức
inspection	thanh tra - mức duyệt trang trọng nhất	**review**	duyệt/rà soát
static analysis	phân tích tĩnh	**static testing**	kiểm thử tĩnh
technical review	duyệt kỹ thuật	**walkthrough**	duyệt từng bước do tác giả dẫn dắt

Mục Tiêu Học Tập Trọng Tâm (Learning Objectives)

MÃ LO	MỨC NHẬN THỨC	NỘI DUNG YÊU CẦU TRONG ĐỀ THI
3.1 Cơ bản về Kiểm thử Tĩnh (Static Testing Basics)		
FL-3.1.1	K1 · Ghi nhớ	Nhận biết các loại sản phẩm công việc có thể được kiểm tra bằng kiểm thử tĩnh
FL-3.1.2	K2 · Hiểu	Giải thích giá trị của kiểm thử tĩnh
FL-3.1.3	K2 · Hiểu	So sánh và đối chiếu kiểm thử tĩnh và kiểm thử động
3.2 Phản hồi và Quy trình Duyệt (Feedback and Review Process)		
FL-3.2.1	K1 · Ghi nhớ	Xác định lợi ích của việc phản hồi sớm và thường xuyên từ các bên liên quan
FL-3.2.2	K2 · Hiểu	Tóm tắt các hoạt động của quy trình duyệt (*review process*)
FL-3.2.3	K1 · Ghi nhớ	Nhớ lại các trách nhiệm được giao cho các vai trò chính khi thực hiện duyệt
FL-3.2.4	K2 · Hiểu	So sánh và đối chiếu các loại duyệt (*review types*) khác nhau
FL-3.2.5	K1 · Ghi nhớ	Nhớ lại các yếu tố góp phần tạo nên một buổi duyệt thành công

3.1. Cơ bản về Kiểm thử Tĩnh (Static Testing Basics)

Trái ngược với kiểm thử động *(dynamic testing)*, trong kiểm thử tĩnh *(static testing)*, phần mềm đang được kiểm thử **không cần phải được thực thi** *(does not need to be executed)*. Mã nguồn, đặc tả quy trình, đặc tả kiến trúc hệ thống hoặc các sản phẩm công việc khác được đánh giá thông qua việc kiểm tra thủ công (ví dụ: *reviews*) hoặc với sự trợ giúp của một công cụ (ví dụ: phân tích tĩnh *static analysis*). Các mục tiêu kiểm thử bao gồm cải thiện chất lượng, phát hiện khiếm khuyết và đánh giá các đặc tính như tính dễ đọc *(readability)*, tính đầy đủ

(completeness), tính đúng đắn *(correctness)*, khả năng kiểm thử *(testability)* và tính nhất quán *(consistency)*. Kiểm thử tĩnh có thể được áp dụng cho cả xác minh *(verification)* và thẩm định *(validation)*.

Kiểm thử viên, đại diện nghiệp vụ (*Product Owner*, *business analyst*, v.v.) và lập trình viên làm việc cùng nhau trong các buổi lập bản đồ ví dụ *(example mappings)*, viết câu chuyện người dùng cộng tác *(collaborative user story writing)* và các phiên làm mịn backlog *(backlog refinement sessions)* để đảm bảo rằng các *user stories* và các sản phẩm công việc liên quan đáp ứng các tiêu chí đã định nghĩa, ví dụ: Định nghĩa sự Sẵn sàng (*Definition of Ready*, xem mục 5.1.3). Các kỹ thuật duyệt *(review techniques)* có thể được áp dụng để đảm bảo các *user stories* đầy đủ, dễ hiểu và bao gồm các tiêu chí chấp nhận có thể kiểm thử được *(testable acceptance criteria)*. Bằng cách đặt ra những câu hỏi đúng, kiểm thử viên khám phá, phản biện và giúp cải thiện các *user stories* được đề xuất.

Phân tích tĩnh *(Static analysis)* có thể xác định các vấn đề trước khi kiểm thử động trong khi thường đòi hỏi ít công sức hơn, vì không cần các ca kiểm thử *(no test cases are required)*, và các công cụ (xem Chương 6) thường được sử dụng. Phân tích tĩnh thường được tích hợp vào các khung CI (xem mục 2.1.4). Mặc dù phần lớn được dùng để phát hiện các khiếm khuyết mã nguồn cụ thể, phân tích tĩnh cũng được dùng để đánh giá khả năng bảo trì *(maintainability)* và tính bảo mật *(security)*. Công cụ kiểm tra chính tả *(spelling checkers)* và công cụ đánh giá tính dễ đọc *(readability tools)* là những ví dụ khác về công cụ phân tích tĩnh.

3.1.1. Các Sản phẩm Công việc có thể Kiểm tra bằng Kiểm thử Tĩnh — FL-3.1.1 • K1

Hầu như bất kỳ sản phẩm công việc nào cũng có thể được kiểm tra bằng kiểm thử tĩnh. Ví dụ bao gồm: tài liệu đặc tả yêu cầu, mã nguồn, kế hoạch kiểm thử, ca kiểm thử, các mục tồn đọng sản phẩm *(product backlog items)*, điều lệ kiểm thử *(test charters)*, tài liệu dự án, hợp đồng và các mô hình *(models)*.

- **Đối với Duyệt** *(Review)*: Bất kỳ sản phẩm công việc nào **có thể đọc và hiểu được** *(can be read and understood)* đều có thể là đối tượng của một buổi duyệt.
- **Đối với Phân tích tĩnh** *(Static analysis)*: Các sản phẩm công việc **cần có một cấu trúc** *(structure)* để dựa vào đó chúng có thể được kiểm tra (ví dụ: các mô hình, mã nguồn hoặc văn bản có cú pháp hình thức *formal syntax*).

Các sản phẩm công việc **không phù hợp** cho kiểm thử tĩnh bao gồm những sản phẩm khó diễn dịch bởi con người và không nên được phân tích bởi các công cụ (ví dụ: mã thực thi của bên thứ 3 *3rd party executable code* vì lý do pháp lý).

3.1.2. Giá trị của Kiểm thử Tĩnh (Value of Static Testing) — FL-3.1.2 • K2

Kiểm thử tĩnh có thể phát hiện các khiếm khuyết ngay trong những giai đoạn sớm nhất của SDLC, thực hiện đúng nguyên lý kiểm thử sớm (xem mục 1.3). Nó cũng có thể xác định những khiếm khuyết **không thể phát hiện được bằng kiểm thử động** (ví dụ: đoạn mã không thể chạm tới *unreachable code*, các mẫu thiết kế *design patterns* không được triển khai như mong muốn, các khiếm khuyết trong các sản phẩm công việc không thể thực thi).

Kiểm thử tĩnh cung cấp khả năng đánh giá chất lượng và xây dựng sự tự tin vào các sản phẩm công việc. Bằng cách xác minh các yêu cầu đã được ghi thành tài liệu, các bên liên quan cũng có thể đảm bảo rằng các yêu cầu này mô tả đúng nhu cầu thực tế của họ *(validation)*. Vì kiểm thử tĩnh có thể được thực hiện sớm trong SDLC, một sự hiểu biết chung *(shared understanding)* có thể được tạo ra giữa các bên liên quan tham gia. Giao tiếp cũng sẽ được cải thiện giữa các bên liên quan. Vì lý do này, người ta khuyến nghị nên lôi kéo nhiều bên liên quan đa dạng tham gia vào kiểm thử tĩnh.

Mặc dù việc thực hiện các buổi duyệt *(reviews)* có thể tốn kém chi phí, nhưng tổng chi phí dự án thường thấp hơn nhiều so với khi không thực hiện duyệt bởi vì cần ít thời gian và công sức hơn cho việc sửa các khiếm khuyết ở giai đoạn muộn hơn trong dự án.

Một số khiếm khuyết mã nguồn nhất định *(certain code defects)* có thể được phát hiện bằng phân tích tĩnh hiệu quả hơn *(more efficiently)* so với trong kiểm thử động, thường dẫn đến việc vừa có ít khiếm khuyết mã nguồn hơn vừa giảm tổng công sức phát triển chung.

3.1.3. Sự khác biệt giữa Kiểm thử Tĩnh và Kiểm thử Động

FL-3.1.3 • K2

Các thực hành kiểm thử tĩnh và kiểm thử động bổ trợ cho nhau *(complement each other)*. Chúng có các mục tiêu tương tự nhau, chẳng hạn như hỗ trợ phát hiện các khiếm khuyết trong các sản phẩm công việc (xem mục 1.1.1), nhưng cũng có một số điểm khác biệt, chẳng hạn như:

- Kiểm thử tĩnh và kiểm thử động (kèm phân tích hỏng hóc) đều có thể dẫn đến việc phát hiện các khiếm khuyết, tuy nhiên có một số loại khiếm khuyết chỉ có thể tìm thấy bằng kiểm thử tĩnh hoặc chỉ bằng kiểm thử động.
- **Kiểm thử tĩnh tìm thấy khiếm khuyết trực tiếp** ***(directly)***, trong khi **kiểm thử động gây ra các hỏng hóc** ***(causes failures)*** mà từ đó các khiếm khuyết liên quan mới được xác định thông qua phân tích tiếp theo.
- Kiểm thử tĩnh có thể dễ dàng phát hiện hơn các khiếm khuyết nằm trên các đường dẫn xuyên qua mã nguồn vốn hiếm khi được thực thi hoặc khó chạm tới bằng kiểm thử động.
- Kiểm thử tĩnh có thể được áp dụng cho các sản phẩm công việc không thực thi được ***(non-executable work products)***, trong khi kiểm thử động chỉ có thể áp dụng cho các sản phẩm công việc thực thi được ***(executable work products)***.
- Kiểm thử tĩnh có thể được dùng để đo lường các đặc tính chất lượng không phụ thuộc vào việc thực thi mã nguồn (ví dụ: khả năng bảo trì - ***maintainability***), trong khi kiểm thử động có thể được dùng để đo lường các đặc tính chất lượng phụ thuộc vào việc thực thi mã nguồn (ví dụ: hiệu suất hoạt động - ***performance efficiency***).

Các khiếm khuyết điển hình dễ tìm hơn và/hoặc rẻ hơn khi tìm thông qua kiểm thử tĩnh bao gồm:

- Khiếm khuyết trong các yêu cầu (ví dụ: sự không nhất quán, mơ hồ, mâu thuẫn, thiếu sót, không chính xác, trùng lặp).
- Khiếm khuyết thiết kế (ví dụ: cấu trúc cơ sở dữ liệu kém hiệu quả, chia module kém ***poor modularization***).
- Một số loại khiếm khuyết viết mã nhất định (ví dụ: biến có giá trị chưa xác định, biến chưa khai báo, mã không thể chạm tới ***unreachable code*** hoặc mã trùng lặp, độ phức tạp mã nguồn quá mức).
- Sự sai lệch so với các tiêu chuẩn (ví dụ: không tuân thủ quy ước đặt tên trong tiêu chuẩn viết mã).
- Đặc tả giao diện không chính xác (ví dụ: sai lệch về số lượng, kiểu hoặc thứ tự của các tham số).
- Các loại lỗ hổng bảo mật cụ thể (ví dụ: tràn bộ đệm ***buffer overflows***).
- Khoảng trống hoặc sự thiếu chính xác trong độ bao phủ cơ sở kiểm thử *(test basis coverage)* (ví dụ: thiếu các bài kiểm thử cho một tiêu chí chấp nhận).

3.2. Phản hồi và Quy trình Duyệt (Feedback and Review Process)

3.2.1. Lợi ích của Phản hồi Sớm và Thường xuyên từ các Bên liên quan

FL-3.2.1 • K1

Phản hồi sớm và thường xuyên cho phép truyền đạt sớm các vấn đề chất lượng tiềm ẩn. Nếu có ít sự tham gia của các bên liên quan trong suốt SDLC, sản phẩm đang được phát triển có thể không đáp ứng được tầm nhìn ban đầu hoặc hiện tại của các bên liên quan. Việc không chuyển giao được những gì bên liên quan mong muốn có thể dẫn đến việc làm lại tốn kém *(costly rework)*, trễ hạn *(missed deadlines)*, đổ lỗi cho nhau *(blame games)*, và thậm chí có thể dẫn đến thất bại hoàn toàn của dự án.

Phản hồi thường xuyên từ các bên liên quan xuyên suốt SDLC có thể ngăn ngừa những hiểu lầm về yêu cầu và đảm bảo rằng các thay đổi đối với yêu cầu được hiểu rõ và được triển khai sớm hơn. Điều này giúp nhóm phát triển cải thiện sự hiểu biết của họ về những gì họ đang xây dựng. Nó cho phép họ tập trung vào những tính năng mang lại nhiều giá trị nhất cho các bên liên quan và có tác động tích cực nhất đến các rủi ro đã xác định.

3.2.2. Các Hoạt động của Quy trình Duyệt (Review Process Activities)

FL-3.2.2 • K2

Tiêu chuẩn ISO/IEC 20246 định nghĩa một quy trình duyệt chung *(generic review process)* cung cấp một khung làm việc có cấu trúc nhưng linh hoạt, từ đó một quy trình duyệt cụ thể có thể được điều chỉnh cho phù hợp với một tình huống cụ thể. Nếu buổi duyệt được yêu cầu càng trang trọng/chính thức *(more formal)*, thì càng cần nhiều nhiệm vụ được mô tả cho các hoạt động khác nhau đó.

Kích thước của nhiều sản phẩm công việc khiến chúng quá lớn để có thể bao quát chỉ bằng một buổi duyệt duy nhất. Quy trình duyệt có thể được gọi thực hiện nhiều lần *(invoked multiple times)* để hoàn thành việc duyệt cho toàn bộ sản phẩm công việc.

5 hoạt động trong quy trình duyệt là:

1. **Lập kế hoạch** ***(Planning)*****:** Trong giai đoạn lập kế hoạch, phạm vi của buổi duyệt — bao gồm mục đích, sản phẩm công việc cần duyệt, các đặc tính chất lượng cần đánh giá, các khu vực cần tập trung, tiêu chí kết thúc *(exit criteria)*, thông tin hỗ trợ như các tiêu chuẩn, công sức và khung thời gian cho buổi duyệt — sẽ được xác định.
2. **Khởi động duyệt** ***(Review initiation)*****:** Trong quá trình khởi động duyệt, mục tiêu là đảm bảo rằng tất cả mọi người và mọi thứ liên quan đều đã sẵn sàng để bắt đầu buổi duyệt. Điều này bao gồm việc đảm bảo rằng mọi người tham gia đều có quyền truy cập vào sản phẩm công việc đang được duyệt, hiểu rõ vai trò và trách nhiệm của mình, và nhận được mọi thứ cần thiết để thực hiện việc duyệt.
3. **Duyệt cá nhân** ***(Individual review)*****:** Mỗi người duyệt *(reviewer)* thực hiện một đợt duyệt cá nhân để đánh giá chất lượng của sản phẩm công việc đang được duyệt, và để xác định các điểm bất thường *(anomalies)*, các khuyến nghị *(recommendations)*, và các câu hỏi bằng cách áp dụng một hoặc nhiều kỹ thuật duyệt (ví dụ: duyệt dựa trên danh sách kiểm tra ***checklist-based reviewing***, duyệt dựa trên kịch bản ***scenario-based reviewing***). Tiêu chuẩn ISO/IEC 20246 cung cấp chiều sâu hơn về các kỹ thuật duyệt khác nhau. Những người duyệt ghi nhật ký *(log)* tất cả các điểm bất thường, khuyến nghị và câu hỏi mà họ đã xác định được.
4. **Giao tiếp và phân tích** ***(Communication and analysis)*****:** Vì các điểm bất thường *(anomalies)* được xác định trong quá trình duyệt không nhất thiết đều là khiếm khuyết *(defects)*, tất cả các điểm bất thường này cần được phân tích và thảo luận. Đối với mỗi điểm bất thường, cần đưa ra quyết định về trạng thái của nó, người chịu trách nhiệm *(ownership)* và các hành động cần thiết. Việc này thường được thực hiện trong một cuộc họp duyệt *(review meeting)*, trong đó những người tham gia cũng quyết định mức chất lượng của sản phẩm công

việc được duyệt là gì và những hành động theo dõi *(follow-up actions)* nào là cần thiết. Một buổi duyệt theo dõi *(follow-up review)* có thể được yêu cầu để hoàn tất các hành động.

5. **Sửa lỗi và báo cáo** *(Fixing and reporting)***:** Đối với mỗi khiếm khuyết *(defect)*, một báo cáo khiếm khuyết *(defect report)* cần được tạo ra để có thể theo dõi các hành động khắc phục. Một khi đạt được các tiêu chí kết thúc *(exit criteria)*, sản phẩm công việc có thể được chấp nhận. Kết quả duyệt sẽ được báo cáo.

3.2.3. Vai trò và Trách nhiệm trong các Buổi duyệt (Roles and Responsibilities in Reviews)

FL-3.2.3 • K1

Các buổi duyệt có sự tham gia của nhiều bên liên quan khác nhau, những người có thể đảm nhận một số vai trò. **6 vai trò chính và trách nhiệm của họ là:**

- **Người quản lý** *(Manager)***:** Quyết định cái gì sẽ được duyệt và cung cấp các nguồn lực, chẳng hạn như nhân sự và thời gian cho việc duyệt.
- **Tác giả** *(Author)***:** Tạo ra và sửa sản phẩm công việc đang được duyệt.
- **Người điều phối (***Moderator* **- còn được gọi là** *facilitator***):** Đảm bảo việc điều hành hiệu quả các cuộc họp duyệt, bao gồm hòa giải *(mediation)*, quản lý thời gian, và tạo ra một môi trường duyệt an toàn nơi mọi người đều có thể phát biểu tự do.
- **Thư ký (***Scribe* **- còn được gọi là** *recorder***):** Tập hợp các điểm bất thường từ những người duyệt và ghi chép lại thông tin buổi duyệt, chẳng hạn như các quyết định và các điểm bất thường mới được tìm thấy trong cuộc họp duyệt.
- **Người duyệt** *(Reviewer)***:** Thực hiện việc duyệt. Người duyệt có thể là một người đang làm việc trong dự án, một chuyên gia về lĩnh vực chuyên môn *(subject matter expert - SME)*, hoặc bất kỳ bên liên quan nào khác.
- **Trưởng nhóm duyệt** *(Review leader)***:** Chịu trách nhiệm tổng thể về việc duyệt như quyết định ai sẽ tham gia, và tổ chức thời gian cũng như địa điểm diễn ra buổi duyệt.

Các vai trò chi tiết hơn khác cũng có thể có, như được mô tả trong tiêu chuẩn ISO/IEC 20246.

3.2.4. Các Loại Duyệt (Review Types)

FL-3.2.4 • K2

Tồn tại nhiều loại duyệt trải dài từ duyệt không chính thức *(informal reviews)* đến duyệt chính thức *(formal reviews)*. Mức độ trang trọng/chính thức *(level of formality)* cần thiết phụ thuộc vào các yếu tố như SDLC đang được tuân theo, mức độ trưởng thành của quy trình phát triển, mức độ quan trọng và độ phức tạp của sản phẩm công việc đang được duyệt, các yêu cầu pháp lý hoặc quy định chế tài, và nhu cầu về một dấu vết kiểm toán *(audit trail)*. Cùng một sản phẩm công việc có thể được duyệt bằng các loại duyệt khác nhau, ví dụ: đầu tiên là một buổi duyệt không chính thức và sau đó là một buổi duyệt chính thức hơn.

Việc lựa chọn đúng loại duyệt là chìa khóa để đạt được các mục tiêu duyệt theo yêu cầu (xem mục 3.2.5). Việc lựa chọn không chỉ dựa trên các mục tiêu, mà còn dựa trên các yếu tố như nhu cầu dự án, nguồn lực sẵn có, loại sản phẩm công việc và rủi ro, lĩnh vực kinh doanh, và văn hóa công ty.

4 loại duyệt thường được sử dụng là:

1. **Duyệt không chính thức** *(Informal review)***:** Duyệt không chính thức không tuân theo một quy trình đã được định nghĩa và không đòi hỏi một đầu ra được ghi thành tài liệu chính thức. Mục tiêu chính là phát hiện các điểm bất thường *(detecting anomalies)*.

2. **Duyệt từng bước *(Walkthrough)*:** Một buổi ***walkthrough***, **được dẫn dắt bởi tác giả *(led by the author)***, có thể phục vụ nhiều mục tiêu, chẳng hạn như đánh giá chất lượng và xây dựng sự tự tin vào sản phẩm công việc, đào tạo/hướng dẫn cho những người duyệt ***(educating reviewers)***, đạt được sự đồng thuận, tạo ra các ý tưởng mới, thúc đẩy và tạo điều kiện cho các tác giả cải thiện cũng như phát hiện các điểm bất thường. Những người duyệt có thể thực hiện việc duyệt cá nhân trước buổi ***walkthrough***, nhưng điều này **không bắt buộc**.
3. **Duyệt kỹ thuật *(Technical Review)*:** Một buổi duyệt kỹ thuật được thực hiện bởi những người duyệt có trình độ kỹ thuật ***(technically qualified reviewers)*** và **được dẫn dắt bởi một người điều phối *(led by a moderator)***. Mục tiêu của duyệt kỹ thuật là đạt được sự đồng thuận và đưa ra các quyết định liên quan đến một vấn đề kỹ thuật, nhưng đồng thời cũng để phát hiện các điểm bất thường, đánh giá chất lượng và xây dựng sự tự tin vào sản phẩm công việc, tạo ra các ý tưởng mới, và thúc đẩy cũng như tạo điều kiện cho các tác giả cải thiện.
4. **Thanh tra *(Inspection)*:** Vì thanh tra ***(inspections)*** là **loại duyệt trang trọng/chính thức nhất *(most formal type of review)***, chúng tuân theo đầy đủ quy trình duyệt chung (xem mục 3.2.2). Mục tiêu chính là tìm ra số lượng tối đa các điểm bất thường ***(find the maximum number of anomalies)***. Các mục tiêu khác là đánh giá chất lượng, xây dựng sự tự tin vào sản phẩm công việc, và thúc đẩy cũng như tạo điều kiện cho các tác giả cải thiện. Các số liệu đo lường ***(metrics)*** được thu thập và sử dụng để cải tiến SDLC, bao gồm cả quy trình thanh tra. **Trong thanh tra *(inspections)*, tác giả KHÔNG ĐƯỢC đóng vai trò là trưởng nhóm duyệt *(review leader)* hoặc thư ký *(scribe)*** *(và theo ngầm định chuẩn ISTQB, cũng không làm **moderator**)*.

3.2.5. Các Yếu tố Thành công cho các Buổi duyệt (Success Factors for Reviews) FL-3.2.5 • K1

Có một số yếu tố quyết định sự thành công của các buổi duyệt, bao gồm:

- Xác định các mục tiêu rõ ràng và các tiêu chí kết thúc có thể đo lường được ***(measurable exit criteria)***. **Việc đánh giá cá nhân những người tham gia *(Evaluation of participants)* không bao giờ được phép là một mục tiêu.**
- Chọn loại duyệt phù hợp để đạt được các mục tiêu đã đề ra, và phù hợp với loại sản phẩm công việc, những người tham gia duyệt, nhu cầu dự án và ngữ cảnh.
- Thực hiện duyệt trên các phần nhỏ ***(small chunks)***, để những người duyệt không bị mất tập trung trong quá trình duyệt cá nhân và/hoặc cuộc họp duyệt (khi được tổ chức).
- Cung cấp phản hồi từ các buổi duyệt cho các bên liên quan và các tác giả để họ có thể cải thiện sản phẩm và các hoạt động của mình (xem mục 3.2.1).
- Cung cấp đủ thời gian cho những người tham gia để chuẩn bị cho buổi duyệt.
- Sự hỗ trợ từ cấp quản lý ***(management)*** đối với quy trình duyệt.
- Biến việc duyệt thành một phần trong văn hóa của tổ chức, nhằm thúc đẩy việc học hỏi và cải tiến quy trình.
- Đào tạo đầy đủ cho tất cả những người tham gia để họ biết cách hoàn thành vai trò của mình.
- Điều phối tốt các cuộc họp ***(Facilitating meetings)***.

CHƯƠNG 4

🕑 Thời lượng phân bổ: 390 phút

Phân tích và Thiết kế Kiểm thử (Test Analysis and Design)

K1 Bảng Từ Khóa Chuẩn ISTQB (Keywords) — Cần nhớ chính xác thuật ngữ gốc Tiếng Anh khi làm bài thi

acceptance criteria	tiêu chí chấp nhận	**acceptance test-driven development**	phát triển hướng kiểm thử chấp nhận - ATDD
black-box test technique	kỹ thuật kiểm thử hộp đen	**boundary value analysis**	phân tích giá trị biên - BVA
branch coverage	độ bao phủ nhánh	**checklist-based testing**	kiểm thử dựa trên danh sách kiểm tra
collaboration-based test approach	cách tiếp cận kiểm thử dựa trên sự cộng tác	**coverage**	độ bao phủ
coverage item	mục bao phủ	**decision table testing**	kiểm thử bảng quyết định
equivalence partitioning	phân vùng tương đương - EP	**error guessing**	đoán lỗi
experience-based test technique	kỹ thuật kiểm thử dựa trên kinh nghiệm	**exploratory testing**	kiểm thử khám phá
state transition testing	kiểm thử chuyển đổi trạng thái	**statement coverage**	độ bao phủ câu lệnh
test technique	kỹ thuật kiểm thử	**white-box test technique**	kỹ thuật kiểm thử hộp trắng

Mục Tiêu Học Tập Trọng Tâm (Learning Objectives)

MÃ LO	MỨC NHẬN THỨC	NỘI DUNG YÊU CẦU TRONG ĐỀ THI
4.1 Tổng quan về Kỹ thuật Kiểm thử (Test Techniques Overview)		
FL-4.1.1	K2 · Hiểu	Phân biệt các kỹ thuật kiểm thử hộp đen, kỹ thuật kiểm thử hộp trắng và kỹ thuật kiểm thử dựa trên kinh nghiệm
4.2 Các Kỹ thuật Kiểm thử Hộp đen (Black-box Test Techniques)		
FL-4.2.1	K3 · Áp dụng	**Áp dụng** phân vùng tương đương (*equivalence partitioning*) để dẫn xuất các ca kiểm thử

MÃ LO	MỨC NHẬN THỨC	NỘI DUNG YÊU CẦU TRONG ĐỀ THI
FL-4.2.2	K3 · Áp dụng	**Áp dụng** phân tích giá trị biên (***boundary value analysis***) để dẫn xuất các ca kiểm thử
FL-4.2.3	K3 · Áp dụng	**Áp dụng** kiểm thử bảng quyết định (***decision table testing***) để dẫn xuất các ca kiểm thử
FL-4.2.4	K3 · Áp dụng	**Áp dụng** kiểm thử chuyển đổi trạng thái (***state transition testing***) để dẫn xuất các ca kiểm thử
4.3 Các Kỹ thuật Kiểm thử Hộp trắng (White-box Test Techniques)		
FL-4.3.1	K2 · Hiểu	Giải thích kiểm thử câu lệnh (***statement testing***)
FL-4.3.2	K2 · Hiểu	Giải thích kiểm thử nhánh (***branch testing***)
FL-4.3.3	K2 · Hiểu	Giải thích giá trị của kiểm thử hộp trắng (***white-box testing***)
4.4 Các Kỹ thuật Kiểm thử Dựa trên Kinh nghiệm (Experience-based Test Techniques)		
FL-4.4.1	K2 · Hiểu	Giải thích kỹ thuật đoán lỗi (***error guessing***)
FL-4.4.2	K2 · Hiểu	Giải thích kiểm thử khám phá (***exploratory testing***)
FL-4.4.3	K2 · Hiểu	Giải thích kiểm thử dựa trên danh sách kiểm tra (***checklist-based testing***)
4.5 Các Cách tiếp cận Kiểm thử Dựa trên sự Cộng tác (Collaboration-based Test Approaches)		
FL-4.5.1	K2 · Hiểu	Giải thích cách viết câu chuyện người dùng (***user stories***) với sự cộng tác của lập trình viên và đại diện nghiệp vụ
FL-4.5.2	K2 · Hiểu	Phân loại các tùy chọn khác nhau để viết tiêu chí chấp nhận (***acceptance criteria***)
FL-4.5.3	K3 · Áp dụng	**Áp dụng** phát triển hướng kiểm thử chấp nhận (***ATDD***) để dẫn xuất các ca kiểm thử

4.1. Tổng quan về Kỹ thuật Kiểm thử (Test Techniques Overview) FL-4.1.1 • K2

Các kỹ thuật kiểm thử *(Test techniques)* hỗ trợ kiểm thử viên trong việc phân tích kiểm thử (***test analysis*** - kiểm thử cái gì ***what to test***) và trong thiết kế kiểm thử (***test design*** - kiểm thử như thế nào ***how to test***). Các kỹ thuật kiểm thử giúp phát triển một tập hợp các ca kiểm thử tương đối nhỏ nhưng đủ *(relatively small, but sufficient)* một cách có hệ thống. Các kỹ thuật kiểm thử cũng giúp kiểm thử viên xác định các điều kiện kiểm thử *(test conditions)*, xác định các mục bao phủ *(coverage items)*, và xác định dữ liệu kiểm thử *(test data)* trong quá trình phân tích và thiết kế kiểm thử. Thông tin thêm về các kỹ thuật kiểm thử có thể tìm thấy trong tiêu chuẩn ISO/IEC/IEEE 29119-4, và trong (Beizer 1990, Craig 2002, Copeland 2004, Koomen 2006, Jorgensen 2014, Ammann 2016, Forgács 2019).

Trong đề cương này, các kỹ thuật kiểm thử được phân loại thành **hộp đen** *(black-box)*, **hộp trắng** *(white-box)*, và **dựa trên kinh nghiệm** *(experience-based)*:

- **Các kỹ thuật kiểm thử hộp đen (*Black-box test techniques* - còn gọi là kỹ thuật dựa trên đặc tả *specification-based techniques*):** Dựa trên việc phân tích hành vi đã được quy định *(specified behavior)* của

đối tượng kiểm thử mà không tham chiếu đến cấu trúc bên trong của nó. Do đó, các ca kiểm thử độc lập với cách phần mềm được triển khai/viết mã. Hệ quả là, nếu việc triển khai thay đổi, nhưng hành vi yêu cầu vẫn giữ nguyên, thì các ca kiểm thử đó vẫn hữu ích.

- **Các kỹ thuật kiểm thử hộp trắng (*White-box test techniques* - còn gọi là kỹ thuật dựa trên cấu trúc *structure-based techniques*):** Dựa trên việc phân tích cấu trúc bên trong và quá trình xử lý của đối tượng kiểm thử. Vì các ca kiểm thử phụ thuộc vào cách phần mềm được thiết kế, chúng chỉ có thể được tạo ra **sau khi** có thiết kế hoặc việc triển khai của đối tượng kiểm thử.
- **Các kỹ thuật kiểm thử dựa trên kinh nghiệm *(Experience-based test techniques)*:** Sử dụng hiệu quả kiến thức và kinh nghiệm của kiểm thử viên để thiết kế và triển khai các ca kiểm thử. Tính hiệu quả của các kỹ thuật kiểm thử này phụ thuộc rất nhiều vào kỹ năng của kiểm thử viên. Các kỹ thuật kiểm thử dựa trên kinh nghiệm có thể phát hiện các khiếm khuyết có thể bị bỏ sót khi sử dụng kỹ thuật kiểm thử hộp đen và kỹ thuật kiểm thử hộp trắng. Do đó, các kỹ thuật kiểm thử dựa trên kinh nghiệm mang tính bổ trợ *(complementary)* cho các kỹ thuật kiểm thử hộp đen và hộp trắng.

4.2. Các Kỹ thuật Kiểm thử Hộp đen (Black-Box Test Techniques)

Các kỹ thuật kiểm thử hộp đen thường được sử dụng và thảo luận trong các phần tiếp theo là:

- Phân vùng tương đương *(Equivalence Partitioning - EP)*
- Phân tích giá trị biên *(Boundary Value Analysis - BVA)*
- Kiểm thử bảng quyết định *(Decision Table Testing)*
- Kiểm thử chuyển đổi trạng thái *(State Transition Testing)*

4.2.1. Phân vùng Tương đương (Equivalence Partitioning - EP) FL-4.2.1 • K3

Phân vùng tương đương *(Equivalence Partitioning - EP)* chia dữ liệu thành các phân vùng (được gọi là các phân vùng tương đương *equivalence partitions*) dựa trên kỳ vọng rằng tất cả các phần tử của một phân vùng nhất định sẽ được xử lý theo cùng một cách bởi đối tượng kiểm thử. Lý thuyết đằng sau kỹ thuật này là nếu một ca kiểm thử, kiểm thử một giá trị từ một phân vùng tương đương, phát hiện ra một khiếm khuyết, thì khiếm khuyết này cũng sẽ được phát hiện bởi các ca kiểm thử kiểm thử bất kỳ giá trị nào khác từ cùng phân vùng đó. Do đó, **một bài kiểm thử cho mỗi phân vùng là đủ** *(one test for each partition is sufficient)*.

Các phân vùng tương đương có thể được xác định cho bất kỳ phần tử dữ liệu nào liên quan đến đối tượng kiểm thử, bao gồm đầu vào *(inputs)*, đầu ra *(outputs)*, các hạng mục cấu hình *(configuration items)*, các giá trị nội bộ *(internal values)*, các giá trị liên quan đến thời gian *(time-related values)*, và các tham số giao diện *(interface parameters)*. Các phân vùng có thể là liên tục hoặc rời rạc *(continuous or discrete)*, có thứ tự hoặc không có thứ tự *(ordered or unordered)*, hữu hạn hoặc vô hạn *(finite or infinite)*. **Các phân vùng không được giao nhau/chồng lấp** *(must not overlap)* **và phải là các tập hợp khác rỗng** *(non-empty sets)*.

Đối với các hạng mục kiểm thử *(test items)* đơn giản, EP có thể dễ dàng, nhưng trong thực tế, việc hiểu cách đối tượng kiểm thử sẽ đối xử với các giá trị khác nhau thường phức tạp. Do đó, việc phân vùng cần được thực hiện cẩn thận.

Một phân vùng chứa các giá trị hợp lệ được gọi là **phân vùng hợp lệ** *(valid partition)*. Một phân vùng chứa các giá trị không hợp lệ được gọi là **phân vùng không hợp lệ** *(invalid partition)*. Định nghĩa về giá trị hợp lệ và không hợp lệ có thể khác nhau giữa các nhóm và tổ chức. Ví dụ, các giá trị hợp lệ có thể được hiểu là những giá trị

phải được xử lý bởi đối tượng kiểm thử hoặc là những giá trị mà đặc tả có định nghĩa cách xử lý chúng. Các giá trị không hợp lệ có thể được hiểu là những giá trị phải bị bỏ qua hoặc bị từ chối bởi đối tượng kiểm thử hoặc là những giá trị mà không có cách xử lý nào được định nghĩa trong đặc tả của đối tượng kiểm thử. *(Lưu ý từ Errata v4.0.1 trang 73: các phân vùng tương đương không hợp lệ nên được kiểm thử riêng lẻ/cô lập **tested in isolation** để tránh hiện tượng che khuất khiếm khuyết **defect masking**).*

Trong EP, **các mục bao phủ *(coverage items)* chính là các phân vùng tương đương**. Để đạt được độ bao phủ 100% với kỹ thuật kiểm thử này, các ca kiểm thử phải thực thi tất cả các phân vùng đã xác định (bao gồm cả các phân vùng không hợp lệ) bằng cách bao phủ mỗi phân vùng ít nhất một lần. Công thức tính độ bao phủ:

CÔNG THỨC TÍNH ĐỘ BAO PHỦ PHÂN VÙNG TƯƠNG ĐƯƠNG (EP COVERAGE)

Coverage (%) = Số phân vùng được thực thi bởi ít nhất 1 test case/Tổng số phân vùng đã xác định × 100%

Nhiều hạng mục kiểm thử *(test items)* bao gồm nhiều tập hợp phân vùng (***multiple sets of partitions***, ví dụ: các hạng mục kiểm thử có nhiều hơn một tham số đầu vào), nghĩa là một ca kiểm thử sẽ bao phủ các phân vùng từ các tập hợp phân vùng khác nhau. Tiêu chí bao phủ đơn giản nhất trong trường hợp có nhiều tập hợp phân vùng được gọi là **Bao phủ Mỗi Lựa chọn** ***(Each Choice coverage)*** (Ammann 2016). ***Each Choice coverage*** đòi hỏi các ca kiểm thử phải thực thi mỗi phân vùng từ mỗi tập hợp phân vùng ít nhất một lần. ***Each Choice coverage*** không tính đến sự kết hợp ***(combinations)*** của các phân vùng.

4.2.2. Phân tích Giá trị Biên (Boundary Value Analysis - BVA) FL-4.2.2 • K3

Phân tích giá trị biên ***(Boundary Value Analysis - BVA)*** là một kỹ thuật kiểm thử dựa trên việc thực thi các ranh giới/biên ***(boundaries)*** của các phân vùng tương đương. Do đó, **BVA chỉ có thể được sử dụng cho các phân vùng có thứ tự** ***(ordered partitions)***. Giá trị nhỏ nhất ***(minimum)*** và lớn nhất ***(maximum)*** của một phân vùng là các **giá trị biên** ***(boundary values)*** của nó. Trong trường hợp của BVA, nếu hai phần tử thuộc cùng một phân vùng, tất cả các phần tử nằm giữa chúng cũng phải thuộc về phân vùng đó.

BVA tập trung vào các giá trị biên của các phân vùng bởi vì các lập trình viên có nhiều khả năng mắc sai sót ***(errors)*** hơn với các giá trị biên này. Các khiếm khuyết điển hình được tìm thấy bởi BVA nằm ở nơi các ranh giới được lập trình bị đặt sai vị trí lên trên hoặc xuống dưới so với vị trí dự kiến của chúng, hoặc bị bỏ sót hoàn toàn.

Đề cương này đề cập đến 2 phiên bản của BVA: **BVA 2 giá trị** ***(2-value BVA)*** và **BVA 3 giá trị** ***(3-value BVA)***. Chúng khác nhau về số lượng mục bao phủ ***(coverage items)*** trên mỗi biên cần được thực thi để đạt độ bao phủ 100%:

- **Trong BVA 2 giá trị** ***(2-value BVA)*** (Craig 2002, Myers 2011): đối với mỗi giá trị biên có **2 mục bao phủ** ***(two coverage items)***: chính giá trị biên này và **giá trị lân cận gần nhất của nó thuộc về phân vùng liền kề** ***(its closest neighbor belonging to the adjacent partition)***. Để đạt được độ bao phủ 100% với BVA 2 giá trị, các ca kiểm thử phải thực thi tất cả các mục bao phủ, tức là tất cả các giá trị biên đã xác định. Độ bao phủ được đo bằng số lượng giá trị biên đã được thực thi, chia cho tổng số giá trị biên đã xác định, và được biểu thị bằng phần trăm.
- **Trong BVA 3 giá trị** ***(3-value BVA)*** (Koomen 2006, O'Regan 2019): đối với mỗi giá trị biên có **3 mục bao phủ** ***(three coverage items)***: chính giá trị biên này và **cả hai giá trị lân cận hai bên của nó** ***(both its neighbors)***. Do đó, trong BVA 3 giá trị, một số mục bao phủ có thể không phải là giá trị biên (mà là giá trị nằm bên trong phân vùng sát biên). Để đạt được độ bao phủ 100% với BVA 3 giá trị, các ca kiểm thử phải thực thi tất cả các

mục bao phủ, tức là các giá trị biên đã xác định và các giá trị lân cận của chúng. Độ bao phủ được đo bằng số lượng giá trị biên và các giá trị lân cận của chúng được thực thi, chia cho tổng số giá trị biên đã xác định và các giá trị lân cận của chúng, và được biểu thị bằng phần trăm.

BVA 3 giá trị chặt chẽ/nghiêm ngặt hơn ***(more rigorous)*** **BVA 2 giá trị** vì nó có thể phát hiện ra các khiếm khuyết bị BVA 2 giá trị bỏ sót. Ví dụ: nếu câu lệnh quyết định `if (x <= 10) ...` bị lập trình sai thành `if (x = 10) ...`, không có dữ liệu kiểm thử nào dẫn xuất từ BVA 2 giá trị (`x = 10`, `x = 11`) có thể phát hiện ra khiếm khuyết này. Tuy nhiên, `x = 9`, được dẫn xuất từ BVA 3 giá trị, có khả năng phát hiện ra nó.

4.2.3. Kiểm thử Bảng Quyết định (Decision Table Testing) FL-4.2.3 • K3

Bảng quyết định *(Decision tables)* được sử dụng để kiểm thử việc triển khai các yêu cầu quy định cách thức mà các tổ hợp điều kiện khác nhau *(combinations of conditions)* dẫn đến các kết quả/hành động khác nhau *(different outcomes)*. Bảng quyết định là một cách hiệu quả để ghi lại các logic phức tạp, chẳng hạn như các quy tắc nghiệp vụ *(business rules)*.

Khi tạo bảng quyết định, các điều kiện *(conditions)* và các hành động kết quả *(resulting actions)* của hệ thống được xác định. Chúng tạo thành các **hàng** *(rows)* của bảng. Mỗi **cột** *(column)* tương ứng với một **quy tắc quyết định** *(decision rule)* xác định một tổ hợp duy nhất của các điều kiện, cùng với các hành động đi kèm:

- **Bảng quyết định mục nhập giới hạn** ***(Limited-entry decision tables)*****:** tất cả các giá trị của điều kiện và hành động (ngoại trừ những cái không liên quan hoặc bất khả thi; xem bên dưới) được biểu thị dưới dạng giá trị Boolean (`true` hoặc `false`).
- **Bảng quyết định mục nhập mở rộng** ***(Extended-entry decision tables)*****:** một số hoặc tất cả các điều kiện và hành động cũng có thể nhận nhiều giá trị (ví dụ: các khoảng số, các phân vùng tương đương, các giá trị rời rạc).

Ký hiệu chuẩn ***(Notation)*****:**

- Đối với điều kiện *(Conditions)*:
  - `"T"` (`true`): điều kiện được thỏa mãn.
  - `"F"` (`false`): điều kiện không được thỏa mãn.
  - `"–"`: giá trị của điều kiện không liên quan/không ảnh hưởng *(irrelevant)* đến kết quả hành động.
  - `"N/A"`: điều kiện là bất khả thi *(infeasible)* đối với một quy tắc nhất định.
- Đối với hành động *(Actions)*:
  - `"X"`: hành động phải xảy ra.
  - Để trống *(Blank)*: hành động không được xảy ra. (Các ký hiệu khác cũng có thể được sử dụng).

Một bảng quyết định đầy đủ *(full decision table)* có đủ số cột để bao phủ mọi tổ hợp điều kiện (với *n* điều kiện Boolean sẽ có 2^n cột). Bảng có thể được **đơn giản hóa** *(simplified)* bằng cách xóa các cột chứa các tổ hợp điều kiện bất khả thi *(infeasible)*. Bảng cũng có thể được **rút gọn/tối thiểu hóa (*minimized*/*collapsed*)** bằng cách gộp các cột mà trong đó một số điều kiện không ảnh hưởng đến kết quả thành một cột duy nhất (dùng dấu `–`). Các thuật toán tối thiểu hóa bảng quyết định nằm ngoài phạm vi của đề cương này.

Trong kiểm thử bảng quyết định, **các mục bao phủ *(coverage items)* là các cột chứa các tổ hợp điều kiện khả thi *(columns containing feasible combinations of conditions)***. Để đạt được độ bao phủ 100% với kỹ thuật này, các ca kiểm thử phải thực thi tất cả các cột khả thi này.

> **CÔNG THỨC TÍNH ĐỘ BAO PHỦ BẢNG QUYẾT ĐỊNH (DECISION TABLE COVERAGE)**
>
> **Coverage (%)** = Số cột khả thi đã được thực thi/Tổng số cột khả thi (feasible columns) × 100%

Điểm mạnh của kiểm thử bảng quyết định là nó cung cấp một cách tiếp cận có hệ thống để xác định tất cả các tổ hợp điều kiện, mà một số trong đó có thể bị bỏ sót nếu làm cách khác. Nó cũng giúp tìm ra bất kỳ khoảng trống *(gaps)* hoặc sự mâu thuẫn *(contradictions)* nào trong các yêu cầu. Nếu có nhiều điều kiện, việc thực thi tất cả các quy tắc quyết định có thể tốn nhiều thời gian, vì số lượng quy tắc tăng theo cấp số nhân *(exponentially)* với số lượng điều kiện. Trong trường hợp đó, để giảm số lượng quy tắc cần thực thi, một bảng quyết định rút gọn *(minimized decision table)* hoặc một cách tiếp cận dựa trên rủi ro *(risk-based approach)* có thể được sử dụng.

4.2.4. Kiểm thử Chuyển đổi Trạng thái (State Transition Testing) FL-4.2.4 • K3

Một **biểu đồ trạng thái** *(state diagram)* mô hình hóa hành vi của một hệ thống bằng cách hiển thị các trạng thái có thể có *(possible states)* và các chuyển đổi trạng thái hợp lệ *(valid state transitions)* của nó. Một sự chuyển đổi *(transition)* được khởi phát bởi một **sự kiện** *(event)*, và có thể được bổ sung điều kiện bởi một **điều kiện canh/bảo vệ** *(guard condition)*. Các chuyển đổi được giả định là tức thời và đôi khi có thể dẫn đến việc phần mềm thực hiện một **hành động** *(action)*. Cú pháp gán nhãn chuyển đổi phổ biến như sau: `event [guard condition] / action` Điều kiện canh *(guard conditions)* và hành động *(actions)* có thể được lược bỏ nếu chúng không tồn tại hoặc không liên quan đối với kiểm thử viên.

Một **bảng trạng thái** *(state table)* là một mô hình tương đương với biểu đồ trạng thái. Các **hàng** *(rows)* của nó đại diện cho các **trạng thái** *(states)*, và các **cột** *(columns)* của nó đại diện cho các **sự kiện (***events* - cùng với các điều kiện canh nếu có). Các mục trong bảng (các ô - *cells*) đại diện cho các chuyển đổi *(transitions)*, và chứa trạng thái đích *(target state)*, cũng như các hành động kết quả nếu được định nghĩa. **Trái ngược với biểu đồ trạng thái, bảng trạng thái hiển thị rõ ràng các chuyển đổi không hợp lệ *(invalid transitions)*, được biểu thị bằng các ô trống *(empty cells)***.

Một ca kiểm thử dựa trên biểu đồ trạng thái hoặc bảng trạng thái thường được biểu diễn dưới dạng một chuỗi các sự kiện *(sequence of events)*, dẫn đến một chuỗi các thay đổi trạng thái (và các hành động, nếu cần). Một ca kiểm thử có thể, và thường sẽ, bao phủ một số chuyển đổi giữa các trạng thái.

Có nhiều tiêu chí bao phủ cho kiểm thử chuyển đổi trạng thái. Đề cương này thảo luận **3 tiêu chí**:

1. **Bao phủ tất cả các trạng thái *(All states coverage)*:** Các mục bao phủ là các trạng thái *(states)*. Để đạt 100% *all states coverage*, các ca kiểm thử phải đảm bảo rằng tất cả các trạng thái đều được thực thi *(exercised)*. Độ bao phủ được đo bằng số trạng thái đã thực thi chia cho tổng số trạng thái và biểu thị bằng phần trăm.
2. **Bao phủ các chuyển đổi hợp lệ (*Valid transitions coverage* - còn được gọi là *0-switch coverage*):** Các mục bao phủ là các chuyển đổi hợp lệ đơn lẻ *(single valid transitions)*. Để đạt 100% *valid transitions coverage*, các ca kiểm thử phải thực thi tất cả các chuyển đổi hợp lệ. Độ bao phủ được đo bằng số chuyển đổi hợp lệ đã thực thi chia cho tổng số chuyển đổi hợp lệ và biểu thị bằng phần trăm.
3. **Bao phủ tất cả các chuyển đổi *(All transitions coverage)*:** Các mục bao phủ là tất cả các chuyển đổi được hiển thị trong một bảng trạng thái (*state table* - gồm cả hợp lệ và không hợp lệ). Để đạt 100% *all transitions coverage*, các ca kiểm thử phải thực thi tất cả các chuyển đổi hợp lệ và cố gắng thực thi các chuyển đổi không

hợp lệ *(attempt to execute invalid transitions)*. Việc chỉ kiểm thử một chuyển đổi không hợp lệ trong một ca kiểm thử duy nhất giúp tránh **hiện tượng che khuất khiếm khuyết** *(defect masking)*, tức là tình huống trong đó một khiếm khuyết ngăn cản việc phát hiện ra một khiếm khuyết khác. Độ bao phủ được đo bằng số lượng chuyển đổi hợp lệ và không hợp lệ đã được thực thi hoặc cố gắng bao phủ bởi các ca kiểm thử đã chạy, chia cho tổng số chuyển đổi hợp lệ và không hợp lệ, và biểu thị bằng phần trăm.

So sánh độ mạnh giữa 3 tiêu chí:

- *All states coverage* yếu hơn *valid transitions coverage*, bởi vì nó thường có thể đạt được mà không cần thực thi tất cả các chuyển đổi.
- *Valid transitions coverage (0-switch coverage)* là tiêu chí bao phủ được sử dụng rộng rãi nhất. Đạt được 100% *valid transitions coverage* đảm bảo đạt 100% *all states coverage*.
- Đạt được 100% *all transitions coverage* đảm bảo đạt cả 100% *all states coverage* lẫn 100% *valid transitions coverage*, và nên là yêu cầu tối thiểu đối với phần mềm quan trọng về sứ mệnh và an toàn *(mission and safety-critical software)*.

4.3. Các Kỹ thuật Kiểm thử Hộp trắng (White-Box Test Techniques)

Vì sự phổ biến và tính đơn giản của chúng, phần này tập trung vào hai kỹ thuật kiểm thử hộp trắng liên quan đến mã nguồn:

- Kiểm thử câu lệnh *(Statement testing)*
- Kiểm thử nhánh *(Branch testing)*

Có những kỹ thuật kiểm thử hộp trắng nghiêm ngặt hơn được sử dụng trong một số môi trường quan trọng về an toàn *(safety-critical)*, sứ mệnh *(mission-critical)*, hoặc độ toàn vẹn cao *(high-integrity)* để đạt được độ bao phủ mã nguồn kỹ lưỡng hơn. Cũng có các kỹ thuật kiểm thử hộp trắng được sử dụng ở các cấp độ kiểm thử cao hơn (ví dụ: kiểm thử API), hoặc sử dụng độ bao phủ không liên quan đến mã nguồn (ví dụ: độ bao phủ nơ-ron *neuron coverage* trong kiểm thử mạng nơ-ron). Các kỹ thuật này không được thảo luận trong đề cương này.

4.3.1. Kiểm thử Câu lệnh và Độ bao phủ Câu lệnh (Statement Testing and Statement Coverage)

FL-4.3.1 • K2

Trong kiểm thử câu lệnh *(statement testing)*, **các mục bao phủ** *(coverage items)* **là các câu lệnh có thể thực thi** *(executable statements)*. Mục đích là thiết kế các ca kiểm thử thực thi các câu lệnh trong mã nguồn cho đến khi đạt được mức độ bao phủ chấp nhận được. Độ bao phủ được đo bằng số lượng câu lệnh được thực thi bởi các ca kiểm thử chia cho tổng số câu lệnh có thể thực thi trong mã nguồn, và được biểu thị bằng phần trăm.

Khi đạt được 100% độ bao phủ câu lệnh *(100% statement coverage)*, nó đảm bảo rằng tất cả các câu lệnh có thể thực thi trong mã nguồn đã được chạy ít nhất một lần. Cụ thể, điều này có nghĩa là mỗi câu lệnh có chứa khiếm khuyết sẽ được thực thi, điều này có thể gây ra một hỏng hóc chứng minh sự hiện diện của khiếm khuyết đó. Tuy nhiên, việc thực thi một câu lệnh bằng một ca kiểm thử sẽ không phát hiện được khiếm khuyết trong mọi trường hợp. Ví dụ, nó có thể không phát hiện được các khiếm khuyết phụ thuộc vào dữ liệu (*data dependent*, ví dụ: phép chia cho số 0 chỉ bị lỗi khi mẫu số được gán bằng 0). Ngoài ra, 100% độ bao phủ câu lệnh không đảm bảo rằng tất cả logic quyết định *(decision logic)* đã được kiểm thử vì, chẳng hạn, nó có thể không thực thi tất cả các nhánh (*branches*, xem mục 4.3.2) trong mã nguồn (ví dụ: lệnh `if` không có `else`).

4.3.2. Kiểm thử Nhánh và Độ bao phủ Nhánh (Branch Testing and Branch Coverage)

FL-4.3.2 • K2

Một **nhánh** *(branch)* là một sự chuyển giao quyền điều khiển *(transfer of control)* giữa hai nút *(nodes)* trong đồ thị luồng điều khiển *(control flow graph)*, biểu thị các trình tự có thể có mà các câu lệnh mã nguồn được thực thi trong đối tượng kiểm thử. Mỗi sự chuyển giao quyền điều khiển có thể là **vô điều kiện (*unconditional*, tức là mã chạy thẳng *straight-line code*)** hoặc **có điều kiện (*conditional*, tức là một kết quả quyết định *decision outcome*)**.

Trong kiểm thử nhánh *(branch testing)*, **các mục bao phủ là các nhánh** *(branches)* và mục đích là thiết kế các ca kiểm thử để thực thi các nhánh trong mã nguồn cho đến khi đạt được mức độ bao phủ chấp nhận được. Độ bao phủ được đo bằng số lượng nhánh được thực thi bởi các ca kiểm thử chia cho tổng số nhánh và được biểu thị bằng phần trăm.

Khi đạt được 100% độ bao phủ nhánh *(100% branch coverage)*, tất cả các nhánh trong mã nguồn, cả vô điều kiện và có điều kiện, đều được thực thi bởi các ca kiểm thử. Các nhánh có điều kiện thường tương ứng với kết quả `true` hoặc `false` từ một quyết định *"if...then"*, một kết quả từ câu lệnh `switch/case`, hoặc một quyết định thoát hay tiếp tục trong một vòng lặp *(loop)*. Tuy nhiên, việc thực thi một nhánh bằng một ca kiểm thử cũng sẽ không phát hiện được khiếm khuyết trong mọi trường hợp. Ví dụ, nó có thể không phát hiện được các khiếm khuyết đòi hỏi việc thực thi một đường dẫn *(path)* cụ thể trong mã nguồn.

Độ bao phủ nhánh bao hàm độ bao phủ câu lệnh *(Branch coverage subsumes statement coverage)*. Điều này có nghĩa là bất kỳ tập hợp ca kiểm thử nào đạt được **100% độ bao phủ nhánh thì chắc chắn cũng đạt được 100% độ bao phủ câu lệnh (nhưng ngược lại thì KHÔNG đúng - *not vice versa*)**.

4.3.3. Giá trị của Kiểm thử Hộp trắng (The Value of White-box Testing)

FL-4.3.3 • K2

- **Điểm mạnh cơ bản** *(Fundamental strength)* mà tất cả các kỹ thuật kiểm thử hộp trắng đều có chung là **toàn bộ việc triển khai phần mềm** *(entire software implementation)* **đều được tính đến** trong quá trình kiểm thử, điều này tạo điều kiện phát hiện khiếm khuyết ngay cả khi đặc tả phần mềm mơ hồ, lỗi thời hoặc không đầy đủ *(vague, outdated or incomplete)*. *(Ví dụ: phát hiện đoạn mã độc hoặc tính năng thừa trong code mà không có trong tài liệu).*
- **Điểm yếu tương ứng** *(Corresponding weakness)* là nếu phần mềm không triển khai một hoặc nhiều yêu cầu, kiểm thử hộp trắng có thể không phát hiện được các **khiếm khuyết do bỏ sót** *(defects of omission)* đó (Watson 1996).
- Các kỹ thuật kiểm thử hộp trắng có thể được sử dụng trong kiểm thử tĩnh (ví dụ: trong quá trình chạy nháp mã nguồn *dry runs of code*). Chúng rất phù hợp để duyệt mã nguồn chưa sẵn sàng để thực thi (Hetzel 1988), mã giả *(pseudocode)* và các logic cấp cao hoặc từ trên xuống khác có thể mô hình hóa bằng đồ thị luồng điều khiển *(control flow graph)*.
- Việc chỉ thực hiện kiểm thử hộp đen không cung cấp thước đo về độ bao phủ mã nguồn thực tế *(actual code coverage)*. Các thước đo bao phủ hộp trắng cung cấp một phép đo khách quan *(objective measurement)* về độ bao phủ và thông tin cần thiết để cho phép tạo thêm các bài kiểm thử nhằm tăng độ bao phủ này, và từ đó làm tăng sự tự tin vào mã nguồn.

4.4. Các Kỹ thuật Kiểm thử Dựa trên Kinh nghiệm (Experience-based Test Techniques)

Các kỹ thuật kiểm thử dựa trên kinh nghiệm thường được sử dụng và thảo luận trong các phần tiếp theo là:

- Đoán lỗi *(Error guessing)*
- Kiểm thử khám phá *(Exploratory testing)*
- Kiểm thử dựa trên danh sách kiểm tra *(Checklist-based testing)*

4.4.1. Đoán lỗi (Error Guessing) FL-4.4.1 • K2

Đoán lỗi *(Error guessing)* là một kỹ thuật kiểm thử được sử dụng để dự đoán sự xuất hiện của các sai sót *(errors)*, khiếm khuyết *(defects)*, và hỏng hóc *(failures)*, dựa trên kiến thức của kiểm thử viên, bao gồm:

- Cách ứng dụng đã hoạt động trong quá khứ *(How the application has worked in the past)*
- Các loại sai sót mà lập trình viên có xu hướng mắc phải và các loại khiếm khuyết phát sinh từ những sai sót đó
- Các loại hỏng hóc đã xảy ra trong các ứng dụng khác tương tự

Nhìn chung, các sai sót, khiếm khuyết và hỏng hóc có thể liên quan đến: đầu vào (ví dụ: đầu vào đúng không được chấp nhận, tham số sai hoặc thiếu), đầu ra (ví dụ: sai định dạng, sai kết quả), logic (ví dụ: thiếu trường hợp, sai toán tử), tính toán (ví dụ: sai toán hạng, tính toán sai), giao diện (ví dụ: lệch tham số, kiểu không tương thích), hoặc dữ liệu (ví dụ: khởi tạo sai, sai kiểu).

Tấn công lỗi *(Fault attacks)* là một cách có phương pháp để triển khai kỹ thuật đoán lỗi. Kỹ thuật kiểm thử này đòi hỏi kiểm thử viên tạo ra hoặc thu thập một danh sách các sai sót, khiếm khuyết và hỏng hóc có thể xảy ra, và thiết kế các bài kiểm thử sẽ xác định các khiếm khuyết gắn với các sai sót đó, làm bộc lộ các khiếm khuyết, hoặc gây ra các hỏng hóc. Các danh sách này có thể được xây dựng dựa trên kinh nghiệm, dữ liệu khiếm khuyết và hỏng hóc, hoặc từ kiến thức chung về lý do tại sao phần mềm bị hỏng.

4.4.2. Kiểm thử Khám phá (Exploratory Testing) FL-4.4.2 • K2

Trong kiểm thử khám phá *(exploratory testing)*, các bài kiểm thử được **thiết kế, thực thi, và đánh giá đồng thời** *(simultaneously designed, executed, and evaluated)* trong khi kiểm thử viên tìm hiểu về đối tượng kiểm thử. Việc kiểm thử được sử dụng để học hỏi thêm về đối tượng kiểm thử, khám phá nó sâu hơn bằng các bài kiểm thử tập trung, và tạo ra các bài kiểm thử cho các khu vực chưa được kiểm thử.

Kiểm thử khám phá đôi khi được thực hiện bằng cách sử dụng **kiểm thử dựa trên phiên** *(session-based testing)* để cấu trúc hóa việc kiểm thử. Trong cách tiếp cận dựa trên phiên, kiểm thử khám phá được thực hiện trong một **khung thời gian cố định** *(defined time box)*. Kiểm thử viên sử dụng một **điều lệ kiểm thử** *(test charter)* chứa các mục tiêu kiểm thử để định hướng việc kiểm thử. Phiên kiểm thử thường được nối tiếp bằng một buổi **họp báo cáo kết quả** *(debriefing)* bao gồm cuộc thảo luận giữa kiểm thử viên và các bên liên quan quan tâm đến kết quả kiểm thử của phiên kiểm thử đó. Trong cách tiếp cận này, các mục tiêu kiểm thử có thể được coi như các điều kiện kiểm thử cấp cao *(high-level test conditions)*. Các mục bao phủ *(coverage items)* được xác định và thực thi ngay trong phiên kiểm thử. Kiểm thử viên có thể sử dụng các bảng ghi chép phiên kiểm thử *(test session sheets)* để ghi lại các bước đã thực hiện và những khám phá đã tìm ra.

Kiểm thử khám phá rất hữu ích khi **có ít hoặc không đủ đặc tả** ***(few or inadequate specifications)*** hoặc có **áp lực thời gian lớn** ***(significant time pressure)*** đối với việc kiểm thử. Kiểm thử khám phá cũng hữu ích để bổ sung cho các kỹ thuật kiểm thử chính thức hơn khác. Kiểm thử khám phá sẽ hiệu quả hơn nếu kiểm thử viên có nhiều kinh nghiệm, có kiến thức nghiệp vụ ***(domain knowledge)*** và có mức độ cao về các kỹ năng thiết yếu như kỹ năng phân tích, sự tò mò và tính sáng tạo (xem mục 1.5.1).

Kiểm thử khám phá có thể kết hợp việc sử dụng các kỹ thuật kiểm thử khác (ví dụ: phân vùng tương đương).

4.4.3. Kiểm thử Dựa trên Danh sách Kiểm tra (Checklist-Based Testing) FL-4.4.3 • K2

Trong kiểm thử dựa trên danh sách kiểm tra ***(checklist-based testing)***, kiểm thử viên thiết kế, triển khai và thực thi các bài kiểm thử để bao phủ các điều kiện kiểm thử từ một **danh sách kiểm tra** ***(checklist)***. Danh sách kiểm tra có thể được xây dựng dựa trên kinh nghiệm, kiến thức về những gì quan trọng đối với người dùng, hoặc sự hiểu biết về lý do và cách thức phần mềm bị lỗi. **Danh sách kiểm tra KHÔNG NÊN chứa các mục có thể kiểm tra tự động, các mục phù hợp hơn làm tiêu chí bắt đầu** ***(entry criteria)*****, tiêu chí kết thúc** ***(exit criteria)*****, hoặc các mục quá chung chung** ***(too general)*** (Brykczynski 1999).

Các mục trong danh sách kiểm tra thường được diễn đạt dưới dạng một câu hỏi. Phải có khả năng kiểm tra từng mục một cách riêng biệt và trực tiếp. Các mục này có thể tham chiếu đến các yêu cầu, các thuộc tính giao diện đồ họa, các đặc tính chất lượng hoặc các hình thức điều kiện kiểm thử khác. Danh sách kiểm tra có thể được tạo ra để hỗ trợ nhiều loại kiểm thử khác nhau, bao gồm kiểm thử chức năng và phi chức năng (ví dụ: 10 nguyên tắc heuristic cho kiểm thử khả năng sử dụng của Nielsen 1994).

Một số mục trong danh sách kiểm tra có thể dần trở nên kém hiệu quả theo thời gian vì các lập trình viên sẽ học được cách tránh mắc lại cùng những sai sót đó. Các mục mới cũng có thể cần được thêm vào để phản ánh các khiếm khuyết mức độ nghiêm trọng cao mới được tìm thấy. Do đó, **danh sách kiểm tra nên được cập nhật thường xuyên** dựa trên phân tích khiếm khuyết. Tuy nhiên, cần cẩn thận để tránh làm cho danh sách kiểm tra trở nên quá dài (Gawande 2009).

Khi không có các ca kiểm thử chi tiết, kiểm thử dựa trên danh sách kiểm tra có thể cung cấp các hướng dẫn và một mức độ nhất quán nhất định cho việc kiểm thử. Nếu danh sách kiểm tra ở mức cao ***(high-level)***, một số sự biến thiên ***(variability)*** trong kiểm thử thực tế có khả năng xảy ra, dẫn đến **độ bao phủ tiềm năng lớn hơn nhưng khả năng lặp lại thấp hơn** ***(greater coverage but less repeatability)***.

4.5. Các Cách tiếp cận Kiểm thử Dựa trên sự Cộng tác (Collaboration-based Test Approaches)

Mỗi kỹ thuật kiểm thử nêu trên (xem mục 4.2, 4.3, 4.4) đều có một mục tiêu cụ thể đối với việc **phát hiện khiếm khuyết** ***(defect detection)***. Ngược lại, các cách tiếp cận dựa trên sự cộng tác ***(collaboration-based approaches)*** còn tập trung vào việc **tránh/ngăn ngừa khiếm khuyết** ***(defect avoidance)*** thông qua sự cộng tác và giao tiếp.

4.5.1. Viết Câu chuyện Người dùng Cộng tác (Collaborative User Story Writing) FL-4.5.1 • K2

Một câu chuyện người dùng ***(user story)*** đại diện cho một tính năng sẽ mang lại giá trị cho người dùng hoặc người mua hệ thống/phần mềm. Các câu chuyện người dùng có ba khía cạnh quan trọng (Jeffries 2000), được gọi chung là **"3 C's"**:

- **Card (Thẻ):** phương tiện mô tả một câu chuyện người dùng (ví dụ: một thẻ giấy index, một mục trên bảng điện tử).

- **Conversation (Cuộc trò chuyện):** giải thích cách phần mềm sẽ được sử dụng (có thể được ghi thành tài liệu hoặc bằng lời nói).
- **Confirmation (Sự xác nhận):** các tiêu chí chấp nhận (***acceptance criteria***, xem mục 4.5.2).

Định dạng phổ biến nhất cho một câu chuyện người dùng là: `"As a [role], I want [goal to be accomplished], so that I can [resulting business value for the role]"` *(Là một [vai trò], tôi muốn [mục tiêu cần hoàn thành], để tôi có thể [giá trị kinh doanh đạt được cho vai trò đó])*, theo sau là các tiêu chí chấp nhận.

Việc cùng nhau viết câu chuyện người dùng có thể sử dụng các kỹ thuật như động não ***(brainstorming)*** và sơ đồ tư duy ***(mind mapping)***. Sự cộng tác cho phép nhóm có được tầm nhìn chung về những gì cần chuyển giao, bằng cách tính đến **ba góc nhìn (*three perspectives* - bộ ba Amigos): nghiệp vụ *(business)*, phát triển *(development)* và kiểm thử *(testing)***.

Các câu chuyện người dùng tốt nên tuân theo tiêu chuẩn **INVEST**:

- **I**ndependent (Độc lập)
- **N**egotiable (Có thể thương lượng)
- **V**aluable (Có giá trị)
- **E**stimable (Có thể ước lượng được)
- **S**mall (Nhỏ gọn)
- **T**estable (Có thể kiểm thử được)

Nếu một bên liên quan không biết cách kiểm thử một ***user story***, điều này có thể chỉ ra rằng ***user story*** đó chưa đủ rõ ràng, hoặc nó không phản ánh điều gì đó có giá trị đối với họ, hoặc bên liên quan đó chỉ đơn giản là cần sự trợ giúp trong việc kiểm thử (Wake 2003).

4.5.2. Tiêu chí Chấp nhận (Acceptance Criteria) FL-4.5.2 • K2

Các tiêu chí chấp nhận ***(Acceptance criteria)*** cho một ***user story*** là các điều kiện mà việc triển khai ***user story*** đó phải đáp ứng để được các bên liên quan chấp nhận. Từ góc độ này, các tiêu chí chấp nhận có thể được xem như là **các điều kiện kiểm thử *(test conditions)*** cần được thực thi bởi các bài kiểm thử. Các tiêu chí chấp nhận thường là kết quả của Cuộc trò chuyện (***Conversation***, xem mục 4.5.1).

Các tiêu chí chấp nhận được sử dụng để:

- Xác định phạm vi ***(scope)*** của ***user story***
- Đạt được sự đồng thuận giữa các bên liên quan
- Mô tả cả các kịch bản tích cực/hợp lệ ***(positive)*** và tiêu cực/ngoại lệ ***(negative scenarios)***
- Đóng vai trò làm cơ sở cho việc kiểm thử chấp nhận ***user story*** (***user story acceptance testing***, xem mục 4.5.3)
- Cho phép lập kế hoạch và ước lượng chính xác

Có một số cách để viết tiêu chí chấp nhận cho một ***user story***. **Hai định dạng phổ biến nhất là:**

1. **Hướng kịch bản *(Scenario-oriented)*:** ví dụ: định dạng **Given/When/Then** được sử dụng trong BDD (xem mục 2.1.3).

2. **Hướng quy tắc *(Rule-oriented)*:** ví dụ: danh sách xác minh dạng gạch đầu dòng ***(bullet point verification list)***, hoặc dạng bảng ánh xạ đầu vào - đầu ra ***(tabulated form of input-output mapping)***.

Hầu hết các tiêu chí chấp nhận có thể được ghi lại theo một trong hai định dạng này. Tuy nhiên, nhóm có thể sử dụng một định dạng tùy chỉnh khác, miễn là các tiêu chí chấp nhận được định nghĩa rõ ràng và không mơ hồ ***(well-defined and unambiguous)***.

4.5.3. Phát triển Hướng Kiểm thử Chấp nhận (Acceptance Test-driven Development - ATDD)

FL-4.5.3 • K3

ATDD là một cách tiếp cận **kiểm thử trước (*test-first approach*, xem mục 2.1.3). Các ca kiểm thử được tạo ra **trước khi** triển khai ***user story***. Các ca kiểm thử được tạo ra bởi các thành viên trong nhóm với nhiều góc nhìn khác nhau, ví dụ: khách hàng, lập trình viên, và kiểm thử viên (Adzic 2009). Các ca kiểm thử có thể được thực thi thủ công hoặc tự động.

- **Bước đầu tiên** là một buổi hội thảo đặc tả ***(specification workshop)***, nơi ***user story*** và (nếu chưa được định nghĩa) các tiêu chí chấp nhận của nó được phân tích, thảo luận và viết ra bởi các thành viên trong nhóm. Sự thiếu sót, mơ hồ hoặc khiếm khuyết trong ***user story*** sẽ được giải quyết trong quá trình này.
- **Bước tiếp theo** là tạo ra các ca kiểm thử. Việc này có thể được thực hiện bởi cả nhóm cùng nhau hoặc bởi cá nhân kiểm thử viên. Các ca kiểm thử dựa trên các tiêu chí chấp nhận và có thể được xem như là các **ví dụ *(examples)*** về cách phần mềm hoạt động. Điều này sẽ giúp nhóm triển khai ***user story*** một cách chính xác.

Vì các ví dụ ***(examples)*** và các bài kiểm thử ***(tests)*** là như nhau, hai thuật ngữ này thường được sử dụng thay thế cho nhau. Trong quá trình thiết kế kiểm thử, các kỹ thuật kiểm thử được mô tả trong các mục 4.2, 4.3 và 4.4 có thể được áp dụng.

Thông thường, **các ca kiểm thử đầu tiên là tích cực *(positive)***, xác nhận hành vi đúng mà không có ngoại lệ hoặc điều kiện lỗi, bao gồm chuỗi hoạt động được thực thi nếu mọi thứ diễn ra đúng như mong đợi. Sau khi các ca kiểm thử tích cực hoàn tất, nhóm nên thực hiện **kiểm thử tiêu cực *(negative testing)***. Cuối cùng, nhóm cũng nên bao phủ cả **các đặc tính chất lượng phi chức năng** (ví dụ: hiệu suất hoạt động, khả năng sử dụng). Các ca kiểm thử nên được diễn đạt theo cách dễ hiểu đối với các bên liên quan. Thông thường, các ca kiểm thử chứa các câu bằng ngôn ngữ tự nhiên bao gồm các điều kiện tiên quyết cần thiết (***preconditions*** - nếu có), các đầu vào ***(inputs)***, và các điều kiện sau ***(postconditions)***.

Các ca kiểm thử phải bao phủ tất cả các đặc tính của *user story* và KHÔNG ĐƯỢC vượt ra ngoài phạm vi của *user story* đó *(should not go beyond the story)*. Tuy nhiên, các tiêu chí chấp nhận có thể chi tiết hóa một số vấn đề được mô tả trong ***user story***. Ngoài ra, không có hai ca kiểm thử nào nên mô tả cùng một đặc tính của ***user story*** (tránh trùng lặp).

Khi được ghi lại ở định dạng được hỗ trợ bởi một khung tự động hóa kiểm thử ***(test automation framework)***, các lập trình viên có thể tự động hóa các ca kiểm thử bằng cách viết mã hỗ trợ khi họ triển khai tính năng được mô tả bởi ***user story***. Khi đó, các bài kiểm thử chấp nhận sẽ trở thành **các yêu cầu có thể thực thi được *(executable requirements)***.

CHƯƠNG 5

🕑 Thời lượng phân bổ: 335 phút

Quản lý các Hoạt động Kiểm thử (Managing the Test Activities)

K1 Bảng Từ Khóa Chuẩn ISTQB (Keywords) — Cần nhớ chính xác thuật ngữ gốc Tiếng Anh khi làm bài thi

defect management	quản lý khiếm khuyết	**defect report**	báo cáo khiếm khuyết
entry criteria	tiêu chí bắt đầu	**exit criteria**	tiêu chí kết thúc
product risk	rủi ro sản phẩm	**project risk**	rủi ro dự án
risk	rủi ro	**risk analysis**	phân tích rủi ro
risk assessment	đánh giá rủi ro	**risk control**	kiểm soát rủi ro
risk identification	nhận diện rủi ro	**risk level**	mức độ rủi ro
risk management	quản lý rủi ro	**risk mitigation**	giảm thiểu rủi ro
risk monitoring	giám sát rủi ro	**risk-based testing**	kiểm thử dựa trên rủi ro
test approach	cách tiếp cận kiểm thử	**test completion report**	báo cáo hoàn tất kiểm thử
test control	kiểm soát kiểm thử	**test monitoring**	giám sát kiểm thử
test plan	kế hoạch kiểm thử	**test planning**	lập kế hoạch kiểm thử
test progress report	báo cáo tiến độ kiểm thử	**test pyramid**	kim tự tháp kiểm thử
test strategy	chiến lược kiểm thử	**testing quadrants**	góc phần tư kiểm thử

Mục Tiêu Học Tập Trọng Tâm (Learning Objectives)

MÃ LO	MỨC NHẬN THỨC	NỘI DUNG YÊU CẦU TRONG ĐỀ THI
5.1 Lập kế hoạch Kiểm thử (Test Planning)		
FL-5.1.1	K2 · Hiểu	Cho ví dụ về mục đích và nội dung của một kế hoạch kiểm thử (***test plan***)
FL-5.1.2	K1 · Ghi nhớ	Nhận biết cách một kiểm thử viên gia tăng giá trị cho việc lập kế hoạch vòng lặp (***iteration planning***) và lập kế hoạch phát hành (***release planning***)
FL-5.1.3	K2 · Hiểu	So sánh và đối chiếu tiêu chí bắt đầu (***entry criteria***) và tiêu chí kết thúc (***exit criteria***)
FL-5.1.4	K3 · Áp dụng	**Áp dụng** các kỹ thuật ước lượng (***estimation techniques***) để tính toán nỗ lực/công sức kiểm thử cần thiết
FL-5.1.5	K3 · Áp dụng	**Áp dụng** việc sắp xếp độ ưu tiên ca kiểm thử (***test case prioritization***)

MÃ LO	MỨC NHẬN THỨC	NỘI DUNG YÊU CẦU TRONG ĐỀ THI
FL-5.1.6	K1 · Ghi nhớ	Nhớ lại các khái niệm của kim tự tháp kiểm thử (***test pyramid***)
FL-5.1.7	K2 · Hiểu	Tóm tắt các góc phần tư kiểm thử (***testing quadrants***) và mối quan hệ của chúng với các cấp độ kiểm thử và các loại kiểm thử
5.2 Quản lý Rủi ro (Risk Management)		
FL-5.2.1	K1 · Ghi nhớ	Xác định mức độ rủi ro (***risk level***) bằng cách sử dụng khả năng xảy ra rủi ro (***risk likelihood***) và mức độ tác động của rủi ro (***risk impact***)
FL-5.2.2	K2 · Hiểu	Phân biệt giữa rủi ro dự án (***project risks***) và rủi ro sản phẩm (***product risks***)
FL-5.2.3	K2 · Hiểu	Giải thích cách phân tích rủi ro sản phẩm có thể ảnh hưởng đến mức độ kỹ lưỡng (***thoroughness***) và phạm vi kiểm thử (***test scope***)
FL-5.2.4	K2 · Hiểu	Giải thích những biện pháp nào có thể được thực hiện để ứng phó với các rủi ro sản phẩm đã được phân tích
5.3 Giám sát Kiểm thử, Kiểm soát Kiểm thử và Hoàn tất Kiểm thử (Test Monitoring, Test Control and Test Completion)		
FL-5.3.1	K1 · Ghi nhớ	Nhớ lại các số liệu đo lường (***metrics***) được sử dụng cho kiểm thử
FL-5.3.2	K2 · Hiểu	Tóm tắt mục đích, nội dung và đối tượng độc giả của các báo cáo kiểm thử (***test reports***)
FL-5.3.3	K2 · Hiểu	Cho ví dụ về cách truyền đạt trạng thái của kiểm thử (***status of testing***)
5.4 Quản lý Cấu hình (Configuration Management)		
FL-5.4.1	K2 · Hiểu	Tóm tắt cách quản lý cấu hình hỗ trợ cho kiểm thử
5.5 Quản lý Khiếm khuyết (Defect Management)		
FL-5.5.1	K3 · Áp dụng	**Chuẩn bị** một báo cáo khiếm khuyết (***defect report***)

5.1. Lập kế hoạch Kiểm thử (Test Planning)

5.1.1. Mục đích và Nội dung của một Kế hoạch Kiểm thử

FL-5.1.1 • K2

Một kế hoạch kiểm thử *(test plan)* mô tả các mục tiêu kiểm thử, nguồn lực và quy trình cho một dự án kiểm thử. Một kế hoạch kiểm thử:

- Ghi lại phương tiện và lịch trình để đạt được các mục tiêu kiểm thử.
- Giúp đảm bảo rằng các hoạt động kiểm thử được thực hiện sẽ đáp ứng các tiêu chí đã thiết lập.
- Đóng vai trò là phương tiện giao tiếp với các thành viên trong nhóm và các bên liên quan khác.
- Chứng minh rằng việc kiểm thử sẽ tuân thủ chính sách kiểm thử *(test policy)* và chiến lược kiểm thử *(test strategy)* hiện hành (hoặc giải thích lý do tại sao việc kiểm thử sẽ đi chệch khỏi chúng).

Việc lập kế hoạch kiểm thử định hướng suy nghĩ của kiểm thử viên và buộc họ phải đối mặt với những thách thức trong tương lai liên quan đến rủi ro, lịch trình, con người, công cụ, chi phí, công sức, v.v. Quá trình chuẩn bị một kế hoạch kiểm thử là một cách hữu ích để suy tính kỹ lưỡng về những nỗ lực cần thiết nhằm đạt được các mục tiêu kiểm thử.

Nội dung điển hình của một kế hoạch kiểm thử bao gồm:

- **Ngữ cảnh của kiểm thử** ***(Context of testing)*****:** ví dụ: phạm vi kiểm thử ***(test scope)***, mục tiêu kiểm thử ***(test objectives)***, cơ sở kiểm thử ***(test basis)***.
- **Các giả định và ràng buộc** ***(Assumptions and constraints)*** của dự án kiểm thử.
- **Các bên liên quan** ***(Stakeholders)*****:** ví dụ: vai trò, trách nhiệm, sự liên quan đến kiểm thử, nhu cầu tuyển dụng và đào tạo.
- **Giao tiếp** ***(Communication)*****:** ví dụ: hình thức và tần suất giao tiếp, các biểu mẫu tài liệu ***(documentation templates)***.
- **Sổ đăng ký rủi ro** ***(Risk register)*****:** ví dụ: rủi ro sản phẩm, rủi ro dự án.
- **Cách tiếp cận kiểm thử** ***(Test approach)*****:** ví dụ: các cấp độ kiểm thử, các loại kiểm thử, các kỹ thuật kiểm thử, các sản phẩm bàn giao kiểm thử ***(test deliverables)***, tiêu chí bắt đầu và tiêu chí kết thúc, tính độc lập của kiểm thử, các số liệu đo lường ***(metrics)*** cần thu thập, các yêu cầu dữ liệu kiểm thử, các yêu cầu môi trường kiểm thử, những điểm sai lệch so với chính sách kiểm thử và chiến lược kiểm thử.
- **Ngân sách và lịch trình** ***(Budget and schedule)***.

5.1.2. Đóng góp của Kiểm thử viên vào Lập kế hoạch Vòng lặp và Phát hành

FL-5.1.2 • K1

Trong các SDLC lặp ***(iterative SDLCs)***, thông thường có hai loại lập kế hoạch diễn ra: **lập kế hoạch phát hành** ***(release planning)*** và **lập kế hoạch vòng lặp** ***(iteration planning)***.

- **Lập kế hoạch phát hành** ***(Release planning)*****:** Nhìn về phía trước tới đợt phát hành của một sản phẩm, định nghĩa và định nghĩa lại tồn đọng sản phẩm ***(product backlog)***, và có thể bao gồm việc làm mịn ***(refining)*** các ***user stories*** lớn thành một tập hợp các ***user stories*** nhỏ hơn. Nó cũng làm cơ sở cho cách tiếp cận kiểm thử và kế hoạch kiểm thử xuyên suốt tất cả các vòng lặp. **Kiểm thử viên tham gia vào lập kế hoạch phát hành sẽ:** tham gia viết các ***user stories*** có thể kiểm thử được và các tiêu chí chấp nhận (xem mục 4.5), tham gia phân tích rủi ro dự án và rủi ro chất lượng (xem mục 5.2), ước lượng công sức kiểm thử gắn với các ***user stories*** (xem mục 5.1.4), xác định cách tiếp cận kiểm thử, và lập kế hoạch kiểm thử cho đợt phát hành.
- **Lập kế hoạch vòng lặp** ***(Iteration planning)*****:** Nhìn về phía trước tới cuối một vòng lặp đơn lẻ và quan tâm đến tồn đọng của vòng lặp ***(iteration backlog)***. **Kiểm thử viên tham gia vào lập kế hoạch vòng lặp sẽ:** tham gia vào việc phân tích rủi ro chi tiết của các ***user stories***, xác định khả năng kiểm thử ***(testability)*** của các ***user stories***, phân rã các ***user stories*** thành các công việc (***tasks*** - đặc biệt là các công việc kiểm thử), ước lượng công sức kiểm thử cho tất cả các công việc kiểm thử, và xác định cũng như làm mịn các khía cạnh chức năng và phi chức năng của đối tượng kiểm thử.

5.1.3. Tiêu chí Bắt đầu và Tiêu chí Kết thúc (Entry Criteria and Exit Criteria)

FL-5.1.3 • K2

- **Tiêu chí bắt đầu** ***(Entry criteria)*** định nghĩa các điều kiện tiên quyết ***(preconditions)*** để tiến hành một hoạt động nhất định. Nếu các tiêu chí bắt đầu không được đáp ứng, rất có khả năng hoạt động đó sẽ trở nên khó khăn hơn, tốn thời gian hơn, đắt đỏ hơn và rủi ro hơn.

- **Tiêu chí kết thúc** *(Exit criteria)* định nghĩa những gì phải đạt được để tuyên bố một hoạt động đã hoàn tất. Tiêu chí bắt đầu và tiêu chí kết thúc cần được định nghĩa cho mỗi cấp độ kiểm thử, và sẽ khác nhau tùy theo mục tiêu kiểm thử.

Các tiêu chí bắt đầu *(entry criteria)* **điển hình bao gồm:**

- Sự sẵn có của các nguồn lực (ví dụ: nhân sự, công cụ, môi trường, dữ liệu kiểm thử, ngân sách, thời gian).
- Sự sẵn có của ***testware*** (ví dụ: cơ sở kiểm thử, các yêu cầu có thể kiểm thử được, ***user stories***, các ca kiểm thử).
- Mức chất lượng ban đầu của một đối tượng kiểm thử (ví dụ: tất cả các bài kiểm thử khói ***smoke tests*** đều đã vượt qua ***passed***).

Các tiêu chí kết thúc *(exit criteria)* **điển hình bao gồm:**

- Các thước đo về mức độ kỹ lưỡng (***measures of thoroughness***, ví dụ: mức độ bao phủ đạt được, số lượng khiếm khuyết chưa được giải quyết, mật độ khiếm khuyết ***defect density***, số lượng ca kiểm thử bị thất bại).
- Các tiêu chí "có/không" nhị phân (***binary "yes/no" criteria*** / ***completion criteria***, ví dụ: các bài kiểm thử theo kế hoạch đã được thực thi, kiểm thử tĩnh đã được thực hiện, tất cả các khiếm khuyết tìm thấy đã được báo cáo, tất cả các bài kiểm thử hồi quy đã được tự động hóa).

Việc hết thời gian hoặc cạn ngân sách *(Running out of time or budget)* **cũng có thể được xem là các tiêu chí kết thúc hợp lệ** *(valid exit criteria)*. Ngay cả khi các tiêu chí kết thúc khác chưa được thỏa mãn, việc dừng kiểm thử trong những hoàn cảnh như vậy vẫn có thể chấp nhận được, nếu các bên liên quan đã rà soát và chấp nhận rủi ro khi đưa sản phẩm vào vận hành *(go live)* mà không kiểm thử thêm.

Trong phát triển phần mềm Agile:

- Tiêu chí kết thúc thường được gọi là **Định nghĩa sự Hoàn thành** *(Definition of Done - DoD)*, xác định các chỉ số khách quan của nhóm đối với một hạng mục có thể phát hành được.
- Tiêu chí bắt đầu mà một ***user story*** phải đáp ứng để bắt đầu các hoạt động phát triển và/hoặc kiểm thử được gọi là **Định nghĩa sự Sẵn sàng** *(Definition of Ready - DoR)*.

5.1.4. Các Kỹ thuật Ước lượng (Estimation Techniques) FL-5.1.4 • K3

Ước lượng công sức kiểm thử *(Test effort estimation)* bao gồm việc dự đoán khối lượng công việc liên quan đến kiểm thử cần thiết để đáp ứng các mục tiêu kiểm thử của một dự án kiểm thử. Điều quan trọng là phải làm rõ cho các bên liên quan biết rằng bản ước lượng được dựa trên một số giả định và luôn chịu sai số ước lượng *(estimation error)*. Việc ước lượng cho các công việc nhỏ thường chính xác hơn so với các công việc lớn. Do đó, khi ước lượng một công việc lớn, nó có thể được phân rã thành một tập hợp các công việc nhỏ hơn để từ đó có thể ước lượng từng công việc nhỏ.

Trong đề cương này, **4 kỹ thuật ước lượng** (chia làm 2 nhóm: *dựa trên số liệu* ***metrics-based*** và *dựa trên chuyên gia* ***expert-based***) được mô tả:

1. **Ước lượng dựa trên tỷ lệ (*Estimation based on ratios* - *Metrics-based*):** Trong kỹ thuật dựa trên số liệu này, các con số được thu thập từ các dự án trước đó trong tổ chức, giúp có thể rút ra các tỷ lệ "chuẩn" *("standard" ratios)* cho các dự án tương tự. Tỷ lệ từ chính các dự án của tổ chức (ví dụ: lấy từ dữ liệu lịch sử) nhìn chung là nguồn tốt nhất để sử dụng trong quá trình ước lượng. Các tỷ lệ chuẩn này sau đó có thể được dùng để ước lượng công sức kiểm thử cho dự án mới.

- *Ví dụ tính toán (K3):* Nếu ở dự án trước tỷ lệ công sức phát triển so với kiểm thử *(development-to-test effort ratio)* là `3:2`, và trong dự án hiện tại công sức phát triển dự kiến là *600 ngày-người (person-days)*, thì công sức kiểm thử có thể được ước lượng là: **600 × (2/3) = 400 person-days**.

2. **Ngoại suy (*Extrapolation* - *Metrics-based*):** Trong kỹ thuật dựa trên số liệu này, các phép đo được thực hiện càng sớm càng tốt trong dự án hiện tại để thu thập dữ liệu. Khi đã có đủ các quan sát, công sức cần thiết cho phần công việc còn lại có thể được tính xấp xỉ bằng cách ngoại suy dữ liệu này (thường bằng cách áp dụng một mô hình toán học). Phương pháp này rất phù hợp trong các SDLC lặp *(iterative SDLCs)*.
   - *Ví dụ tính toán (K3):* Nhóm có thể ngoại suy công sức kiểm thử trong vòng lặp sắp tới bằng trung bình cộng công sức của 3 vòng lặp gần nhất.
3. **Wideband Delphi (*Expert-based*):** Trong kỹ thuật lặp, dựa trên chuyên gia này, các chuyên gia đưa ra các ước lượng dựa trên kinh nghiệm. Mỗi chuyên gia, một cách độc lập/cô lập *(in isolation)*, ước lượng công sức. Kết quả được thu thập lại và nếu có những sai lệch trong ước lượng của một chuyên gia vượt ra ngoài phạm vi ranh giới đã thỏa thuận, các chuyên gia sẽ thảo luận về các ước lượng hiện tại của họ. Mỗi chuyên gia sau đó được yêu cầu đưa ra một ước lượng mới dựa trên phản hồi đó, một lần nữa trong sự cô lập *(in isolation)*. Quy trình này được lặp lại cho đến khi đạt được sự đồng thuận *(consensus)*. **Planning Poker** là một biến thể của Wideband Delphi, thường được sử dụng trong Agile, trong đó các ước lượng thường được thực hiện bằng cách sử dụng các lá bài có ghi số đại diện cho kích thước công sức.
4. **Ước lượng ba điểm (*Three-point estimation* - *Expert-based*):** Trong kỹ thuật dựa trên chuyên gia này, ba con số ước lượng được đưa ra bởi các chuyên gia:
   - Ước lượng lạc quan nhất *(most optimistic estimation)*: **a**
   - Ước lượng có khả năng xảy ra nhất *(most likely estimation)*: **m**
   - Ước lượng bi quan nhất *(most pessimistic estimation)*: **b**

   Ước lượng cuối cùng (**E**) là trung bình cộng có trọng số của chúng. Trong phiên bản phổ biến nhất của kỹ thuật này, ước lượng và sai số đo lường (độ lệch chuẩn **SD**) được tính theo công thức:

   CÔNG THỨC ƯỚC LƯỢNG BA ĐIỂM (THREE-POINT ESTIMATION - K3)

   E = (a + 4m + b) ÷ 6 • **SD** = (b − a) ÷ 6

   - *Ví dụ tính toán (K3):* Nếu các ước lượng (tính theo giờ-người *person-hours*) là **a = 6, m = 9** và **b = 18**, thì ước lượng cuối cùng là **10 ± 2 person-hours** (tức là nằm trong khoảng từ **8** đến **12 person-hours**), bởi vì:

   E = (6 + 4×9 + 18) ÷ 6 = **10 person-hours** • **SD** = (18 − 6) ÷ 6 = **2 person-hours**

5.1.5. Sắp xếp Độ ưu tiên Ca kiểm thử (Test Case Prioritization) FL-5.1.5 • K3

Một khi các ca kiểm thử và các thủ tục kiểm thử được quy định và tập hợp thành các bộ kiểm thử *(test suites)*, các bộ kiểm thử này có thể được sắp xếp vào một lịch trình thực thi kiểm thử *(test execution schedule)* xác định thứ tự mà chúng sẽ được chạy. Khi sắp xếp độ ưu tiên cho các ca kiểm thử, nhiều yếu tố khác nhau có thể được tính đến.
3 chiến lược sắp xếp độ ưu tiên ca kiểm thử phổ biến nhất là:

1. **Sắp xếp độ ưu tiên dựa trên rủi ro *(Risk-based prioritization)*:** thứ tự thực thi kiểm thử dựa trên kết quả của việc phân tích rủi ro (xem mục 5.2.3). Các ca kiểm thử bao phủ các rủi ro quan trọng nhất sẽ được thực thi

trước.

2. **Sắp xếp độ ưu tiên dựa trên độ bao phủ** ***(Coverage-based prioritization)*****:** thứ tự thực thi kiểm thử dựa trên độ bao phủ (ví dụ: độ bao phủ câu lệnh). Các ca kiểm thử đạt được độ bao phủ cao nhất được thực thi trước. Trong một biến thể khác, được gọi là **sắp xếp độ ưu tiên độ bao phủ bổ sung** ***(additional coverage prioritization)***, ca kiểm thử đạt độ bao phủ cao nhất được thực thi đầu tiên; mỗi ca kiểm thử tiếp theo là ca kiểm thử đạt được **độ bao phủ tăng thêm** ***(additional coverage)*** cao nhất so với những gì đã được bao phủ trước đó.
3. **Sắp xếp độ ưu tiên dựa trên yêu cầu** ***(Requirements-based prioritization)*****:** thứ tự thực thi kiểm thử dựa trên độ ưu tiên của các yêu cầu được truy vết ngược về các ca kiểm thử tương ứng. Độ ưu tiên của yêu cầu được xác định bởi các bên liên quan. Các ca kiểm thử liên quan đến các yêu cầu quan trọng nhất được thực thi trước.

Lý tưởng nhất, các ca kiểm thử sẽ được sắp xếp để chạy dựa trên mức độ ưu tiên của chúng. Tuy nhiên, thực hành này có thể không hoạt động nếu các ca kiểm thử hoặc các tính năng đang được kiểm thử có **sự phụ thuộc lẫn nhau** ***(dependencies)***. **Nếu một ca kiểm thử có độ ưu tiên cao hơn lại phụ thuộc vào một ca kiểm thử có độ ưu tiên thấp hơn, thì ca kiểm thử có độ ưu tiên thấp hơn đó PHẢI được thực thi trước.**

Thứ tự thực thi kiểm thử cũng phải tính đến **sự sẵn có của các nguồn lực** ***(availability of resources)*** (ví dụ: công cụ kiểm thử, môi trường kiểm thử hoặc nhân sự chỉ sẵn sàng trong một khung thời gian cụ thể).

5.1.6. Kim tự tháp Kiểm thử (Test Pyramid) FL-5.1.6 • K1

Kim tự tháp kiểm thử ***(Test pyramid)*** là một mô hình cho thấy các bài kiểm thử khác nhau có thể có **độ mịn/chi tiết** ***(granularity)*** khác nhau. Mô hình kim tự tháp kiểm thử hỗ trợ nhóm trong việc tự động hóa kiểm thử và phân bổ công sức kiểm thử bằng cách chỉ ra rằng các mục tiêu kiểm thử khác nhau được hỗ trợ bởi các mức độ tự động hóa kiểm thử khác nhau. Các tầng của kim tự tháp đại diện cho các nhóm bài kiểm thử:

- **Tầng càng lên cao** ***(The higher the layer)*****:** độ mịn kiểm thử càng thấp (***lower test granularity*** - tức là kiểm thử khối lớn hơn), độ cô lập kiểm thử càng thấp (***lower test isolation*** - tức là mức độ phụ thuộc vào các phần tử khác của hệ thống cao hơn) và thời gian thực thi kiểm thử càng cao/chậm hơn ***(higher test execution time)***.
- **Tầng đáy** ***(Bottom layer)*****:** Các bài kiểm thử nhỏ, cô lập, nhanh, và kiểm tra một phần nhỏ chức năng, vì vậy thường cần số lượng lớn chúng để đạt được độ bao phủ hợp lý.
- **Tầng đỉnh** ***(Top layer)*****:** Đại diện cho các bài kiểm thử phức tạp, cấp cao, đầu-cuối ***(end-to-end tests)***. Các bài kiểm thử cấp cao này nhìn chung chậm hơn so với các bài kiểm thử ở các tầng thấp hơn, và chúng thường kiểm tra một phần lớn chức năng, vì vậy thường chỉ cần một số lượng ít chúng để đạt được mức độ bao phủ hợp lý.

Số lượng và cách đặt tên các tầng có thể khác nhau. Ví dụ: mô hình kim tự tháp kiểm thử gốc của Cohn (2009) định nghĩa 3 tầng từ dưới lên: ***"unit tests"***, ***"service tests"*** và ***"UI tests"***. Một mô hình phổ biến khác định nghĩa: ***unit (component) tests***, ***integration (component integration) tests***, và ***end-to-end tests***.

5.1.7. Các Góc phần tư Kiểm thử (Testing Quadrants) FL-5.1.7 • K2

Các góc phần tư kiểm thử ***(Testing quadrants)***, được định nghĩa bởi Brian Marick (Marick 2003, Crispin 2008), nhóm các cấp độ kiểm thử với các loại kiểm thử, hoạt động, kỹ thuật kiểm thử và sản phẩm công việc phù hợp trong phát triển phần mềm Agile. Mô hình này hỗ trợ quản lý kiểm thử trong việc trực quan hóa chúng để đảm bảo rằng tất cả các loại kiểm thử và cấp độ kiểm thử phù hợp đều được đưa vào SDLC.

Trong mô hình này, các bài kiểm thử có thể **hướng nghiệp vụ** *(business facing)* hoặc **hướng công nghệ** *(technology facing)*. Các bài kiểm thử cũng có thể **hỗ trợ nhóm (***support the team* **- tức là định hướng phát triển)** hoặc **phê bình/đánh giá sản phẩm (***critique the product* **- tức là đo lường hành vi của nó so với kỳ vọng)**. Sự kết hợp của hai góc nhìn này xác định **4 góc phần tư**:

- **Góc phần tư Q1 (***Quadrant Q1* **- hướng công nghệ** *technology facing***, hỗ trợ nhóm** *support the team***):** Chứa kiểm thử thành phần *(component tests)* và kiểm thử tích hợp thành phần *(component integration tests)*. Các bài kiểm thử này nên được **tự động hóa** và đưa vào quy trình CI.
- **Góc phần tư Q2 (***Quadrant Q2* **- hướng nghiệp vụ** *business facing***, hỗ trợ nhóm** *support the team***):** Chứa các bài kiểm thử chức năng *(functional tests)*, các ví dụ *(examples)*, kiểm thử *user story*, nguyên mẫu trải nghiệm người dùng *(user experience prototypes)*, kiểm thử API *(API testing)*, và các mô phỏng *(simulations)*. Các bài kiểm thử này kiểm tra các tiêu chí chấp nhận và có thể là **thủ công hoặc tự động**.
- **Góc phần tư Q3 (***Quadrant Q3* **- hướng nghiệp vụ** *business facing***, phê bình sản phẩm** *critique the product***):** Chứa kiểm thử khám phá *(exploratory testing)*, kiểm thử khả năng sử dụng *(usability testing)*, kiểm thử chấp nhận người dùng *(user acceptance testing - UAT)*. Các bài kiểm thử này hướng tới người dùng và **thường là thủ công** *(often manual)*.
- **Góc phần tư Q4 (***Quadrant Q4* **- hướng công nghệ** *technology facing***, phê bình sản phẩm** *critique the product***):** Chứa kiểm thử khói *(smoke tests)* và các bài kiểm thử phi chức năng (*non-functional tests* - **ngoại trừ kiểm thử khả năng sử dụng** *except usability tests* vì usability nằm ở Q3). Các bài kiểm thử này **thường được tự động hóa** *(often automated)*.

5.2. Quản lý Rủi ro (Risk Management)

Các hoạt động quản lý rủi ro chính là:

- **Phân tích rủi ro** *(Risk analysis)***:** bao gồm **nhận diện rủi ro** *(risk identification)* và **đánh giá rủi ro** *(risk assessment)* (xem mục 5.2.3).
- **Kiểm soát rủi ro** *(Risk control)***:** bao gồm **giảm thiểu rủi ro** *(risk mitigation)* và **giám sát rủi ro** *(risk monitoring)* (xem mục 5.2.4).

Cách tiếp cận kiểm thử mà trong đó các hoạt động kiểm thử được lựa chọn, sắp xếp độ ưu tiên và quản lý dựa trên phân tích rủi ro và kiểm soát rủi ro được gọi là **kiểm thử dựa trên rủi ro** *(risk-based testing)*.

5.2.1. Định nghĩa Rủi ro và các Thuộc tính của Rủi ro

FL-5.2.1 • K1

Rủi ro *(Risk)* là một sự kiện, mối nguy, mối đe dọa hoặc tình huống tiềm ẩn mà nếu xảy ra sẽ gây ra một hệ quả bất lợi *(adverse effect)*. Một rủi ro được đặc trưng bởi **hai yếu tố**:

- **Khả năng xảy ra rủi ro** *(Risk likelihood)***:** xác suất xảy ra rủi ro (lớn hơn 0 và nhỏ hơn 1: **0 < likelihood < 1**).
- **Mức độ tác động của rủi ro (***Risk impact* **/** *harm***):** hậu quả của việc rủi ro đó xảy ra.

Hai yếu tố này biểu thị **mức độ rủi ro** *(risk level)*, là thước đo cho rủi ro. Mức độ rủi ro càng cao thì việc xử lý nó càng quan trọng.

5.2.2. Rủi ro Dự án và Rủi ro Sản phẩm

FL-5.2.2 • K2

Trong kiểm thử phần mềm, người ta thường quan tâm đến hai loại rủi ro: **rủi ro dự án** *(project risks)* và **rủi ro sản phẩm** *(product risks)*.

1. **Rủi ro dự án** *(Project risks)*: Liên quan đến việc quản lý và kiểm soát dự án. Rủi ro dự án bao gồm:
    - **Vấn đề tổ chức** *(Organizational issues)*: ví dụ: chậm trễ trong việc bàn giao sản phẩm công việc, ước lượng không chính xác, cắt giảm chi phí.
    - **Vấn đề con người** *(People issues)*: ví dụ: kỹ năng không đủ, xung đột, vấn đề giao tiếp, thiếu hụt nhân sự.
    - **Vấn đề kỹ thuật** *(Technical issues)*: ví dụ: phình phạm vi yêu cầu *(scope creep)*, hỗ trợ công cụ kém.
    - **Vấn đề nhà cung cấp** *(Supplier issues)*: ví dụ: bên thứ ba giao hàng thất bại, công ty hỗ trợ bị phá sản.

    Khi xảy ra, rủi ro dự án có thể tác động đến lịch trình, ngân sách hoặc phạm vi của dự án, làm ảnh hưởng đến khả năng đạt được các mục tiêu của dự án.
2. **Rủi ro sản phẩm** *(Product risks)*: Liên quan đến các đặc tính chất lượng của sản phẩm (ví dụ: được mô tả trong mô hình chất lượng ISO 25010). Ví dụ về rủi ro sản phẩm bao gồm: thiếu hoặc sai chức năng, tính toán sai, lỗi thời gian chạy *(runtime errors)*, kiến trúc kém, thuật toán kém hiệu quả, thời gian phản hồi không đạt yêu cầu, trải nghiệm người dùng kém, lỗ hổng bảo mật. Khi xảy ra, rủi ro sản phẩm có thể dẫn đến nhiều hậu quả tiêu cực khác nhau, bao gồm:
    - Sự không hài lòng của người dùng *(User dissatisfaction)*
    - Mất doanh thu, uy tín, danh tiếng
    - Thiệt hại cho bên thứ ba
    - Chi phí bảo trì cao, quá tải bộ phận hỗ trợ *(help desk)*
    - Bị phạt hình sự *(Criminal penalties)*
    - Trong trường hợp cực đoan: thiệt hại vật chất, thương tích hoặc thậm chí tử vong.

5.2.3. Phân tích Rủi ro Sản phẩm (Product Risk Analysis)

FL-5.2.3 • K2

Từ góc độ kiểm thử, mục tiêu của phân tích rủi ro sản phẩm là cung cấp nhận thức về rủi ro sản phẩm nhằm tập trung công sức kiểm thử theo cách giảm thiểu mức độ rủi ro sản phẩm còn lại *(residual level of product risk)*. Lý tưởng nhất, phân tích rủi ro sản phẩm bắt đầu sớm trong SDLC.

Phân tích rủi ro sản phẩm bao gồm **nhận diện rủi ro** *(risk identification)* và **đánh giá rủi ro** *(risk assessment)*:

- **Nhận diện rủi ro** là việc tạo ra một danh sách toàn diện các rủi ro bằng các kỹ thuật như động não *(brainstorming)*, hội thảo *(workshops)*, phỏng vấn *(interviews)*, hoặc biểu đồ nhân quả *(cause-effect diagrams)*.
- **Đánh giá rủi ro** bao gồm: phân loại các rủi ro đã xác định, xác định khả năng xảy ra *(likelihood)*, mức độ tác động *(impact)* và mức độ rủi ro *(risk level)*, sắp xếp độ ưu tiên, và đề xuất các cách xử lý chúng. Trong cách tiếp cận định lượng *(quantitative)*, mức độ rủi ro được tính bằng phép nhân giữa ***risk likelihood*** và ***risk impact***. Trong cách tiếp cận định tính *(qualitative)*, mức độ rủi ro có thể được xác định bằng ma trận rủi ro *(risk matrix)*.

Kết quả phân tích rủi ro sản phẩm ảnh hưởng đến mức độ kỹ lưỡng *(thoroughness)* và phạm vi kiểm thử *(test scope)*, được dùng để:

- Xác định phạm vi kiểm thử cần thực hiện
- Xác định các cấp độ kiểm thử cụ thể và đề xuất các loại kiểm thử cần thực hiện
- Xác định các kỹ thuật kiểm thử cần áp dụng và mức độ bao phủ cần đạt được
- Ước lượng công sức kiểm thử cần thiết cho mỗi nhiệm vụ
- Sắp xếp độ ưu tiên kiểm thử nhằm cố gắng tìm ra các khiếm khuyết nghiêm trọng càng sớm càng tốt
- Xác định xem có bất kỳ hoạt động nào ngoài kiểm thử có thể được sử dụng để giảm rủi ro hay không.

5.2.4. Kiểm soát Rủi ro Sản phẩm (Product Risk Control) FL-5.2.4 • K2

Kiểm soát rủi ro sản phẩm bao gồm tất cả các biện pháp được thực hiện để ứng phó với các rủi ro sản phẩm đã được xác định và đánh giá. Kiểm soát rủi ro sản phẩm bao gồm **giảm thiểu rủi ro *(risk mitigation)*** và **giám sát rủi ro *(risk monitoring)***.

Một khi rủi ro đã được phân tích, có một số tùy chọn ứng phó với rủi ro: **giảm thiểu rủi ro bằng kiểm thử *(risk mitigation by testing)***, **chấp nhận rủi ro *(risk acceptance)***, **chuyển giao rủi ro *(risk transfer)***, hoặc **kế hoạch dự phòng *(contingency plan)***.

Các hành động có thể thực hiện để giảm thiểu rủi ro sản phẩm bằng kiểm thử là:

- Chọn các kiểm thử viên có mức độ kinh nghiệm và kỹ năng phù hợp với loại rủi ro đó
- Áp dụng một mức độ độc lập kiểm thử phù hợp
- Thực hiện các buổi duyệt *(reviews)* và phân tích tĩnh *(static analysis)*
- Áp dụng các kỹ thuật kiểm thử và mức độ bao phủ phù hợp
- Áp dụng các loại kiểm thử phù hợp giải quyết các đặc tính chất lượng bị ảnh hưởng
- Thực hiện kiểm thử động, bao gồm cả kiểm thử hồi quy.

5.3. Giám sát Kiểm thử, Kiểm soát Kiểm thử và Hoàn tất Kiểm thử

- **Giám sát kiểm thử *(Test monitoring)*:** Thu thập thông tin về kiểm thử để đánh giá tiến độ kiểm thử và đo lường xem các tiêu chí kết thúc đã được thỏa mãn hay chưa.
- **Kiểm soát kiểm thử *(Test control)*:** Sử dụng thông tin từ giám sát kiểm thử để đưa ra, dưới dạng các chỉ thị kiểm soát *(control directives)*, sự hướng dẫn và các hành động khắc phục cần thiết. **Ví dụ về các chỉ thị kiểm soát *(control directives)* bao gồm:**
  - Sắp xếp lại độ ưu tiên các bài kiểm thử khi một rủi ro đã xác định biến thành vấn đề thực tế *(issue)*.
  - Đánh giá lại xem một hạng mục kiểm thử có đáp ứng tiêu chí bắt đầu hoặc kết thúc hay không do phải làm lại *(rework)*.
  - Điều chỉnh lịch trình kiểm thử để giải quyết sự chậm trễ trong việc bàn giao môi trường kiểm thử.
  - Bổ sung thêm nguồn lực mới khi cần và ở nơi cần thiết.

- **Hoàn tất kiểm thử *(Test completion)*:** Thu thập dữ liệu từ các hoạt động kiểm thử đã hoàn thành để củng cố kinh nghiệm, ***testware*** và các thông tin liên quan khác; diễn ra tại các cột mốc dự án (hoàn thành một cấp độ kiểm thử, kết thúc một vòng lặp Agile, hoàn thành/hủy bỏ dự án kiểm thử, phát hành hệ thống, hoặc hoàn thành một đợt phát hành bảo trì).

5.3.1. Các Số liệu Đo lường được sử dụng trong Kiểm thử (Metrics used in Testing)

FL-5.3.1 • K1

Các số liệu kiểm thử *(Test metrics)* phổ biến bao gồm **7 nhóm**:

1. **Số liệu tiến độ dự án *(Project progress metrics)*:** ví dụ: mức độ hoàn thành công việc *(task completion)*, mức sử dụng nguồn lực, công sức kiểm thử *(test effort)*.
2. **Số liệu tiến độ kiểm thử *(Test progress metrics)*:** ví dụ: tiến độ triển khai ca kiểm thử, tiến độ chuẩn bị môi trường kiểm thử, số lượng ca kiểm thử đã chạy/chưa chạy *(run/not run)*, đạt/trượt *(passed/failed)*, thời gian thực thi kiểm thử.
3. **Số liệu chất lượng sản phẩm *(Product quality metrics)*:** ví dụ: tính sẵn sàng *(availability)*, thời gian phản hồi *(response time)*, thời gian trung bình đến khi hỏng *(mean time to failure - MTTF)*.
4. **Số liệu khiếm khuyết *(Defect metrics)*:** ví dụ: số lượng và độ ưu tiên của các khiếm khuyết tìm thấy/đã sửa, mật độ khiếm khuyết *(defect density)*, tỷ lệ phần trăm phát hiện khiếm khuyết *(defect detection percentage - DDP)*.
5. **Số liệu rủi ro *(Risk metrics)*:** ví dụ: mức độ rủi ro còn lại *(residual risk level)*.
6. **Số liệu độ bao phủ *(Coverage metrics)*:** ví dụ: độ bao phủ yêu cầu *(requirements coverage)*, độ bao phủ mã nguồn *(code coverage)*.
7. **Số liệu chi phí *(Cost metrics)*:** ví dụ: chi phí kiểm thử *(cost of testing)*, chi phí chất lượng của tổ chức *(organizational cost of quality)*.

5.3.2. Mục đích, Nội dung và Độc giả của Báo cáo Kiểm thử

FL-5.3.2 • K2

Có hai loại báo cáo chính:

1. **Báo cáo tiến độ kiểm thử (*Test progress reports* - còn gọi là *test status reports*):** Được tạo định kỳ (hàng ngày, hàng tuần...) để hỗ trợ kiểm soát kiểm thử liên tục, bao gồm:
    - Kỳ kiểm thử *(Testing period)*
    - Tiến độ kiểm thử (vượt hay chậm tiến độ), bao gồm các sai lệch đáng chú ý
    - Các trở ngại đối với kiểm thử *(Impediments for testing)*, và các cách khắc phục tạm thời *(workarounds)*
    - Các số liệu kiểm thử *(Test metrics)*
    - Các rủi ro mới và rủi ro thay đổi trong kỳ kiểm thử
    - Công việc kiểm thử dự kiến cho kỳ tiếp theo *(Testing planned for the next period)*.
2. **Báo cáo hoàn tất kiểm thử *(Test completion report)*:** Được chuẩn bị một lần khi một dự án, cấp độ kiểm thử hoặc loại kiểm thử hoàn tất và đạt tiêu chí kết thúc, bao gồm:
    - Tóm tắt kiểm thử *(Test summary)*

- Đánh giá kiểm thử và chất lượng sản phẩm dựa trên kế hoạch kiểm thử ban đầu (mục tiêu và tiêu chí kết thúc)
- Các sai lệch so với kế hoạch kiểm thử (lịch trình, thời lượng, công sức)
- Các trở ngại kiểm thử và cách khắc phục tạm thời
- Các số liệu kiểm thử dựa trên các báo cáo tiến độ
- Các rủi ro chưa được giảm thiểu *(Unmitigated risks)*, các khiếm khuyết chưa được sửa *(defects not fixed)*
- Các bài học kinh nghiệm *(Lessons learned)* liên quan đến kiểm thử.

5.3.3. Truyền đạt Trạng thái của Kiểm thử

FL-5.3.3 • K2

Các tùy chọn truyền đạt trạng thái kiểm thử bao gồm:

- Giao tiếp bằng lời nói *(Verbal communication)* với các thành viên trong nhóm và các bên liên quan khác
- Bảng thông tin (*Dashboards*, ví dụ: bảng CI/CD, bảng công việc *task boards*, và biểu đồ *burn-down charts*)
- Kênh giao tiếp điện tử (ví dụ: email, chat)
- Tài liệu trực tuyến *(Online documentation)*
- Các báo cáo kiểm thử chính thức *(Formal test reports)*.

5.4. Quản lý Cấu hình (Configuration Management - CM)

FL-5.4.1 • K2

Trong kiểm thử, quản lý cấu hình *(Configuration management - CM)* cung cấp một kỷ luật để xác định, kiểm soát và theo dõi các sản phẩm công việc (như kế hoạch kiểm thử, chiến lược kiểm thử, điều kiện kiểm thử, ca kiểm thử, kịch bản kiểm thử, kết quả kiểm thử, nhật ký kiểm thử và báo cáo kiểm thử) như là các **hạng mục cấu hình** *(configuration items)*.

Nếu hạng mục cấu hình được phê duyệt để kiểm thử, nó sẽ trở thành một **đường cơ sở** *(baseline)* và chỉ có thể được thay đổi thông qua một quy trình kiểm soát thay đổi chính thức *(formal change control process)*. Luôn có thể quay trở lại *(revert)* một *baseline* trước đó để tái hiện lại các kết quả kiểm thử trước đó.

Để hỗ trợ kiểm thử đúng cách, CM đảm bảo:

- Tất cả các hạng mục cấu hình, bao gồm cả các hạng mục kiểm thử *(test items)*, đều được định danh duy nhất *(uniquely identified)*, kiểm soát phiên bản *(version controlled)*, theo dõi thay đổi *(tracked for changes)*, và liên kết với các hạng mục cấu hình khác để duy trì khả năng truy vết *(traceability)*.
- Tất cả các tài liệu và hạng mục phần mềm đã xác định đều được tham chiếu một cách rõ ràng, không mơ hồ trong *testware*.

5.5. Quản lý Khiếm khuyết (Defect Management)

FL-5.5.1 • K3

Tối thiểu, quy trình quản lý khiếm khuyết bao gồm một luồng công việc *(workflow)* để xử lý các khiếm khuyết hoặc điểm bất thường riêng lẻ từ khi phát hiện cho đến khi đóng lại và các quy tắc để phân loại chúng (ghi nhật ký *log* -> phân tích và phân loại *analyze and classify* -> quyết định cách phản hồi như sửa hay giữ nguyên -> đóng báo cáo *close*).

Các báo cáo khiếm khuyết *(defect reports)* **điển hình có các mục tiêu sau:**

- Cung cấp cho những người chịu trách nhiệm xử lý và giải quyết các khiếm khuyết được báo cáo đầy đủ thông tin để giải quyết vấn đề.
- Cung cấp một phương tiện để theo dõi chất lượng của sản phẩm công việc.
- Cung cấp các ý tưởng để cải tiến quy trình phát triển và kiểm thử.

Một báo cáo khiếm khuyết được ghi lại trong kiểm thử động thường bao gồm các trường sau (cực kỳ quan trọng cho câu hỏi tình huống K3):

- **Định danh duy nhất** ***(Unique identifier)***
- **Tiêu đề** ***(Title)*** kèm tóm tắt ngắn gọn về điểm bất thường đang được báo cáo
- **Ngày** ***(Date)*** quan sát thấy điểm bất thường, tổ chức phát hành, và tác giả ***(author)***, bao gồm cả vai trò của họ
- **Định danh của đối tượng kiểm thử** ***(Identification of the test object)*** (ví dụ: phiên bản phần mềm) và **môi trường kiểm thử** ***(test environment)***
- **Ngữ cảnh của khiếm khuyết** ***(Context of the defect)*** (ví dụ: ca kiểm thử đang chạy, hoạt động kiểm thử đang thực hiện, giai đoạn SDLC, kỹ thuật kiểm thử, danh sách kiểm tra hoặc dữ liệu kiểm thử đang dùng)
- **Mô tả hỏng hóc** ***(Description of the failure)*** để cho phép tái hiện và giải quyết, bao gồm **các bước kiểm thử** ***(test steps)*** đã phát hiện ra điểm bất thường, và bất kỳ nhật ký kiểm thử, bản xuất cơ sở dữ liệu ***(database dumps)***, ảnh chụp màn hình ***(screenshots)*** hoặc bản ghi hình nào liên quan
- **Kết quả mong đợi** ***(Expected results)*** **và kết quả thực tế** ***(Actual results)***
- **Mức độ nghiêm trọng của khiếm khuyết** ***(Severity of the defect)*** (mức độ tác động) đối với lợi ích của các bên liên quan hoặc các yêu cầu
- **Độ ưu tiên sửa** ***(Priority to fix)***
- **Trạng thái của khiếm khuyết** ***(Status of the defect)*** (ví dụ: đang mở ***open***, hoãn lại ***deferred***, trùng lặp ***duplicate***, chờ sửa ***waiting to be fixed***, chờ kiểm thử xác nhận ***awaiting confirmation testing***, mở lại ***re-opened***, đã đóng ***closed***, bị từ chối ***rejected***)
- **Các tham chiếu** ***(References)*** (ví dụ: tới ca kiểm thử tương ứng).

CHƯƠNG 6

🕑 Thời lượng phân bổ: 20 phút

Công cụ Kiểm thử (Test Tools)

K1 **Bảng Từ Khóa Chuẩn ISTQB (Keywords)** — Cần nhớ chính xác thuật ngữ gốc Tiếng Anh khi làm bài thi

test automation	tự động hóa kiểm thử

Mục Tiêu Học Tập Trọng Tâm (Learning Objectives)

MÃ LO	MỨC NHẬN THỨC	NỘI DUNG YÊU CẦU TRONG ĐỀ THI
6.1 Công cụ Hỗ trợ cho Kiểm thử (Tool Support for Testing)		
FL-6.1.1	K2 · Hiểu	Giải thích cách các loại công cụ kiểm thử khác nhau hỗ trợ cho kiểm thử
6.2 Lợi ích và Rủi ro của Tự động hóa Kiểm thử (Benefits and Risks of Test Automation)		
FL-6.2.1	K1 · Ghi nhớ	Nhớ lại các lợi ích và rủi ro của tự động hóa kiểm thử

6.1. Công cụ Hỗ trợ cho Kiểm thử (Tool Support for Testing)

FL-6.1.1 • K2

Các công cụ kiểm thử hỗ trợ và tạo điều kiện thuận lợi cho nhiều hoạt động kiểm thử. Ví dụ bao gồm:

- **Công cụ quản lý kiểm thử** *(Test management tools)*: tăng hiệu suất tiến trình kiểm thử bằng cách tạo thuận lợi cho việc quản lý SDLC, yêu cầu, bài kiểm thử, khiếm khuyết, cấu hình.
- **Công cụ kiểm thử tĩnh** *(Static testing tools)*: hỗ trợ kiểm thử viên thực hiện việc duyệt *(reviews)* và phân tích tĩnh *(static analysis)*.
- **Công cụ thiết kế kiểm thử và triển khai kiểm thử** *(Test design and test implementation tools)*: tạo thuận lợi cho việc sinh ra các ca kiểm thử, dữ liệu kiểm thử và thủ tục kiểm thử.
- **Công cụ thực thi kiểm thử và độ bao phủ kiểm thử** *(Test execution and test coverage tools)*: tạo thuận lợi cho việc thực thi kiểm thử tự động và đo lường độ bao phủ.
- **Công cụ kiểm thử phi chức năng** *(Non-functional testing tools)*: cho phép kiểm thử viên thực hiện kiểm thử phi chức năng vốn khó hoặc không thể thực hiện thủ công (như kiểm thử tải, hiệu năng, bảo mật).
- **Công cụ DevOps** *(DevOps tools)*: hỗ trợ đường ống chuyển giao DevOps, theo dõi luồng công việc, quy trình dựng tự động *(automated build processes)*, CI/CD.
- **Công cụ cộng tác** *(Collaboration tools)*: tạo thuận lợi cho việc giao tiếp.
- **Công cụ hỗ trợ khả năng mở rộng và chuẩn hóa triển khai** *(Tools supporting scalability and deployment standardization)*: ví dụ: máy ảo *(virtual machines)*, công cụ container hóa *(containerization tools)*.
- **Bất kỳ công cụ nào khác hỗ trợ trong kiểm thử:** ví dụ: một bảng tính (*spreadsheet* như Excel) cũng là một công cụ kiểm thử trong ngữ cảnh kiểm thử.

6.2. Lợi ích và Rủi ro của Tự động hóa Kiểm thử (Benefits and Risks of Test Automation)

FL-6.2.1 • K1

Việc chỉ đơn thuần mua/sở hữu một công cụ không đảm bảo sự thành công. Mỗi công cụ mới sẽ đòi hỏi công sức để đạt được những lợi ích thực sự và lâu dài (ví dụ: giới thiệu công cụ, bảo trì và đào tạo).

Các lợi ích tiềm năng *(Potential benefits)* của việc sử dụng tự động hóa kiểm thử bao gồm:

- **Tiết kiệm thời gian *(Time saved)*** bằng cách giảm các công việc thủ công lặp đi lặp lại (ví dụ: chạy kiểm thử hồi quy, nhập lại cùng dữ liệu kiểm thử, so sánh kết quả mong đợi với kết quả thực tế, kiểm tra tiêu chuẩn viết mã).
- **Ngăn ngừa các sai sót đơn giản của con người *(Prevention of simple human errors)*** thông qua tính nhất quán và khả năng lặp lại cao hơn ***(greater consistency and repeatability)***.
- **Đánh giá khách quan hơn *(More objective assessment)*** (ví dụ: độ bao phủ) và cung cấp các thước đo quá phức tạp để con người tự xác định.
- **Tiếp cận thông tin về kiểm thử dễ dàng hơn** để hỗ trợ quản lý kiểm thử và báo cáo kiểm thử (ví dụ: thống kê, biểu đồ, dữ liệu tổng hợp về tiến độ kiểm thử, tỷ lệ hỏng hóc ***failure rates***, thời lượng thực thi).
- **Giảm thời gian thực thi kiểm thử** để phát hiện khiếm khuyết sớm hơn, phản hồi nhanh hơn và đưa sản phẩm ra thị trường nhanh hơn ***(faster time to market)***.
- **Có thêm thời gian cho kiểm thử viên** để thiết kế các bài kiểm thử mới, sâu hơn và hiệu quả hơn.

Các rủi ro tiềm ẩn *(Potential risks)* của việc sử dụng tự động hóa kiểm thử bao gồm:

- Kỳ vọng phi thực tế ***(Unrealistic expectations)*** về lợi ích của công cụ (bao gồm chức năng và tính dễ sử dụng).
- Ước lượng không chính xác về thời gian, chi phí, công sức cần thiết để đưa công cụ vào, bảo trì các kịch bản kiểm thử ***(maintain test scripts)*** và thay đổi quy trình kiểm thử thủ công hiện tại.
- Sử dụng công cụ kiểm thử khi kiểm thử thủ công phù hợp hơn.
- Phụ thuộc vào công cụ quá nhiều ***(Relying on a tool too much)***, ví dụ: bỏ qua nhu cầu về tư duy phản biện của con người ***(human critical thinking)***.
- Sự phụ thuộc vào nhà cung cấp công cụ ***(tool vendor)*** vốn có thể phá sản, ngừng phát triển công cụ, bán công cụ cho nhà cung cấp khác hoặc hỗ trợ kém.
- Sử dụng phần mềm mã nguồn mở ***(open-source software)*** có thể bị bỏ rơi ***(abandoned)***, hoặc các thành phần nội bộ của nó đòi hỏi phải cập nhật quá thường xuyên.
- Công cụ tự động hóa không tương thích ***(not compatible)*** với nền tảng phát triển.
- Chọn một công cụ không phù hợp, không tuân thủ các yêu cầu quy định chế tài và/hoặc các tiêu chuẩn an toàn.